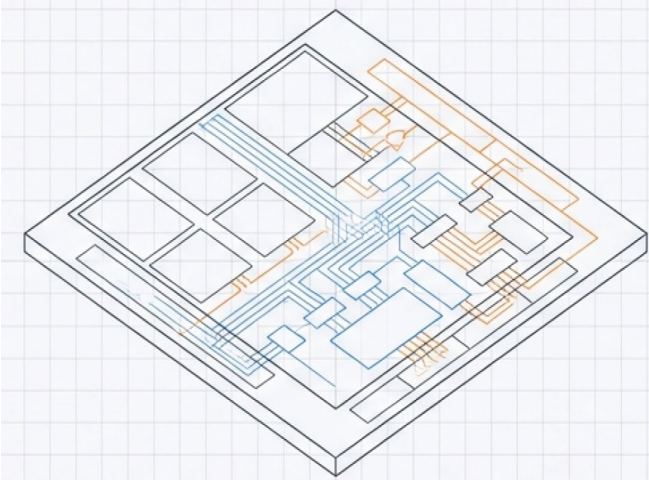




第8章 中央处理器

--非流水线CPU



彻底打通软硬件交界面
搭建现代计算机的核心

吴海军
计算机学院



主要内容

- 第一讲 CPU概述
- 第二讲 单周期CPU设计
- 第三讲 多周期CPU设计
- 第四讲 流水线CPU设计
- 第五讲 流水线冒险及其处理
- 第六讲 高级流水线技术



1 CPU概述

- CPU的基本功能
- CPU的基本组成
 - 操作元件（组合逻辑元件）
 - 状态 / 存储元件（时序逻辑元件）
- 数据通路与时序控制
- 计算机性能与CPU时间



1.1 CPU的基本功能

- CPU的基本职能是**自动执行**程序。
- 程序由**指令**和**数据**组成。
 - **指令存储器 (IM)** : 存放指令的存储器;
 - **数据存储器 (DM)** : 存放数据的存储器;
- 指令周期: CPU取出并执行完一条指令的时间称为**指令周期**, 不同指令的指令周期可能不同。
- CPU在执行指令过程中要能够发现并处理**异常情况**和**中断请求**。
 - 异常是在**CPU内部**发生的
 - 中断是由**外部事件**引起的



1.1 CPU的基本功能

■ CPU执行指令的过程

- 取指令
- $PC + 1$ 送PC
- 指令译码
- 进行主存地址运算
- 取操作数
- 进行算术 / 逻辑运算
- 存结果
- 以上每步都需检测异常
- 若有异常，则自动切换到异常处理程序
- 检测是否有“中断”请求，有则转中断处理程序

取指
阶段

译码
和
执行
阶段

指令
执行
过程

思考问题：

“取指令”一定在最开始做吗？

“ $PC + 1$ ”一定在译码前做吗？

“译码”须在指令执行前做吗？

说出几种“异常”事件？

哪些步骤是所有指令都相同的？



1.1 CPU的基本功能

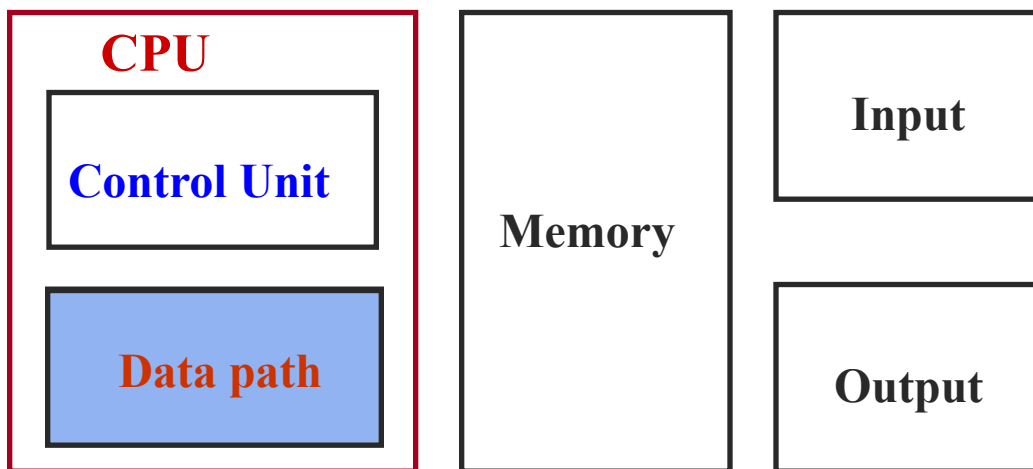
- 每条指令的**功能**总是由以下**基本操作的组合**来实现：
 - **读取**主存储器某个**存储单元**的内容，将其**装入**某个**寄存器**（**取指令**，**取数据**）
 - 把一个数据从某个**寄存器****存入**主存储器某个给定的**存储单元**中（**存结果**）
 - 把一个数据从某个寄存器**送到**另一个寄存器或者ALU（**传输**）
 - 进行算术或逻辑**运算**（**PC+ “1”**，**计算地址**，**运算**）
- 功能操作可以用**形式化的方法**来描述，所用语言称为**寄存器传送语言RTL** (Register Transfer Language)，RTL 语言规定：
 - ◆ $R[r]$ ：表示通用寄存器r的内容；
 - ◆ $M[addr]$ ：表示主存储器addr存储单元的内容；
 - ◆ $M[R[r]]$ ：表示r寄存器里的数据所指向的主存储器的存储单元的内容；
 - ◆ PC：表示程序计数器PC寄存器的内容
 - ◆ $M[PC]$ 表示PC所指向主存储器（**指令存储器**）中存储单元的内容；
 - ◆ **S**EXT[imm]、**Z**EXT[imm]分别表示对imm进行**符号**扩展、**零**扩展；
 - ◆ 传送方向用 “ $\leftarrow$ ” 表示，传送源在右，传送目的在左。

$R[x8] \leftarrow M[R[x9] + \text{SEXT}[imm12]]$ 的含义？



1.2 CPU的基本组成

■ 计算机的组成



■ CPU两大基本组成部分

① 数据通路 Data Path

- ◆ 指令执行过程中，数据所经过的**路径**，包括路径中的部件。
- ◆ 把数据通路中专门进行数据运算的部件称为**执行部件**（execution unit）。

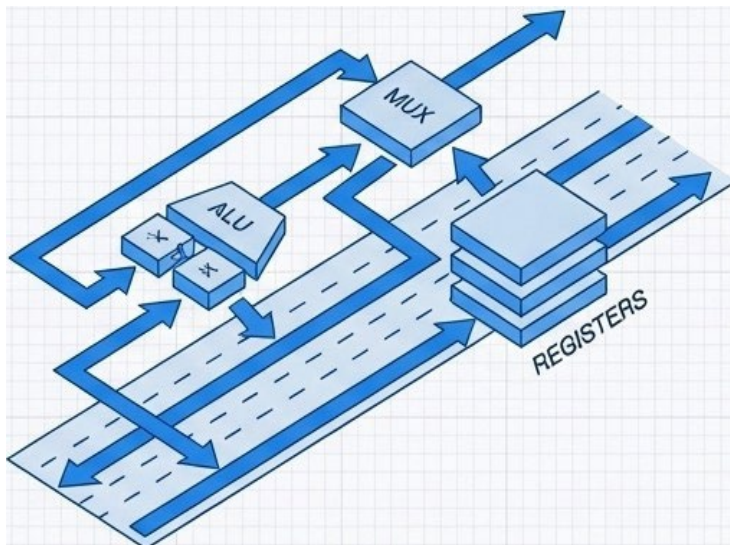
② 控制器 Control Unit

- ◆ 对指令进行译码，生成指令对应的**控制信号**，控制数据通路的动作。
- ◆ 它能对执行部件发出控制信号，是**指令的控制部件**。

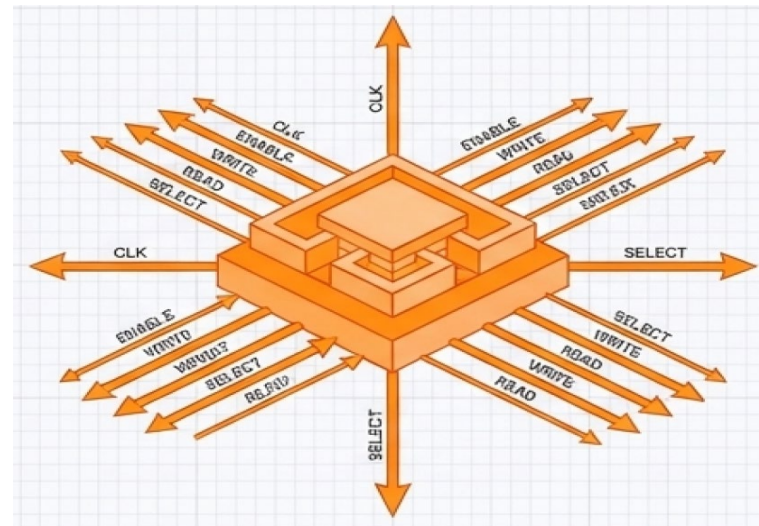


CPU的两大灵魂部件：算力与秩序的统一

■ 数据通路与控制



角色：数据的“高速公路”与加工厂。
功能：承载真实数据存取、传输、算术与逻辑运算。



角色：指挥交通的“大脑”。
功能：解析指令，发出时序控制信号，调度数据通路

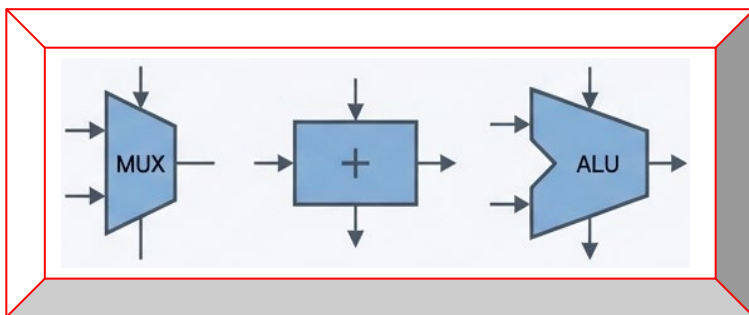
没有控制信号的精准调度，数据通路只是一堆死铁！



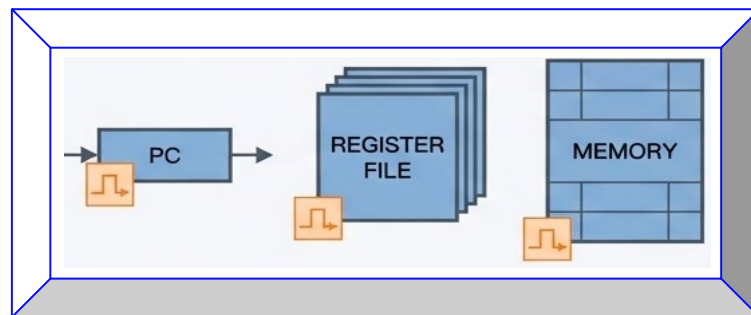
数据通路的物理积木：逻辑与状态

■ 组合逻辑元件+状态元件

组合逻辑元件



状态元件



特性：

- 输出不记历史，只看当下输入。
- 延迟特性：仅受门电路物理传播延迟限制。
- 连接方式：分散或总线。

特性：

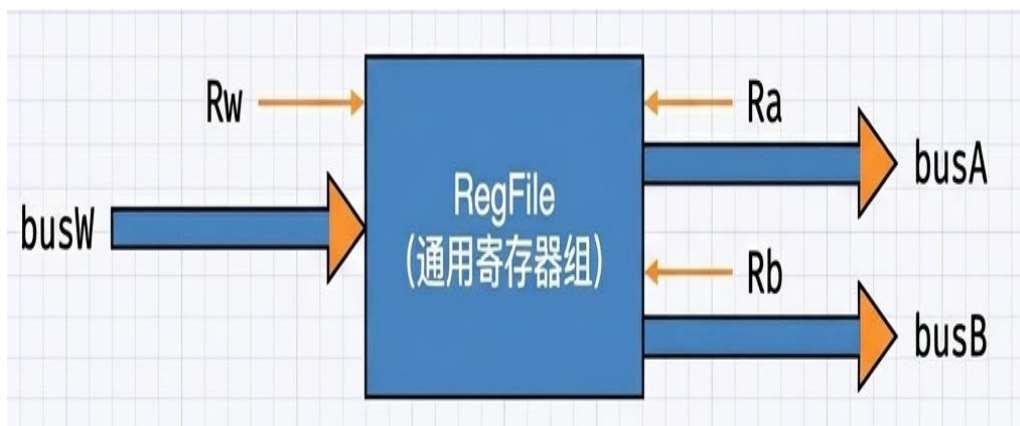
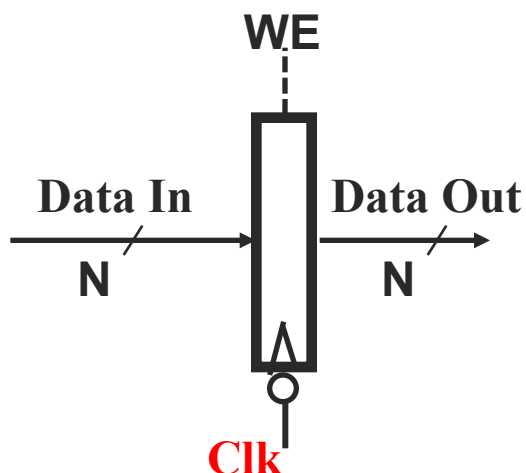
- 时钟边沿驱动，具备状态记忆能力。
- 延迟特性：依赖时钟周期，负责在时间维度上锁存数据。
- 读写方式：通常写入依赖控制信号，输出端**随时可以读出**。



存储架构解析：状态元件的等级

■ 暂存器与通用寄存器组

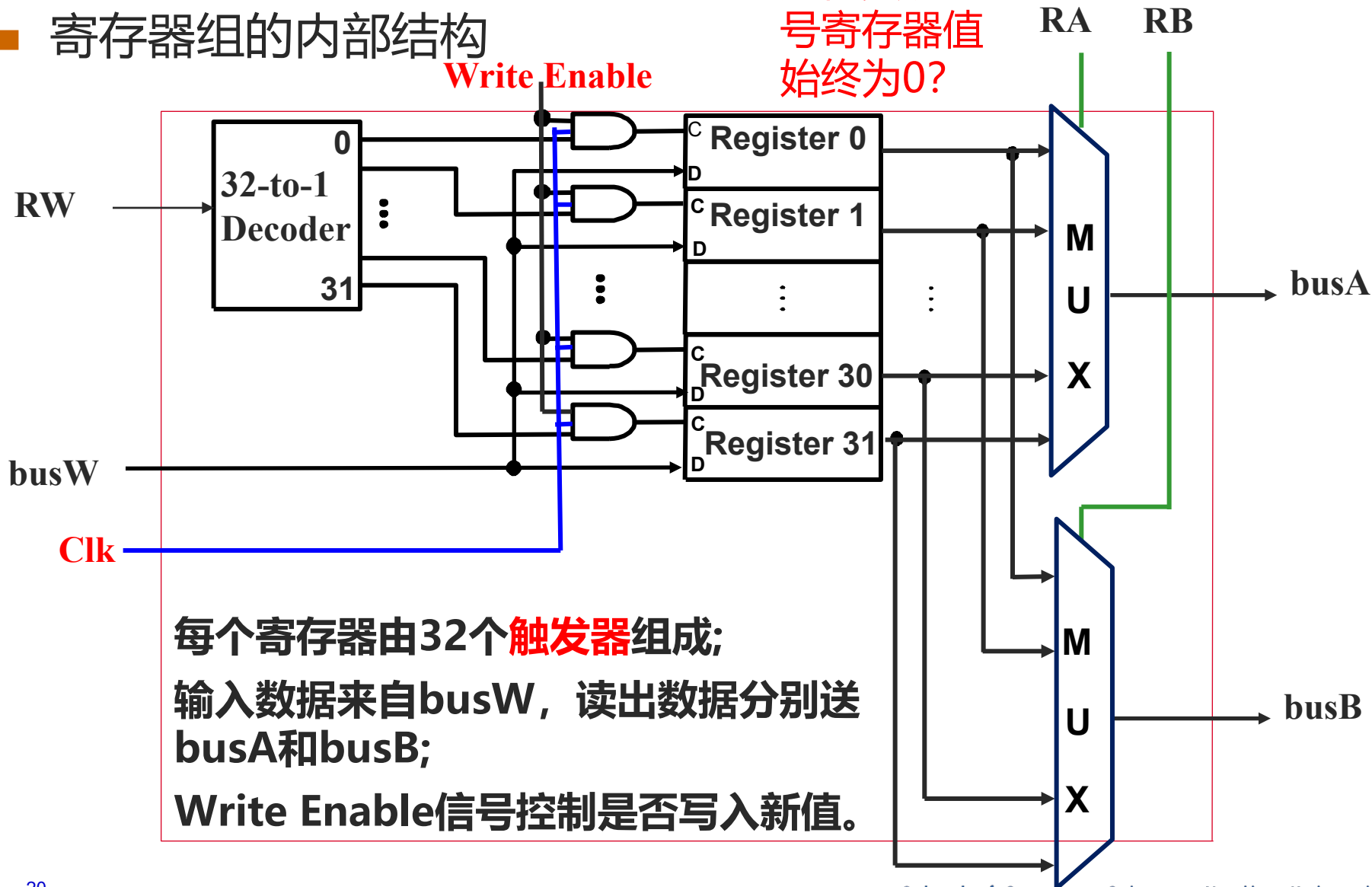
	暂存寄存器	通用寄存器组
端口结构	单入单出（受WE控制）	多端口并发（典型：2读1写）
寻址控制	隐式 / 内部硬连线	显式寻址(Ra, Rb, Rw)
架构可见性	程序员不可见 (仅供微架构过渡)	程序员可见 (如RISC-V的32个通用寄存器)





通用寄存器组/寄存器堆Regfile

■ 寄存器组的内部结构





1.3 时序控制

- 同步系统(Synchronous system)
 - 所有动作有专门时序信号来定时，由时序信号规定何时发出什么动作
- 什么是时序信号？
 - 同步系统中用于进行同步控制的定时信号，如时钟信号
- 什么叫指令周期？
 - 读取并执行一条指令的时间
- 早期计算机的三级时序系统
 - 机器周期 > 节拍 > 脉冲
 - **指令周期**分为取指令、读操作数、执行并写结果等多个基本工作周期，称为**机器周期**。
 - 机器周期有取指令、存储器读、存储器写、中断响应等不同类型的

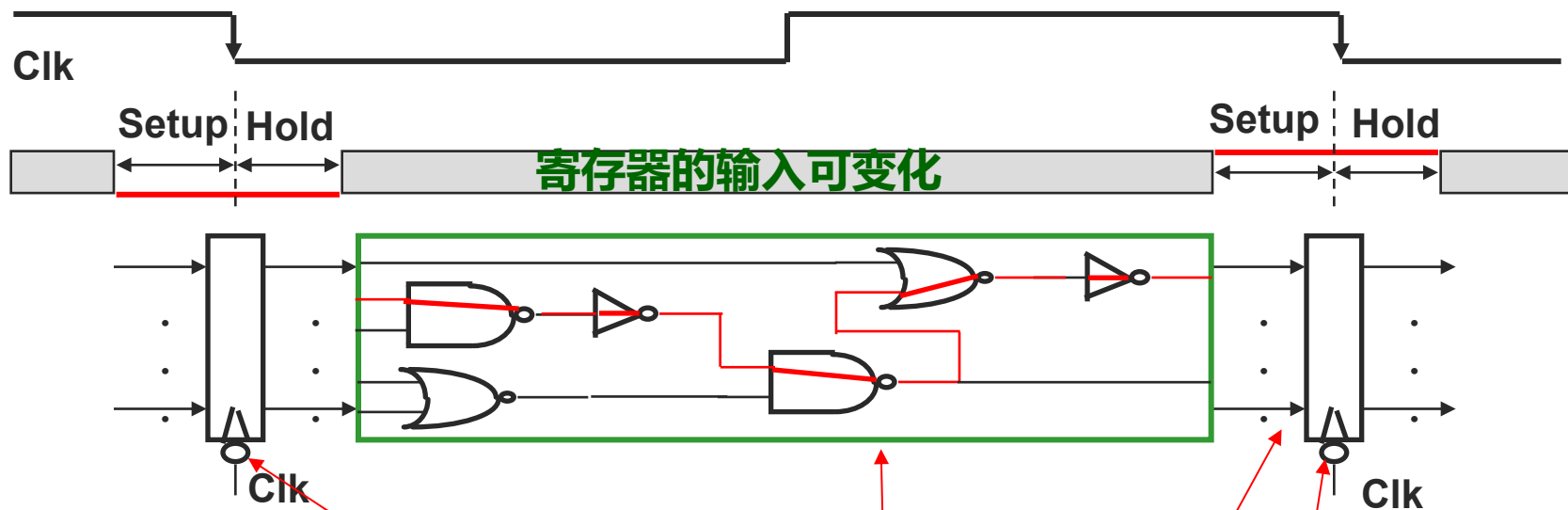
现代计算机已不再采用三级时序系统，机器周期的概念已逐渐消失。

现代计算机的时钟周期，整个数据通路中的定时信号就是时钟，一个时钟周期就是一个节拍。



时间的物理约束：为什么CPU不能无限超频？

- 只有**状态元件**能存储信息，所有**操作元件**都须从状态单元接收输入，并将输出写入状态单元中。
- 状态元件**的输入为前一时钟生成的数据，输出为当前时钟所用的数据。



数据通路由 “... + 状态元件 + 操作元件(组合电路) + 状态元件 + ...” 组成

时钟频率 T_{cycle} 的绝对天花板（上限）：

$$T_{\text{cycle}} \geq t_{\text{clk-to-q}} + t_{\text{comb_delay}} + t_{\text{setup}} + t_{\text{skew}}$$

CPU主频的上限： $1 / T_{\text{cycle}}$ ，由最长的一条物理数据通路（**关键路径**）限制。



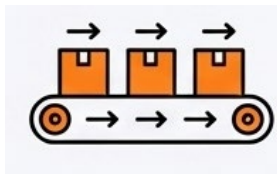
科学定义“性能”：快与多的权衡

■ 衡量计算机系统性能的指标：



响应时间

完成单个任务所需时间。
核心诉求：个人PC、低延迟实时系统。



吞吐率

单位时间内完成的任务总数。
核心诉求：数据库服务器、数据中心。

法则：无论侧重何方，衡量一段代码在特定机器上运行快慢的唯一标准是
用户CPU时间(User CPU Time)。

CPU时间：就是CPU执行程序中每条指令的时间。



1.4 计算机性能与CPU时间

- 由于处理器时间往往被**多个程序共享**使用，因此，用户感觉到的程序执行时间并不是程序真正的执行时间
- 通常把用户感觉到的响应时间分成以下两个时间：
 - **CPU时间**：指CPU真正花在程序执行上的时间。又包括两部分：
 - ◆ **用户CPU时间**：用来运行用户代码的时间
 - ◆ **系统CPU时间**：为了执行用户程序而需要运行操作系统程序的时间
 - **其他时间**：指等待I/O操作完成或CPU花在其他用户程序的时间
- 系统性能和CPU性能**不等价**，有一定的区别
 - 系统性能(System performance)：**系统响应时间**，与CPU外的其他部分也都有关系
 - CPU性能(CPU performance)：用户CPU时间
- 本课程主要讨论CPU性能，即：**CPU真正用在用户程序执行上的时间**
问题：程序CPU时间与系统响应时间哪个更长？



1.4 计算机性能与CPU时间

- 程序的CPU执行时间
 - $= \text{CPU时钟周期数} \times \text{时钟周期}$
 - $= \text{CPU时钟周期数} \div \text{时钟频率}$
- 指令执行的时钟周期数 **CPI: Cycles Per Instruction**
- 程序CPU 执行时间
 - $= \text{指令条数} \times \text{CPI} \times \text{时钟周期}$
- 程序的CPU时钟周期数 $= \text{指令条数} \times \text{CPI}$
- 某个程序平均CPI $= \text{CPU时钟周期数} \div \text{指令条数}$
- CPI 用来衡量以下各方面的综合结果
 - Instruction Set Architecture (ISA)
 - Implementation of that architecture
 - Program (Compiler、Algorithm)



CPU时间评估“铁三角”

例如,
{...
y=4*x;
...}

■ 性能评估 “铁三角” :

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

	指令数目	CPI	时钟周期
Programming	X	X	
Compiler	X	(X)	
Instr. Set Arch.	X	X	
Organization		X	X
Technology			X

计算机性能(程序执行快慢)由三个关键因素决定:指令数目、CPI、时钟周期。

- 指令数目主要由编译器和ISA决定。
- 时钟周期和CPI主要由CPU的设计与实现决定。

问题: 计算机性能与ISA、计算机组织 (Organization)、计算机实现技术 (Technology) 三者的关系是什么?

CPU的设计与实现非常重要!
它直接影响计算机的性能。



1.4 计算机性能与CPU时间

- 对于一个程序或一台机器而言，其CPI是一个平均值，表示该程序或该机器指令集中每条指令执行时平均需要多少时钟周期。

假定 CPI_i 和 C_i 分别为第 i 类指令的CPI和指令条数，则程序的总时钟数为：

$$\text{总时钟数} = \sum_{i=1}^n CPI_i \times C_i \quad \text{所以,} \quad \text{CPU时间} = \text{时钟周期} \times \sum_{i=1}^n CPI_i \times C_i$$

假定 CPI_i 、 F_i 是各指令CPI和在程序中的出现频率，则程序综合CPI为：

$$CPI = \sum_{i=1}^n CPI_i \times F_i \quad \text{where} \quad F_i = \frac{C_i}{\text{Instruction_Count}}$$

已知CPU时间、时钟频率、总时钟数、指令条数，则程序综合CPI为：

$$CPI = (\text{CPU 时间} \times \text{时钟频率}) / \text{指令条数} = \text{总时钟周期数} / \text{指令条数}$$

问题：指令的CPI、机器的CPI、程序的CPI各能反映哪方面的性能？



1.4 计算机性能与CPU时间

程序P在机器A上运行需10s，机器A的时钟频率为400MHz。现在要设计一台机器B，希望该程序在B上运行只需6s.

机器B时钟频率的提高导致了其CPI的增加，使得程序P在机器B上时钟周期数是在机器A上的1.2倍。机器B的时钟频率达到A的多少倍才能使程序P在B上执行速度是A上的 $10/6=1.67$ 倍？

Answer:

CPU时间A = 时钟周期数A / 时钟频率A

时钟周期数A = 10 sec x 400MHz = 4000M个

时钟频率B = 时钟周期数B / CPU时间B

= 1.2 x 4000M / 6 sec = 800 MHz

机器B的频率是A的两倍，但机器B的速度并不是A的两倍！



1.4 计算机性能与CPU时间

- 定点数指令执行速度单位：每秒百万次MIPS
 - MIPS = Instruction Count / Second $\times 1/10^6$
= Clock Rate / CPI $\times 1/10^6$
- 因为每条指令执行时间不同，所以MIPS总是一个平均值。
 - 不同机器的指令集不同
 - 程序由不同的指令混合而成
 - 指令使用的频度动态变化
- 浮点运算的执行速度单位
 - MFLOPS = FP Operations / Second $\times 1/10^6$
 - GFLOPS: 10^9
 - TFLOPS: 10^{12}
 - PFLOPS: 10^{15}

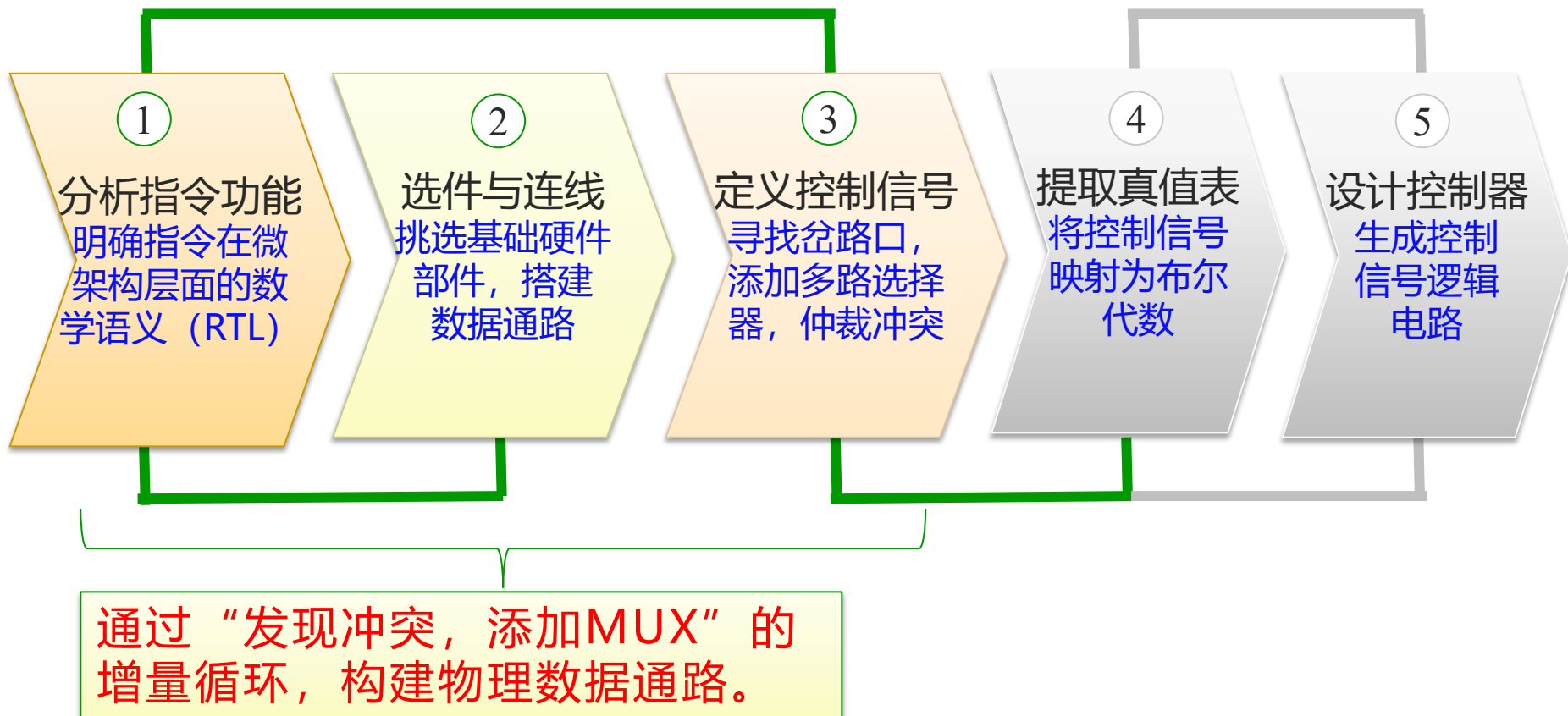


单周期数据通路设计 --从零构建RISC-V核心

增量式架构设计与硬件权衡



单周期数据通路设计的五步法则





用9条指令覆盖一切

- 搭建一个能运行真实机器码的单周期CPU数据通路。

add(R型)
算术加法

slt (R型)
小于置位

sltu(R型)
无符号小于置位

ori (I型)
逻辑或立即数

lw(I型)
内存取数

sw(S型)
内存存数

beq (B型)
相等则分支

lui (U型)
装载高位立即数

jal(J型)
跳转并链接

这9条指令包含了RV32I指令集基本的数据流动模式。
掌握它们，就掌握了体系结构的基石。



指令功能分析

- 第一步都是取指令，**并PC加4**，使PC指向下条指令。表中第二条起都**省略了对第一步的描述**。

- 指令功能RTL描述

- 操作类型

- 操作数的来源

- 寄存器
- 立即数
- 主存储器

冯·诺依曼架构的本质：
一切运算皆源于寄存器 / 内存，最终又归于寄存器 / 内存。

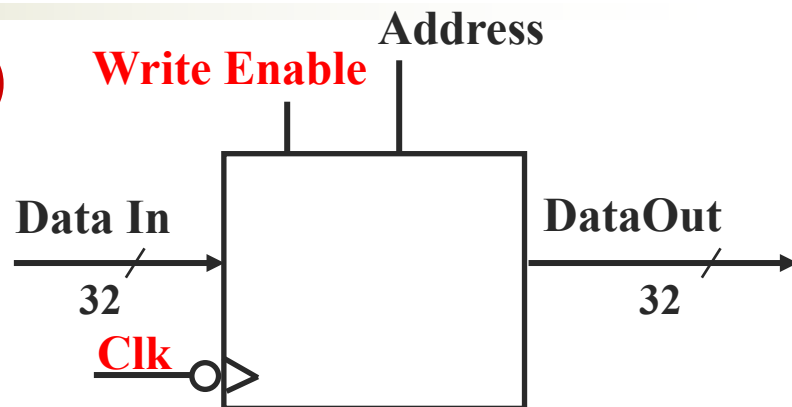
指令格式	功 能
加法： add rd, rs1, rs2	$M[PC], PC \leftarrow PC + 4$ $R[rd] \leftarrow R[rs1] + R[rs2]$
带符号比较： slt rd, rs1, rs2	if ($R[rs1] < R[rs2]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$
无符号数比较： sltu rd, rs1, rs2	if ($R[rs1] < R[rs2]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$
或运算： ori rd, rs1, imm12	$R[rd] \leftarrow R[rs1] \mid \text{SEXT}(\text{imm12})$
读取高位立即数： lui rd, imm20	$R[rd] \leftarrow \text{imm20} \parallel 000H$
读取主存数据： lw rd, imm12(rs1)	$R[rd] \leftarrow M[R[rs1] + \text{SEXT}(\text{imm12})]$
写入数据到主存： sw rs2, imm12(rs1)	$M[R[rs1] + \text{SEXT}(\text{imm12})] \leftarrow R[rs2]$
有条件跳转： beq rs1, rs2, imm12	$\text{Cond} \leftarrow R[rs1] - R[rs2]$ if ($\text{Cond} \text{ eq } 0$) $PC \leftarrow PC + (\text{SEXT}(\text{imm12}) \ll 1)$
无条件跳转： jal rd, imm20	$R[rd] \leftarrow PC + 4$ $PC \leftarrow PC + (\text{SEXT}(\text{imm20}) \ll 1)$



2.2 单周期数据通路的设计

■ 理想存储器 (idealized memory)

- Data Out: 32位读出数据
- Data In: 32位写入数据
- Address: 读写共用一个32位地址



- 读操作 (组合逻辑操作) : 地址Address有效后, 经一个 “取数时间 AccessTime” , Data Out上数据有效。
- 写操作 (时序逻辑操作) : 写使能为1的情况下, 时钟Clk边沿到来时, Data In传来的值开始被写入Address指定的存储单元中。

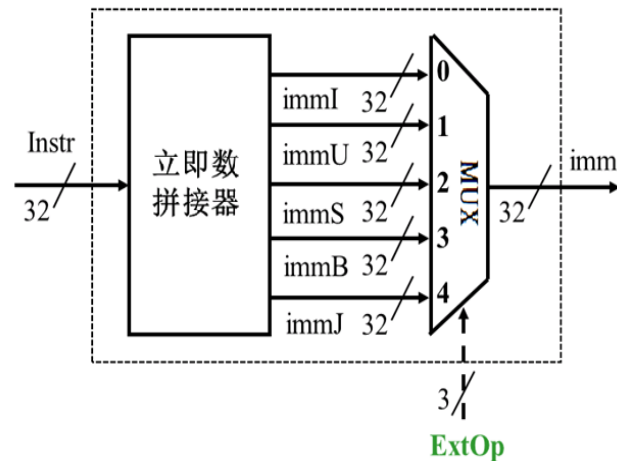
为简化数据通路设计, 假定数据存储器 and 指令存储器 皆为理想存储器。



2.2 单周期数据通路的设计

立即数扩展器的设计

- 除R-型外，其他5类都带有立即数
- 立即数拼接器**：根据指令格式对指令中的立即数进行拼接和**符号位**扩展，形成32位立即数。
- ExtOp**：控制选择和输入指令相匹配的立即操作数输出



	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R	funct7				rs2		rs1		funct3		rd		opcode	
I	imm[11:0]						rs1		funct3		rd		opcode	
S	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
B	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U	imm[31:12]										rd		opcode	
J	imm[20 10:1 11 19:12]										rd		opcode	

assign immI = {20{Instr[31]}, Instr[31:20]};

assign immU = {Instr[31:12], 12'b0};

assign immS = {20{Instr[31]}, Instr[31:25], Instr[11:7]};

assign immB = {19{Instr[31]}, Instr[31], Instr[7], Instr[30:25], Instr[11:8], 1'b0};

assign immJ = {11{Instr[31]}, Instr[31], Instr[19:12], Instr[20], Instr[30:21], 1'b0};

问题：各指令的ExtOp取值如何？



2.2 单周期数据通路的设计

■ 立即数扩展器的设计

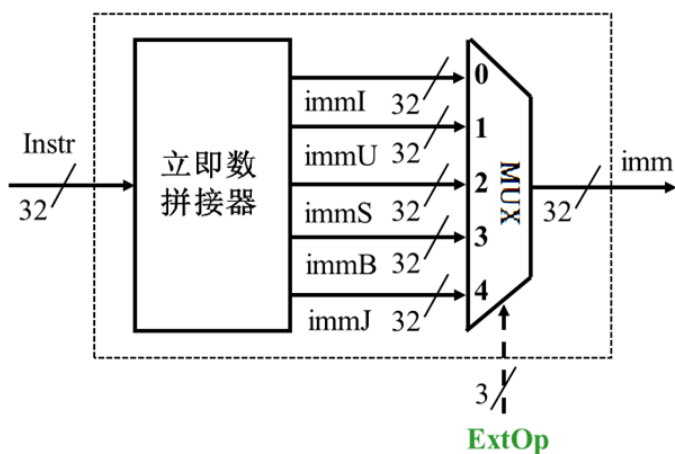
问题：各指令的ExtOp取值如何？
占几位？

有5种情况，至少占3位！

问题：如何根据指令编码Instr
得到ExtOp？

通过指令译码器得到！

将在控制器设计部分介绍。



指 令 ↵	立即数编码类型 ↵	ExtOp<2:0>
add rd, rs1, rs2 ↵	无立即数 ↵	× × × ↵
slt rd, rs1, rs2 ↵		
sltu rd, rs1, rs2 ↵		
ori rd, rs1, imm12 ↵	I-型立即数 (immI)	0 0 0 ↵
lui rd, imm20 ↵	U-型立即数 (immU)	0 0 1 ↵
lw rd, rs1, imm12 ↵	I-型立即数 (immI)	0 0 0 ↵
sw rs1, rs2, imm12 ↵	S-型立即数 (immS)	0 1 0 ↵
beq rs1, rs2, imm12 ↵	B-型立即数 (immB)	0 1 1 ↵
jal rd, imm20 ↵	J-型立即数 (immJ)	1 0 0 ↵



2.2 单周期数据通路的设计

■ 算术逻辑部件ALU的设计

○ 输入信号

- ◆ 两个32位操作数A和B。
- ◆ ALU控制信号ALUctr：生成中间控制信号

○ 输出信号

- ◆ 32位结果信号Result
- ◆ 1位结果为零标注位Zero

○ 3个中间控制信号

- ◆ SUBctr 用来控制加减法运算，SUBctr=1 时，做减法。
- ◆ SIGctr 信号控制执行带符号整数还是无符号整数比较小于置1，SIGctr=1，执行带符号整数比较小于置1。
- ◆ OPctr 用来选择ALU输出结果Result。



2.2 单周期数据通路的设计

■ 9条目标指令对应的3种ALU操作控制信号取值

指 令	功 能	运 算 类 型	SUBctr	SIGctr	OPctr<1:0>
add rd, rs1, rs2	$R[rd] \leftarrow R[rs1] + R[rs2]$	加	0	×	00
slt rd, rs1, rs2	if ($R[rs1] < R[rs2]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$	减, 带符号整数 比较大小	1	1	11
sltu rd, rs1, rs2	if ($R[rs1] < R[rs2]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$	减, 无符号数比 较大小	1	0	11
ori rt, rs1, imm12	$R[rt] \leftarrow R[rs1] \mid \text{SEXT}(\text{imm12})$	按位或	×	×	01
lui rd, imm20	$R[rt] \leftarrow \text{imm20} \parallel 000H$	选择操作数B	×	×	10
lw rd,imm12(rs1)	$\text{Addr} \leftarrow R[rs1] + \text{SEXT}(\text{imm12})$ $R[rd] \leftarrow M[\text{Addr}]$	加	0	×	00
sw rs2, imm12(rs1)	$\text{Addr} \leftarrow R[rs1] + \text{SEXT}(\text{imm12})$ $M[\text{Addr}] \leftarrow R[rs2]$	加	0	×	00
beq rs1, rs2, imm12	$\text{Cond} \leftarrow R[rs1] - R[rs2]$	减 (判0)	1	×	×
	if ($\text{Cond eq } 0$) $\text{PC} \leftarrow \text{PC} + (\text{SEXT}(\text{imm12}) << 1)$	加	0	×	00
jal rd, imm20	$R[rd] \leftarrow \text{PC} + 4$ $\text{PC} \leftarrow \text{PC} + (\text{SEXT}(\text{imm20}) << 1)$	加	0	×	00

归纳为以下6种操作： add、sub、sltu、slt、 or、 srcB， 4种结果输出。



2.2 单周期数据通路的设计

■ ALUctr的一种四位编码方案。

ALUctr<3:0>	操作类型	SUBctr	SIGctr	OPctr<1:0>	OPctr的含义
0 0 0 0	add	0	×	0 0	选择加法器的结果输出
0 0 0 1	(未用)				
0 0 1 0	slt	1	1	1 1	选择小于置位结果输出
0 0 1 1	sltu	1	0	1 1	选择小于置位结果输出
0 1 0 0	(未用)				
0 1 0 1	(未用)				
0 1 1 0	or	×	×	0 1	选择“按位或”结果输出
0 1 1 1	(未用)				
1 0 0 0	sub	1	×	0 0	选择加法器的结果输出
其余	(未用)				
1 1 1 1	srcB	×	×	1 0	选择操作数B直接输出

课堂练习：使用最小风险法列出输出信号表达式



2.2 单周期数据通路的设计

- ALUctr输出控制信号的逻辑表达式(**最小成本法**化简)

$$\text{SUBctr} = \text{ALUctr}[3] + \text{ALUctr}[1]$$

$$\text{SIGctr} = \overline{\text{ALUctr}[0]}$$

$$\text{OPctr}[1] = \overline{\text{ALUctr}[2]} \cdot \text{ALUctr}[1] + \text{ALUctr}[3] \cdot \text{ALUctr}[1]$$

操作(slt, sltu)

操作(srcB)

$$\text{OPctr}[0] = \overline{\text{ALUctr}[2]} \cdot \text{ALUctr}[1] + \text{ALUctr}[1] \cdot \overline{\text{ALUctr}[0]}$$

操作(or)

可实现ALU操作控制信号生成部件。

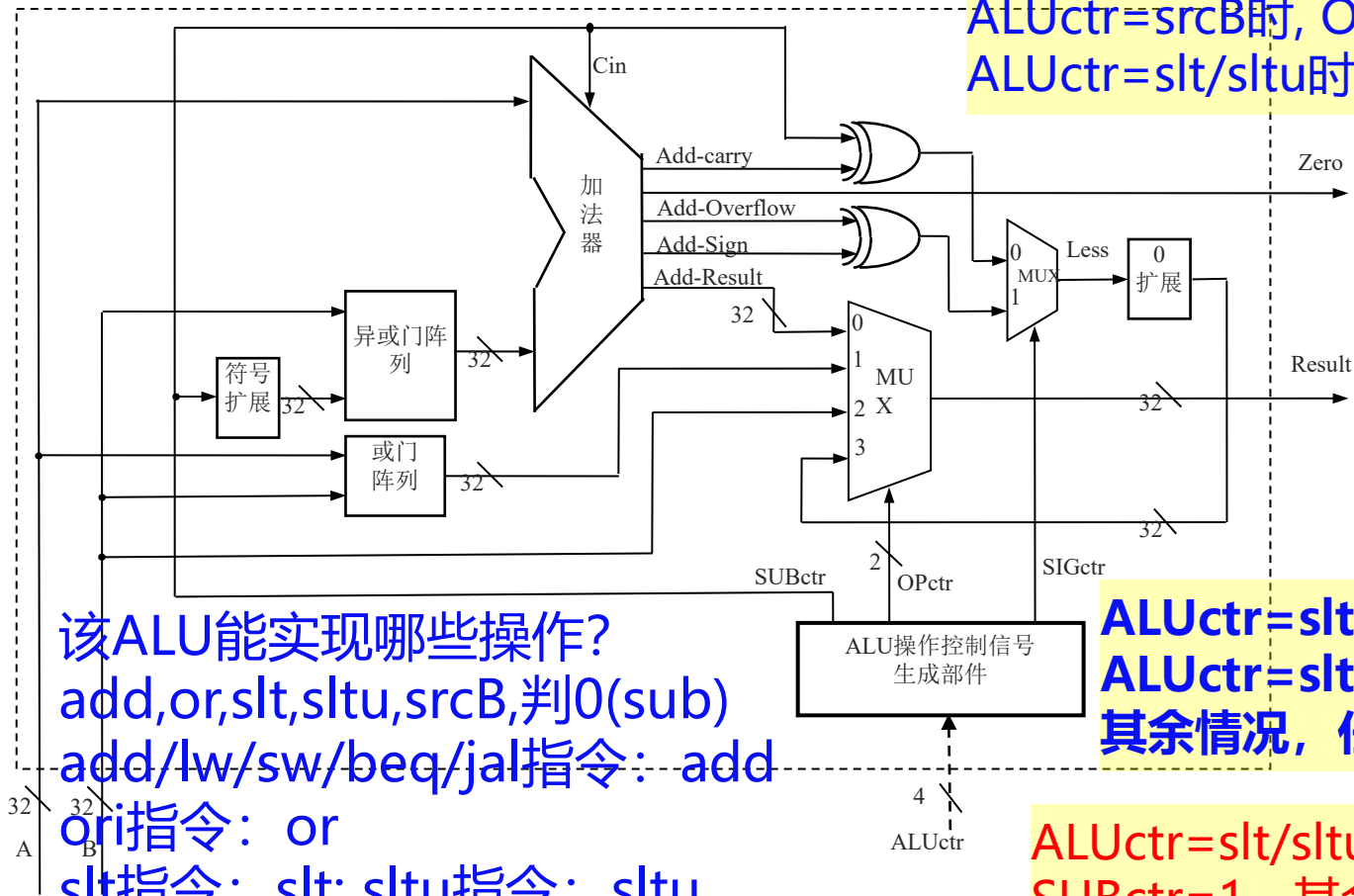
指令越多，ALUctr编码和Opctr编码位数越长。

问题：指令Instr如何决定ALUctr？ 通过指令译码器得到！



2.2 单周期数据通路的设计

■ 算术逻辑部件ALU的设计



ALUctr=add时, OPctr=00
 ALUctr=or时, OPctr=01
 ALUctr=srcB时, OPctr=10
 ALUctr=slt/sltu时, OPctr=11

ALUctr=slt, SIGctr=1
 ALUctr=sltu, SIGctr=0
 其余情况, 任意

ALUctr=slt/sltu/sub时,
 SUBctr=1, 其余 SUBctr=0

如何拓展到全部RV32I?



ALUctr的四位编码 (37条指令)

ALUctr <3:0>	指令操作类型	SUBctr	SIGctr	ALctr	SFTctr	OPctr <2:0>	OPctr的含义
0 0 0 0	auipc,addi,add,jal,jalr,lb,lh,lw,lbu,ldu,sb,sh,sw: 加法	0	×	×	×	000	选择加法器的结果输出
0 0 0 1	sll,slli: 逻辑左移	×	×	0	1	100	选择移位器结果输出
0 0 1 0	slt,slti,beq,bne,blt,bge: 带符号数小于比较	1	1	×	×	110	选择小于置位结果输出
0 0 1 1	sltu,sltui,bltu,bgeu: 无符号数小于比较	1	0	×	×	110	选择小于置位结果输出
0 1 0 0	xor,xori: 异或	×	×			011	选择“按位异或”结果输出
0 1 0 1	srl,srli: 逻辑右移	×	×	0	0	100	选择移位器结果输出
0 1 1 0	or,ori: 或运算	×	×	×	×	010	选择“按位或”结果输出
0 1 1 1	and,andi: 与运算	×	×	×	×	001	选择“按位与”结果输出
1 0 0 0	sub: 减法	1	×	×	×	000	选择加法器的结果输出
1 1 0 1	sra,srai: 算术右移	×	×	1	0	100	选择移位器结果输出
其余	(未用)						
1 1 1 1	lui: 取操作数B	×	×	×	×	101	选择操作数B直接输出

ALUctr位数?

控制信号的逻辑表达式?

可以选择不同的编码方案



引擎的点火与指令取指(IF)

■ 取指令部件IFU的设计

■ 每条指令都有的公共操作：

- 取指令：[IR/数据总线] $\leftarrow$ M[PC]
- 更新PC：
 - ◆ 顺序指令：PC $\leftarrow$ PC + 4
 - ◆ 分支与跳转指令（Branch and Jump）：PC内容可能被更新为“转移目标地址”。

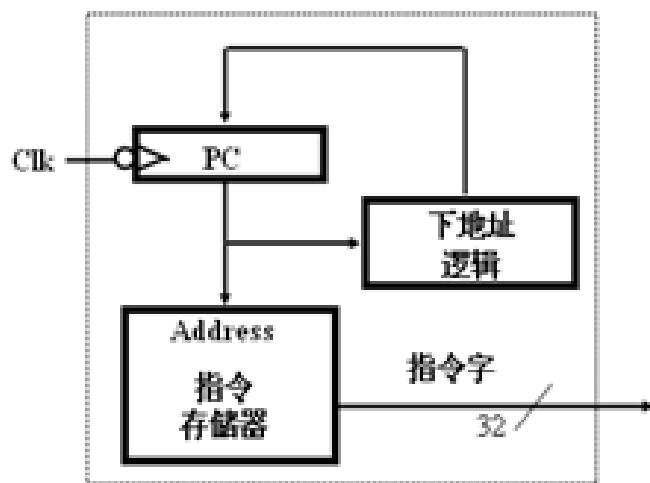


图 8.11 取指令部件示意图

顺序：先取指令，再改PC的值（具体实现时，可以并行）。**绝不能先改PC的值，再取指令。**

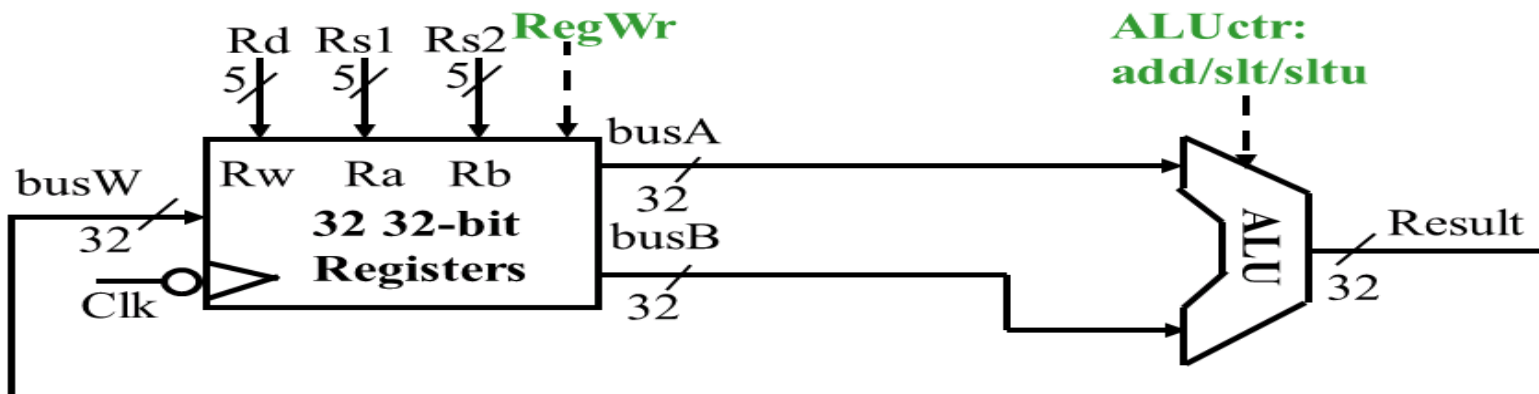
取指后，各指令功能不同，数据通路中信息流动过程也不同。

下面分别对每条指令进行相应数据通路的设计。



构建纯粹的R - 型计算通路

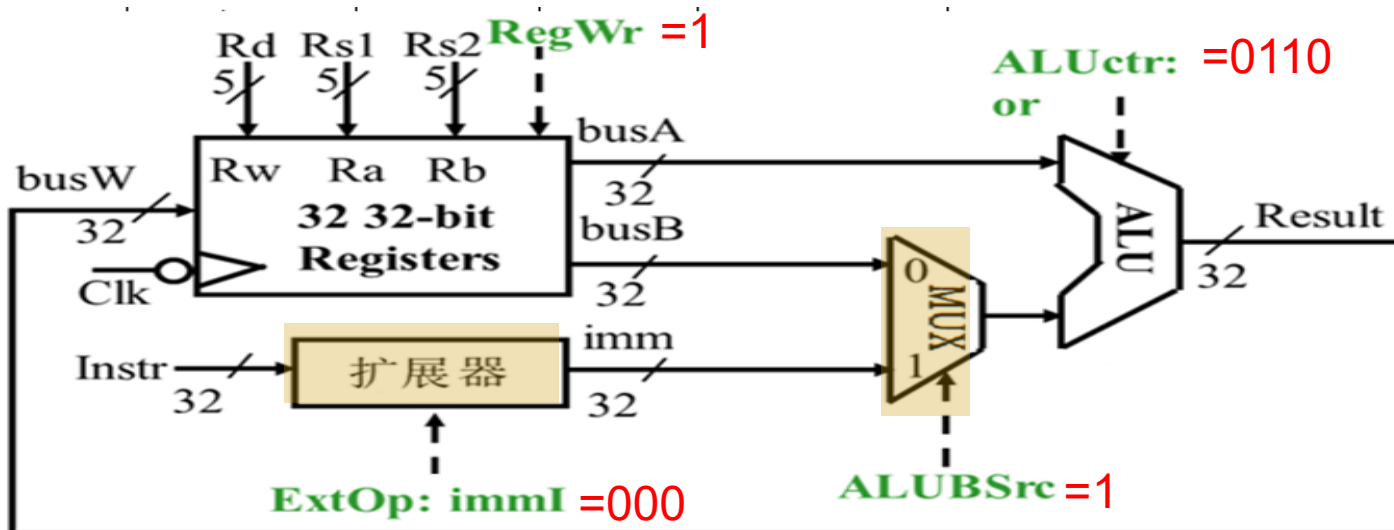
- R-型指令的数据通路： $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$
 - 数据通路中包含寄存器堆和ALU两个组件
- 有三条R-型指令：add、slt和sltu，分别对应ALU的三种操作
 - 将不同的4位编码（0000、0010或0011）送到ALU操作控制端ALUctr，以使在ALU中进行不同指令所对应的运算。
 - ALU两个操作数分别来自busA和busB。
 - 指令执行时，寄存器写使能信号RegWr=1，ALU结果Result才能写入到寄存器堆中Rw指向的寄存器。





I-型带来的第一场硬件冲突

- I-型运算指令的数据通路，**ori** : $R[rd] \leftarrow R[rs1] \text{ or } \text{SEXT}(\text{imm12})$
 - 增加**立即数扩展器**，对12位立即数进行符号扩展
 - ALU端口B的操作数来源增加了**立即数**，增加一个**多路选择器MUX**，需要一个选择信号**ALUBSrc**。
- ALUctr取值为0110，ALU操作为or；立即数扩展器控制信号ExtOp为000：imm1，操作数B的来源选择信号ALUBSrc为**1**，**RegWr**为**1**。





-
- Diagram of a 32-bit RISC processor architecture. The processor consists of a 32-bit Register File, an ALU, a Multiplexer (MUX), and an Extender. The Register File has three 5-bit inputs: Rd, Rs1, and Rs2, and a 5-bit output: RegWr. It has three 32-bit outputs: busA, busB, and imm. The ALU has two 32-bit inputs: busA and busB, and a 32-bit output: Result. The MUX has two 32-bit inputs: busB and imm, and a 32-bit output: Result. The Extender has a 32-bit input: Instr, and a 32-bit output: imm. The processor is controlled by a 32-bit busW and a Clk signal. Handwritten annotations in red and green indicate specific control signals: RegWr = 1, ALUctr: srcB = 1111, ExtOp: immU = 001, and ALUBSrc = 1.



I-型的lw指令的功能: $R[rd] \leftarrow M[R[rs1] + \text{SEXT}(\text{imm12})]$

S-型的sw指令的功能: $M[R[rs1] + \text{SEXT}(\text{imm12})] \leftarrow R[rs2]$

- 数据通路中增加**数据存储器**。
- busW写回总线有两个**不同的**来源，**导致冲突**，增加**多路选择器MUX**。

- 数据存储器的**地址端**Addr1连到ALU的输出，DataIn**数据输入端**连到busB，**数据输出端**连到busW端的多路选择器上。

lw、sw的ALUctr是add; 立即数扩展类型分别为imm1、immS, ExtOp分别为000、010; ALUBSrc=1; MemWr分别为0、1; MemtoReg分别为1、x (x表示任意); RegWr分别为1、0。





- immB控制imm为:**
 $\text{SEXT}(\text{imm}12) \ll 1$
ExtOp=011





- 当时钟Clk边沿有效时，当前PC作为指令地址被送到指令存储器的地址端Addr去取指令的同时，下地址逻辑部件将计算的下条指令地址并送到PC的输入端。



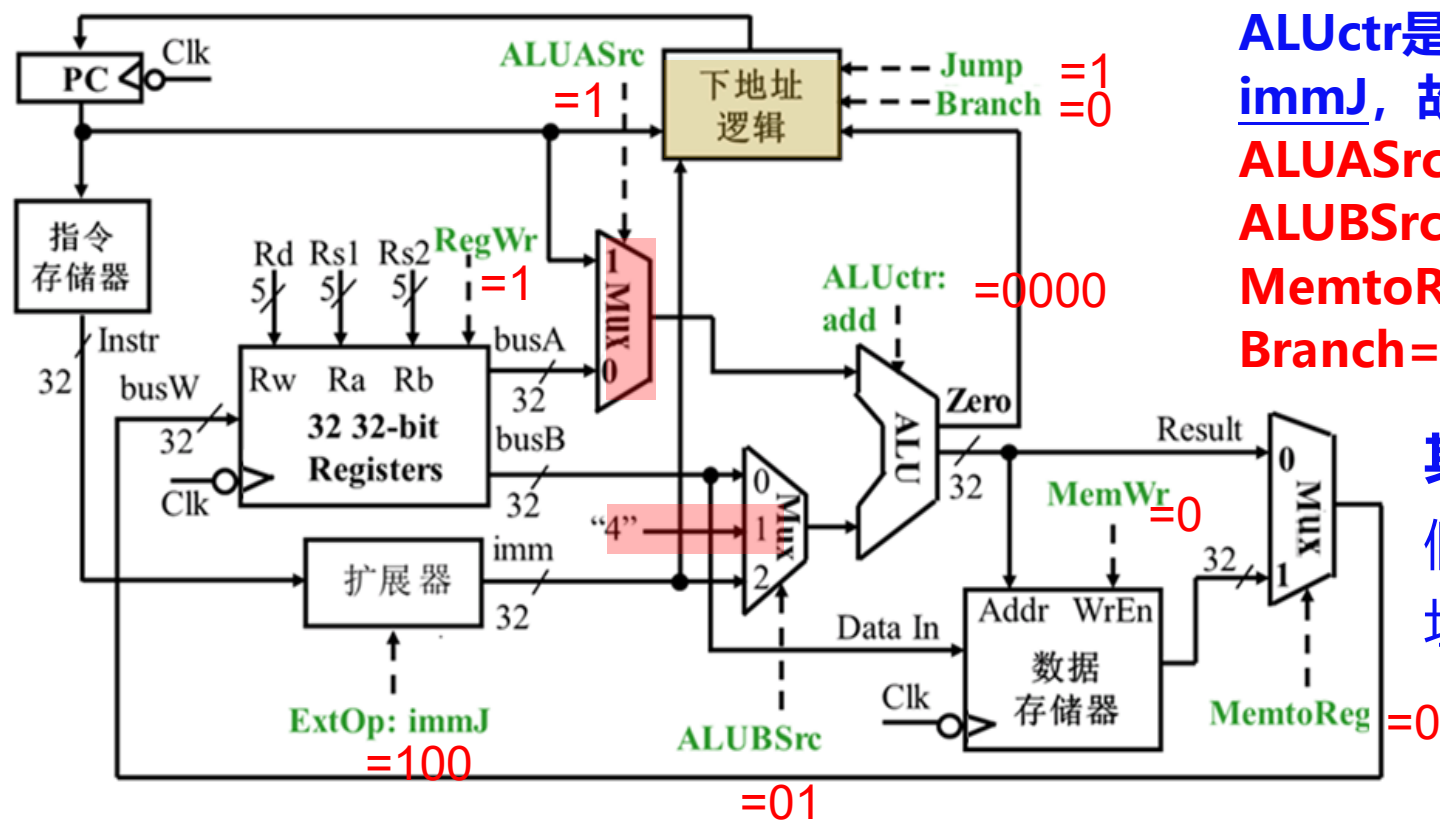


J-型无条件跳转与函数调用的双重代价

- 函数调用的双重使命：1、跳得远；2、记得回。
- J-型指令的数据通路，jal功能是：

$R[rd] \leftarrow PC + 4; \quad PC \leftarrow PC + \text{SEXT}(\text{imm}[20:1] \ll 1);$

- ALU端口A增加一个来源PC，端口B增加一个来源：常量4。



ALUctr是add, 立即数为immJ, 故ExtOp=100;

ALUASrc=1;

ALUBSrc=01; MemWr=0;

MemtoReg=0; RegWr=1;

Branch=0; Jump=1

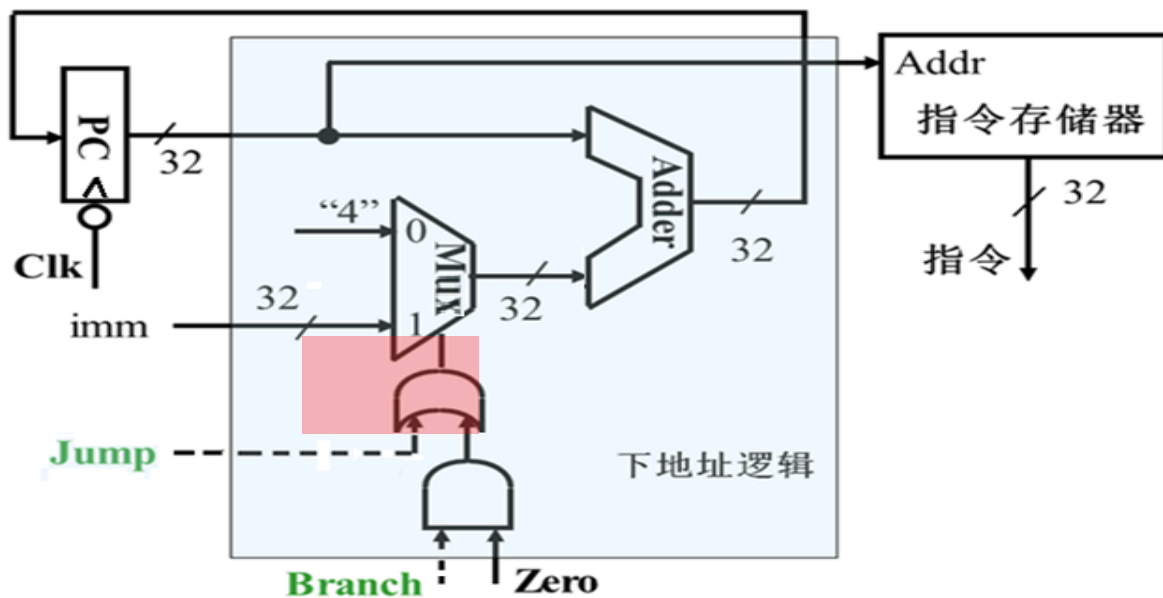
其他实现方法?

假设PC+4来自下地址逻辑, 如何修改?



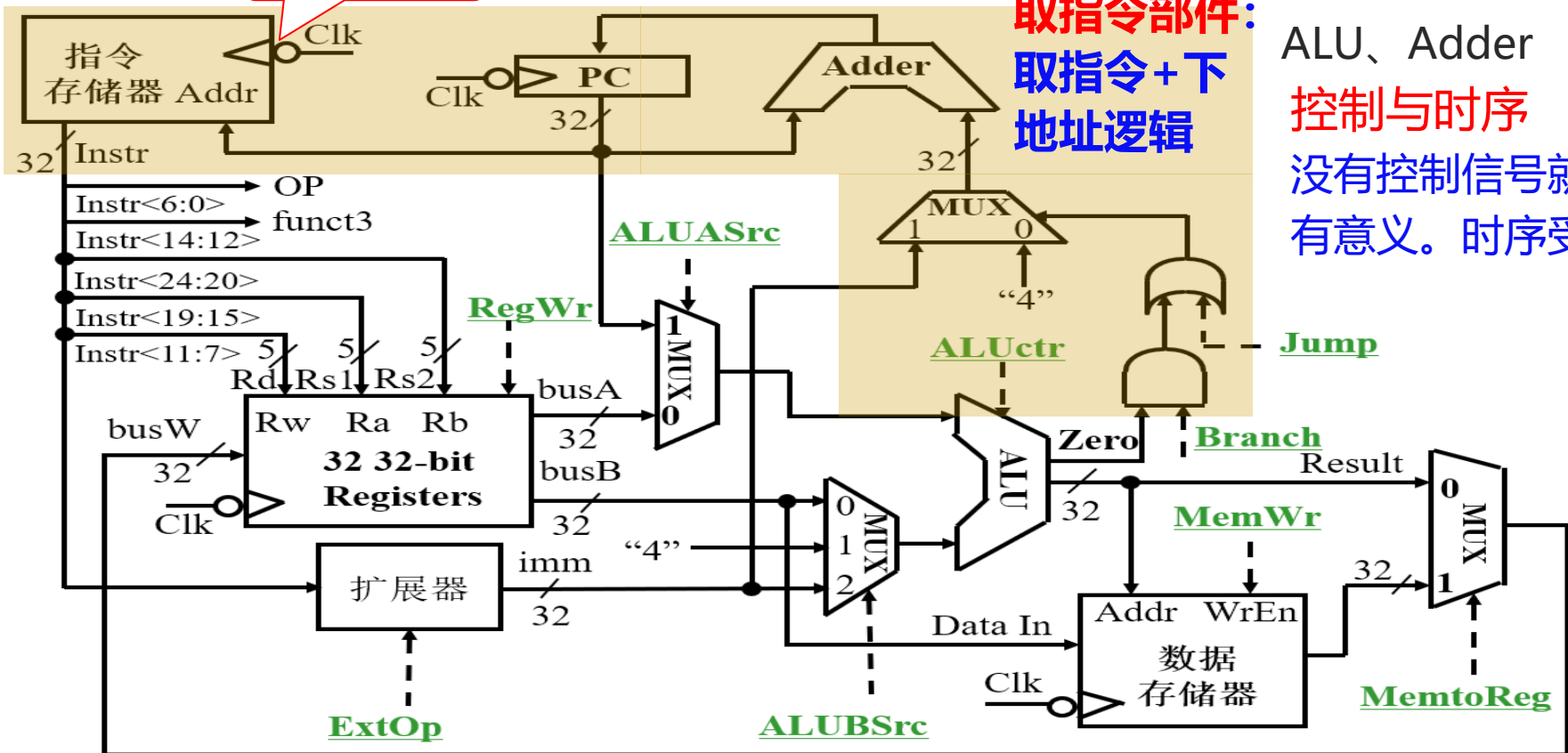
J-型无条件跳转与函数调用的双重代价

- 下地址逻辑如何实现？
- 在B-型指令下地址逻辑实现部件上，增加无条件转移指令功能。
 - 完整取指令部件示意图





能否取消？



发现冲突、加MUX

空间复用

ALU、Adder

控制与时序

没有控制信号就没有意义。时序受限。

指令执行结果总是在下个时钟到来时开始保存在 **寄存器堆** 或 **存储器** 或 **PC** 中!



2.2 单周期数据通路的设计小结

- CPU设计直接决定了**时钟周期宽度**和**CPI**，所以对计算机性能非常重要！
- CPU主要由数据通路和控制器组成
 - **数据通路**：实现指令集中所有指令的操作功能
 - **控制器**：控制数据通路中各部件进行正确操作
- 数据通路中包含两种元件
 - **操作元件**（组合电路）：ALU、MUX、扩展器、Adder、Reg/Mem Read等
 - **状态 / 存储元件**（时序电路）：PC、Reg/Mem Write
- 数据通路的定时
 - 数据通路中的操作元件没有存储功能，其操作结果必须**写到存储元件**中
 - 在时钟到达后**clk-to-Q**时存储元件开始更新状态
- RV32I指令集的一个子集作为CPU的实现目标
 - 公共操作：取指令和PC+4
 - 下址计算：PC，三路选择：顺序、Branch（结合标志Zero）、Jump
 - R-型：ALU两个操作数来自rs1和rs2，结果写到rd
 - 访存：**符号**扩展，数据在**rs2寄存器**和**主存单元**中交换
 - 立即数：不同方式的扩展操作数imm送到ALU的一个输入端



单周期控制器与时序极限

为数据通路注入灵魂
探索微架构的物理边界



2.3 控制器的设计

- 控制器是**纯组合逻辑电路**。
 - 输入是指令**操作码**op和**功能码**（funct3字段、 funct7字段）
 - 输出是**控制信号**。
- 分析每条指令在数据通路中的执行过程和涉及到的控制信号的取值，并列表显示
 - 公共操作：取指令和计算下一址PC
 - R-型指令（add / slt / sltu）
 - I-型运算类指令（ori）
 - U-型指令（lui）
 - 访存指令（lw / sw）
 - B-型指令（beq）
 - J-型指令（jal）
- 根据列出指令和控制信号之间的关系，写出每个控制信号的逻辑表达式。



-
- The diagram illustrates the control logic for a 32-bit processor. A 32-bit PC register is clocked by a Clk signal. The PC output is split into two 32-bit paths: one to the Instruction Memory (Addr<31:0>) and another to an Adder. The Adder also takes a 32-bit constant '4' as input. The output of the Adder is fed into a 32-bit Multiplexer (Mux). The Mux has two inputs: the output of the Adder and a 32-bit immediate value (imm). The Mux is controlled by two AND gates. The first AND gate takes the output of the Branch = previous signal and the Zero = previous signal. The second AND gate takes the output of the Jump = previous signal. The output of the Mux is fed back to the PC register.
- Branch = previous
- Zero = previous
- Jump = previous

取指部件由旧控制信号控制，会不会有问题？

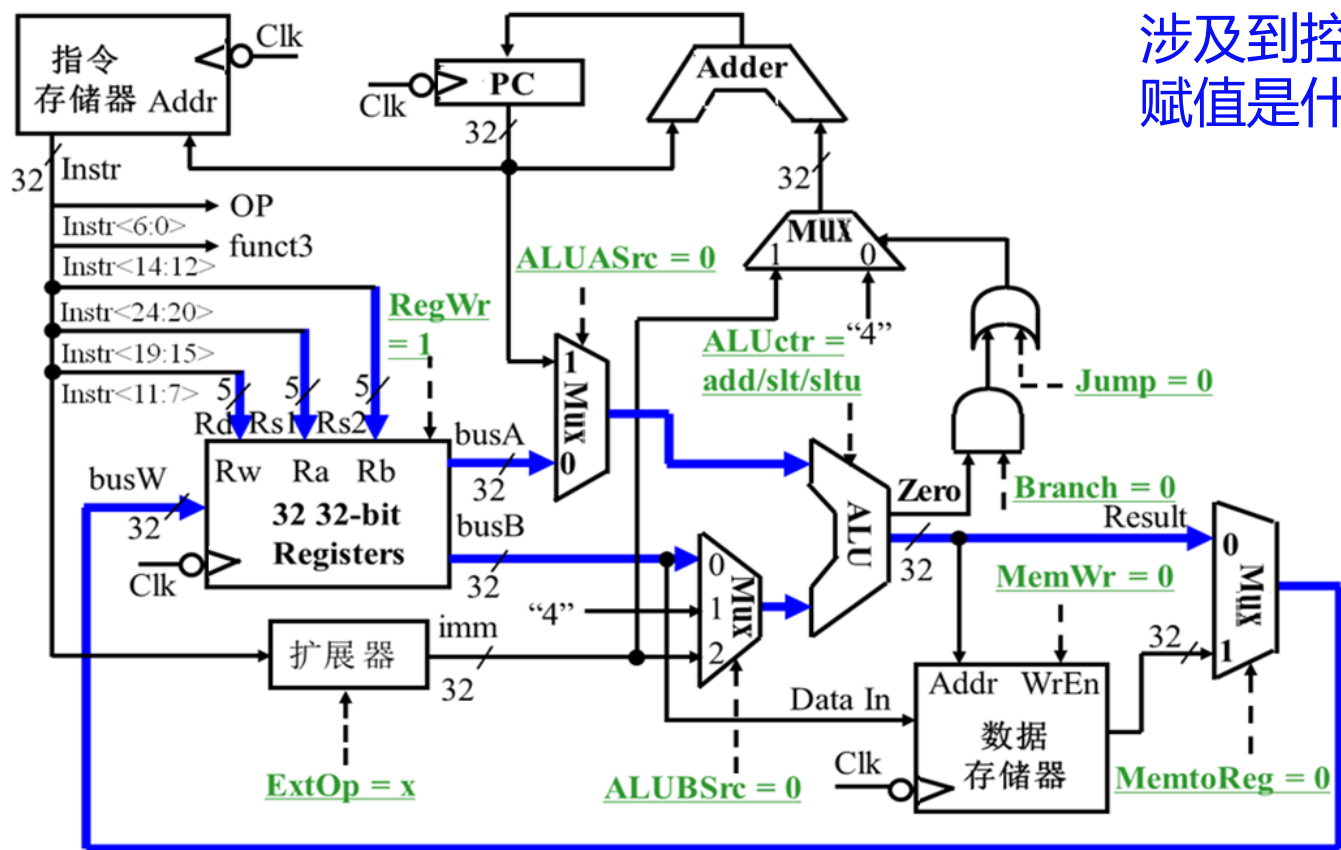
没有问题! Why?

因为在下个Clk到来之前PC输入端的值不会写入，只要保证下个Clk来之前能产生正确的PC即可！



2.3 控制器的设计

- 指令译码后R-型指令操作过程: $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$



涉及到控制信号有哪些?
赋值是什么?

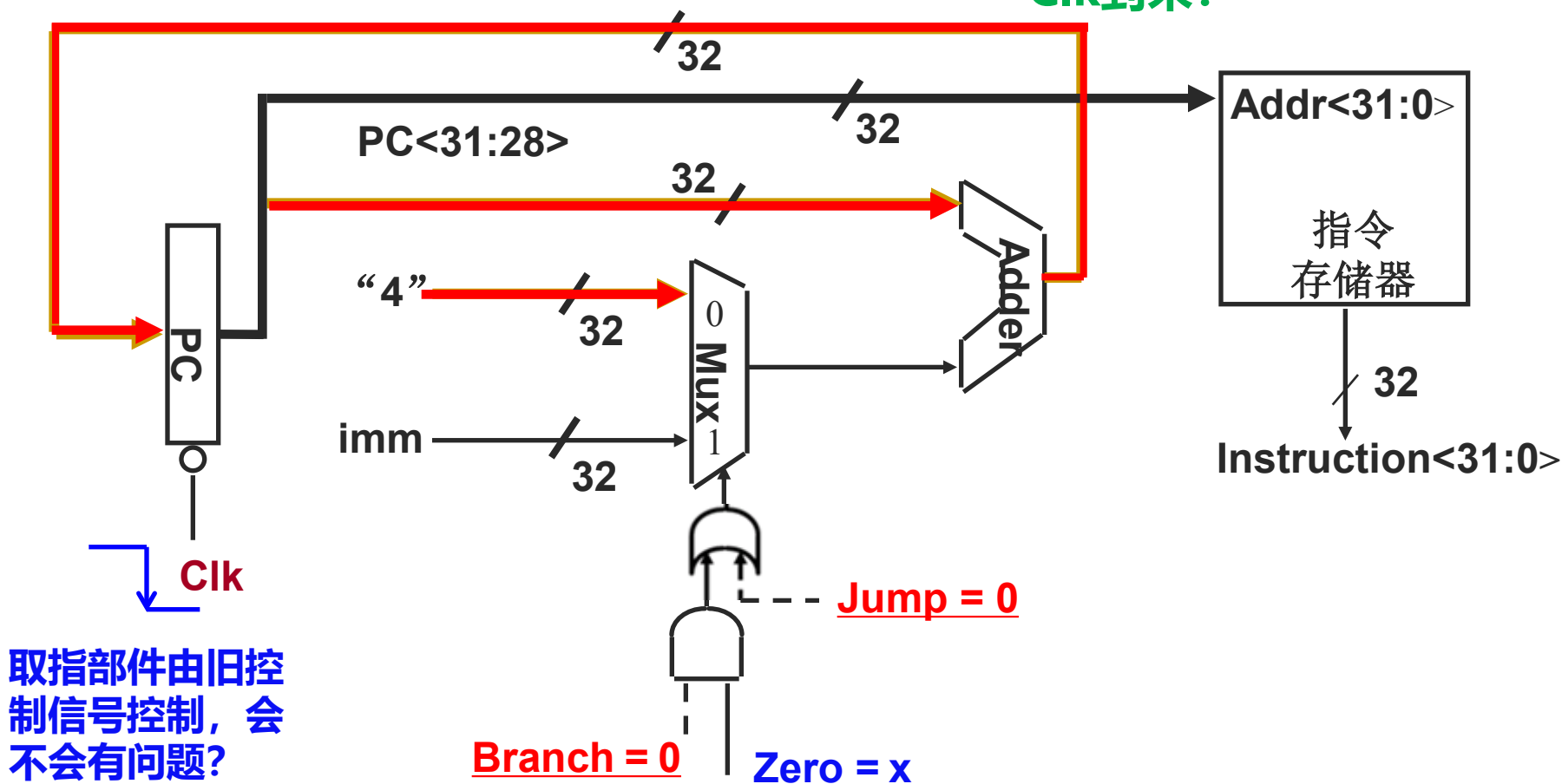
RegWr=1
ExtOp=x
ALUASrc=0
ALUBSrc=0
ALUctr=add/slt/sltu
MemtoReg=0
MemWr=0
Branch=0
Jump=0



2.3 控制器的设计

- R-型指令执行过程中取指部件中的动作
 - $PC \leftarrow PC + 4$
 - 除 Branch and Jump以外的指令都相同

因为新的控制信号保证了正确的PC值的产生，在足够长的时间后，下个时钟Clk到来！

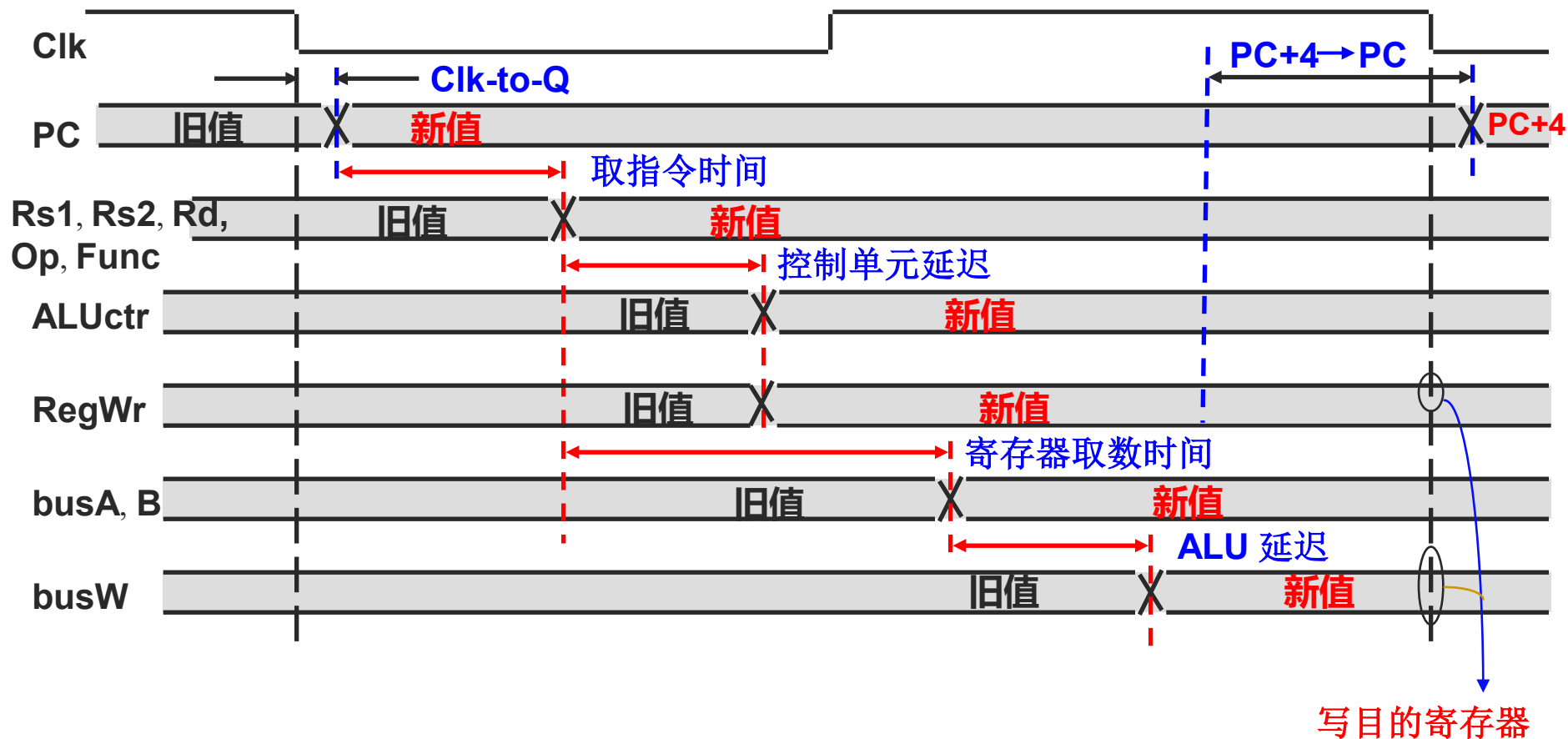


取指部件由旧控制信号控制，会不会有问题？



2.3 控制器的设计

- R-型指令的操作定时: $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$



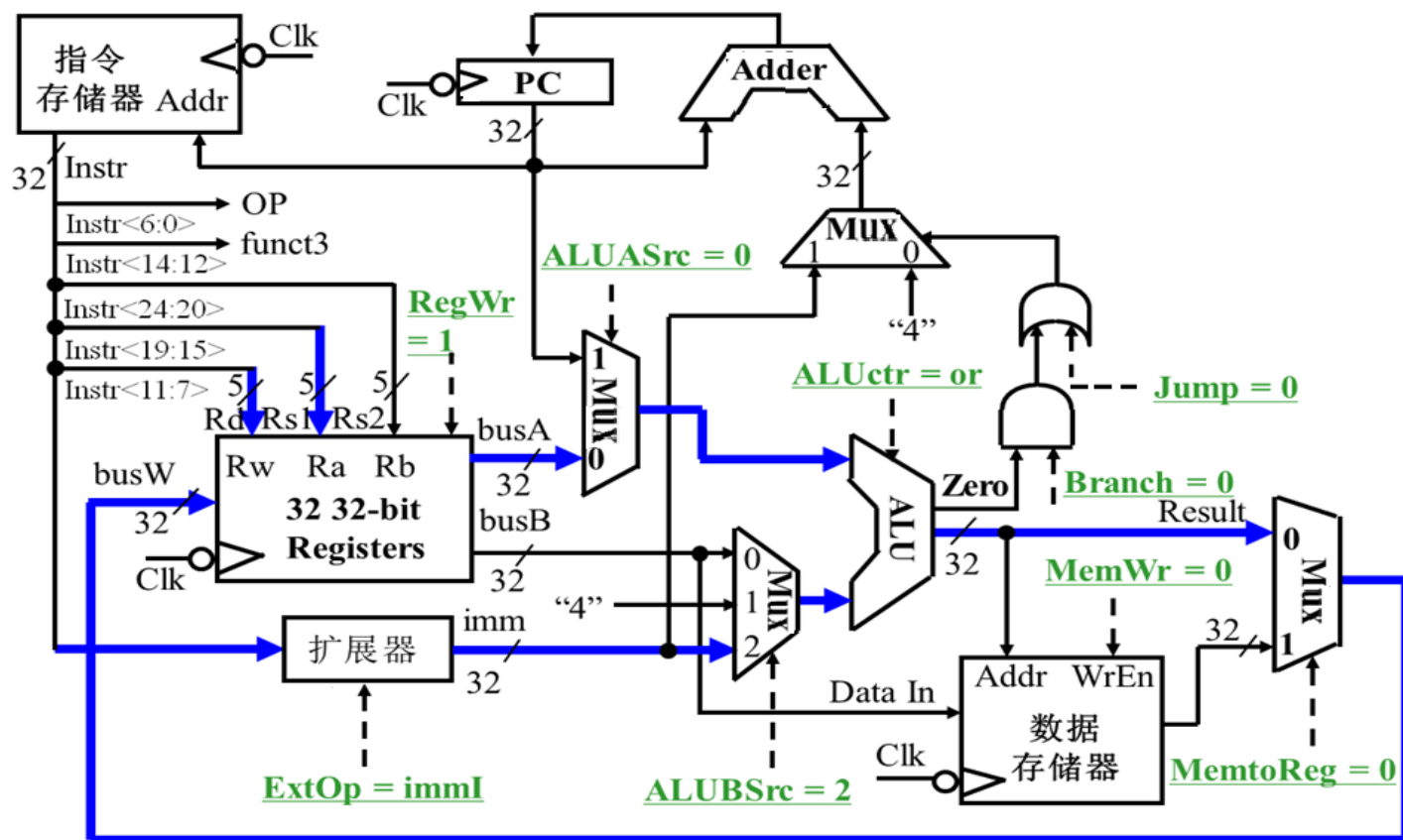


2.3 控制器的设计

■ 指令译码后I-型运算指令操作过程

- ori 功能: $R[rd] \leftarrow R[rs1] \text{ or } \text{SEXT}(\text{imm12})$

涉及到控制信号有哪些?
赋值是什么?



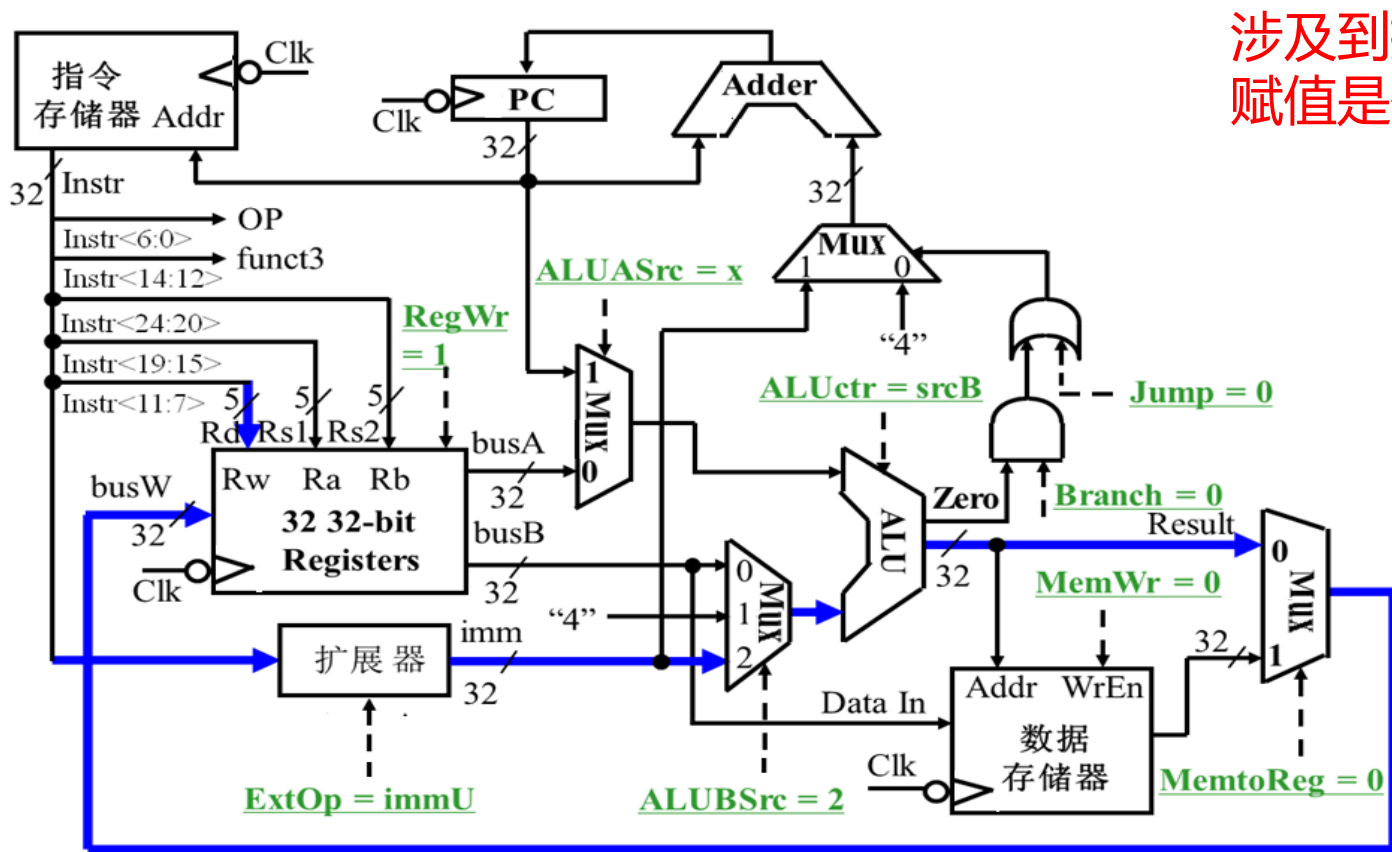
RegWr=1
ExtOp=imm1
ALUASrc=0
ALUBSrc=2
ALUctr=or
MemtoReg=0
MemWr=0
Branch=0
Jump=0



2.3 控制器的设计

■ 指令译码后U-型指令操作过程

- lui的功能: $R[rd] \leftarrow \text{imm20} || 000H$ 即直接将扩展器结果输出



涉及到控制信号有哪些？
赋值是什么？

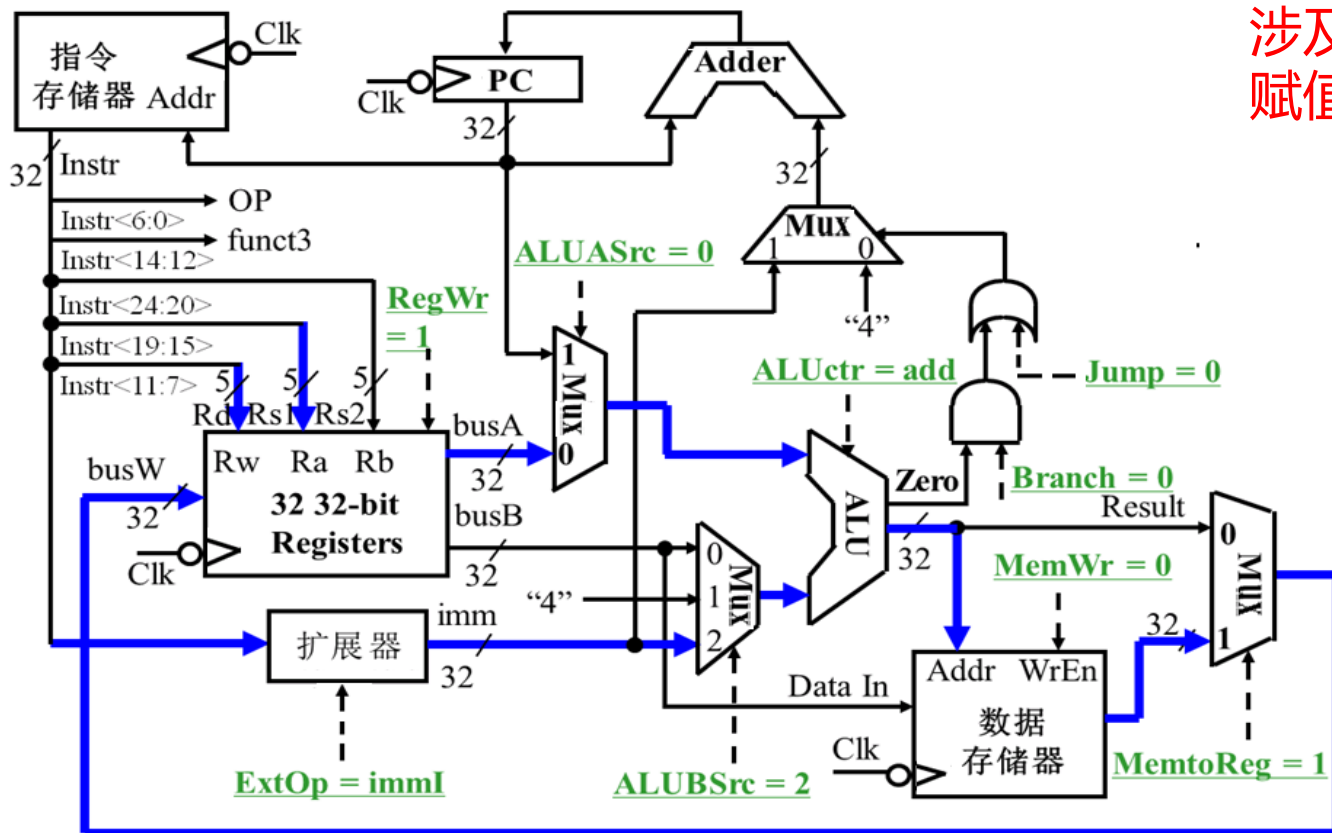
RegWr=1
ExtOp=immU
ALUASrc=x
ALUBSrc=2
ALUctr=srcB
MemtoReg=0
MemWr=0
Branch=0
Jump=0



2.3 控制器的设计

■ 指令译码后Load/Store指令操作过程

- I-型的lw指令的功能: $R[rd] \leftarrow M[R[rs1] + \text{SEXT}(\text{imm12})]$



涉及到控制信号有哪些?
赋值是什么?

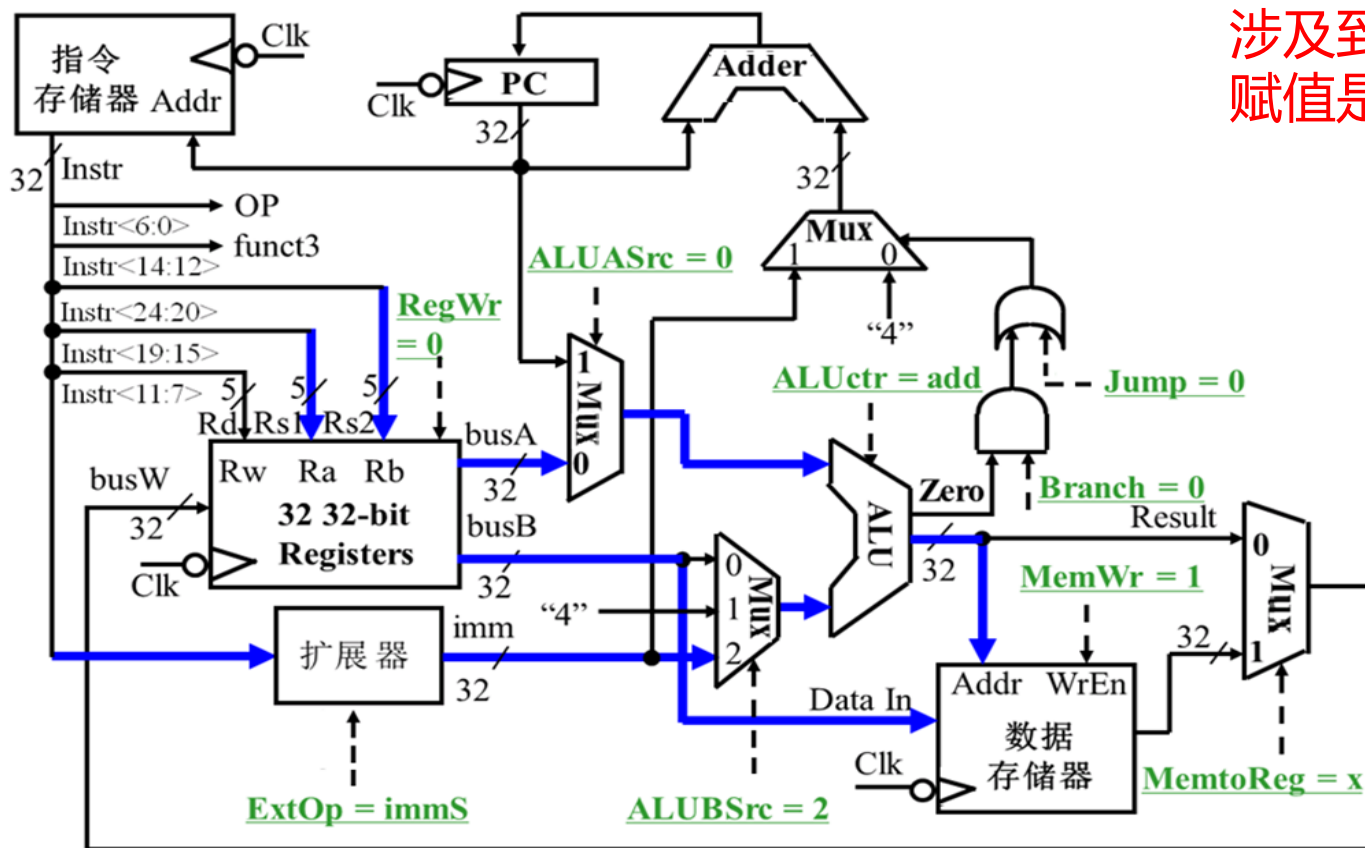
RegWr=1
ExtOp=imml
ALUASrc=0
ALUBSrc=2
ALUctr=add
MemtoReg=1
MemWr=0
Branch=0
Jump=0



2.3 控制器的设计

■ 指令译码后Load/Store指令操作过程

- S-型的sw指令的功能: $M[R[rs1] + \text{SEXT}(\text{imm12})] \leftarrow R[rs2]$



涉及到控制信号有哪些？
赋值是什么？

RegWr=0
ExtOp=immS
ALUASrc=0
ALUBSrc=2
ALUctr=add
MemWr=1
Branch=0
Jump=0

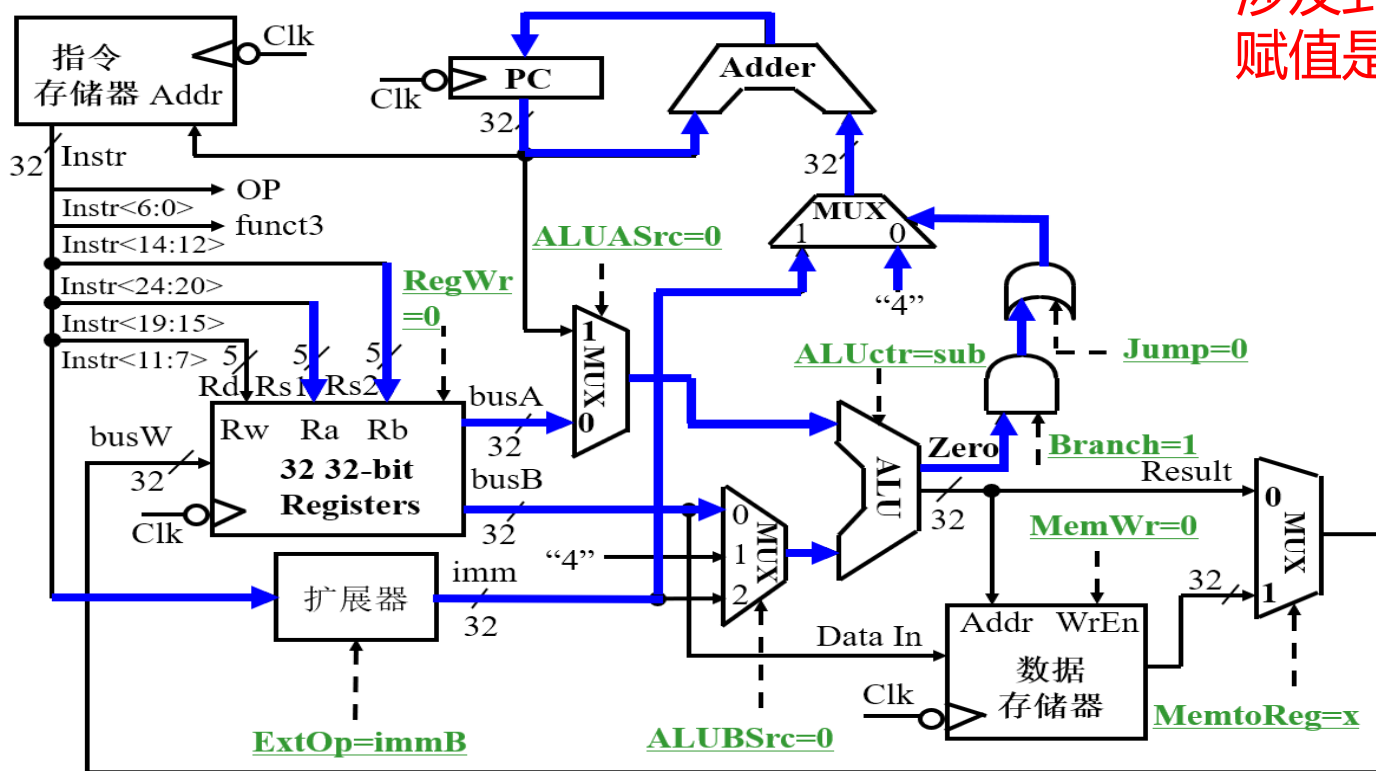


2.3 控制器的设计

■ 指令译码后B-型指令操作过程

- beq功能: $\text{if } (R[\text{rs1}] = R[\text{rs2}]) \text{ PC} \leftarrow \text{PC} + (\text{SEXT}(\text{imm12}) \ll 1) \text{ else } \text{PC} \leftarrow \text{PC} + 4$

涉及到控制信号有哪些?
赋值是什么?

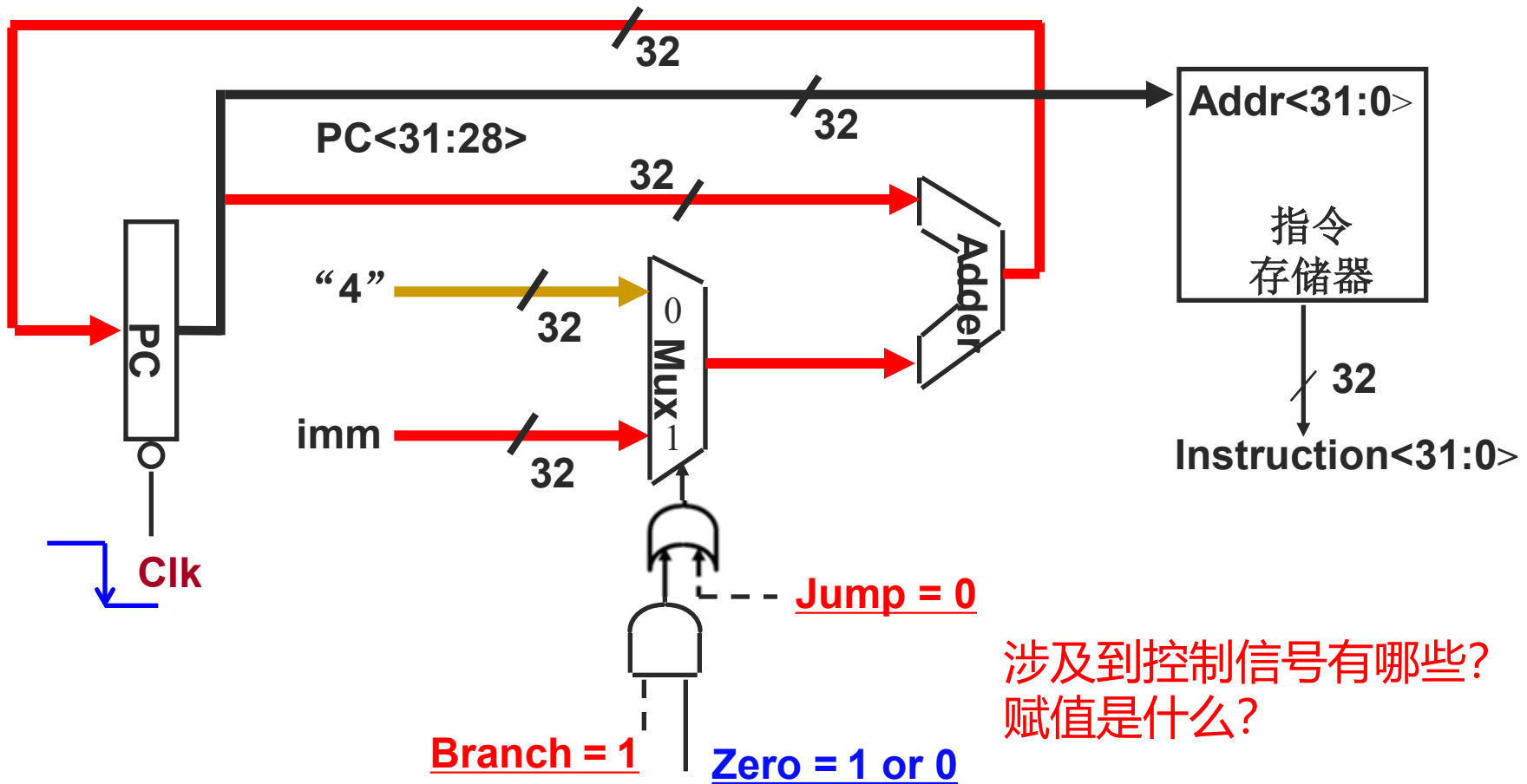


RegWr=0
ExtOp=immB
ALUASrc=0
ALUBSrc=0
ALUctr=add
MemtoReg=x
MemWr=0
Branch=1
Jump=0



2.3 控制器的设计

- beq指令最后阶段取指部件中的动作
 - if (Zero == 1) then $PC = PC + imm$ else $PC = PC + 4$



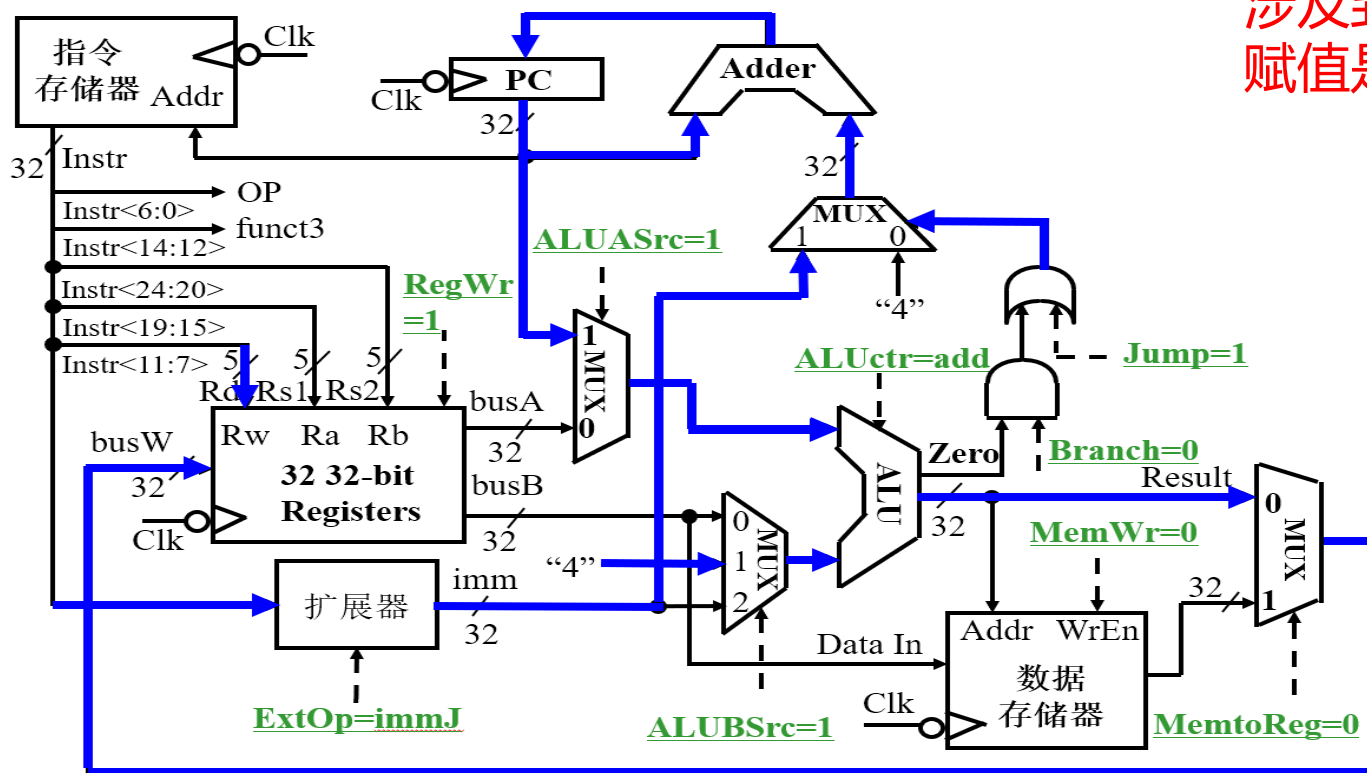


2.3 控制器的设计

■ 指令译码后J-型指令操作过程

- J-型指令jal的功能: $R[rd] \leftarrow PC + 4;$
 $PC \leftarrow PC + \text{SEXT}(\text{imm}[20:1] \ll 1);$

涉及到控制信号有哪些?
赋值是什么?





2.3 控制器的设计

指令与控制信号的关系表

	R-type			I-type		U-type	Load-type	S-type	B-type	J-type
funct3	000	010	011	110	无关	010	010	000	无关	
op	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111	
控制信号	add	slt	sltu	ori	lui	lw	sw	beq	jal	
Branch	0	0	0	0	0	0	0	1	0	
Jump	0	0	0	0	0	0	0	0	1	
ALUASrc	0	0	0	0	x	0	0	0	1	
ALUBSrc<1:0>	00	00	00	10	10	10	10	00	01	
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)	
MemtoReg	0	0	0	0	0	1	x	x	0	
RegWr	1	1	1	1	1	1	0	0	1	
MemWr	0	0	0	0	0	0	1	0	0	
ExtOp<2:0>	x	x	x	000 imml	001 immU	000 imml	010 immS	011 immB	100 immJ	

Branch = B-type

Jump = J-type

ALUASrc = J-type

MemtoReg = Load-type

MemWr = S-type

RegWr = R-type | I-type | U-type | Load-type | J-type



2.3 控制器的设计

■ 多值控制信号逻辑表达式的生成

控制信号 funct3 op	000	010	011	110	无关	010	010	000	无关
	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
	add	slt	sltu	ori	lui	lw	sw	beq	jal
ALUBSrc<1:0>	00	00	00	10	10	10	10	00	01
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)
ExtOp<2:0>	×	×	×	000 imml	001 immU	000 imml	010 immS	011 immB	100 immJ

ALUBSrc<1> = I-type | U-type | Load-type | S-type

ALUBSrc<0> = J-type

ExtOp<2:0> 根据指令类型opcode选择输出信号（LUT）！

设funct3各位分别表示为fn[2] fn[1] fn[0]，ALUctr各位逻辑表达式如下：

ALUctr<3> = U-type | B-type ALUctr<2> = I-type | U-type

ALUctr<1> = (R-type and fn<1>) | I-type | U-type

ALUctr<0> = (R-type and fn<0>) | U-type



2.3 控制器的设计

■ 多值控制信号逻辑表达式的生成

控制信号 funct3 op	000	010	011	110	无关	010	010	000	无关
	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
	add	slt	sltu	ori	lui	lw	sw	beq	jal
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)

- 观察到：R-型和I-型运算类指令的funct3与ALUctr后3位编码相同，而最高位为0，即ALUctr=0||funct3，lw、sw和jal都是对应add操作，即ALUctr=0000；add/slt/sltu的op值相同。

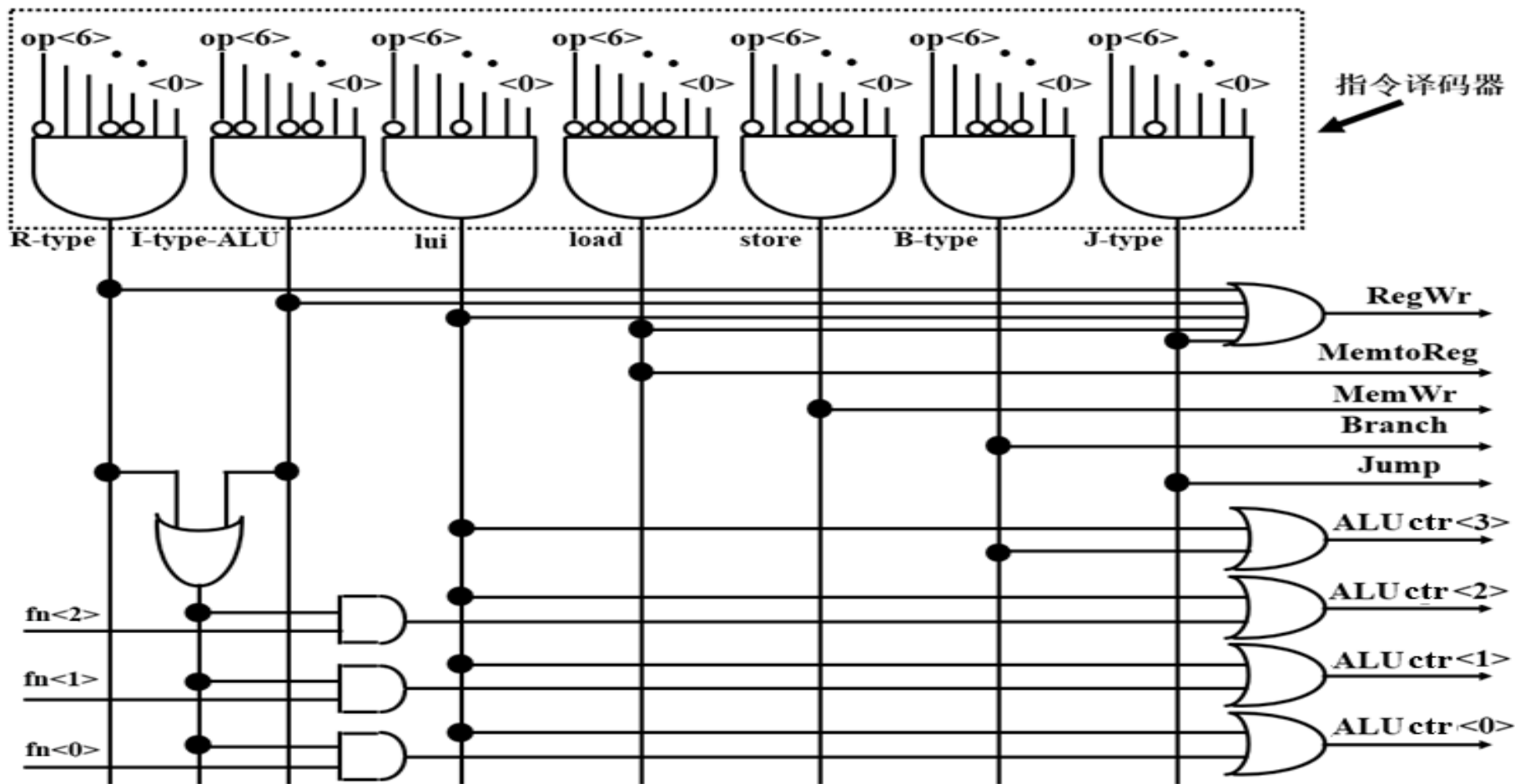
	R-type	I-type-ALU	lui	Load/Store	B-type	J-type
ALUctr<3:0>	0 funct3 (为串接操作)	1111	0000	1000	0000	

可使用多路选择器输出4位ALUctr



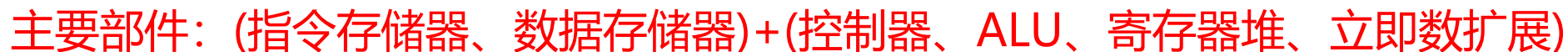
2.3 控制器的设计

- 根据逻辑表达式实现控制器的PLA电路





■ 执行前述9条指令完整的单周期处理器





2.4 时钟周期的确定

- 单周期处理器每条指令在一个时钟周期内完成，CPI=1。
- 时钟周期的确定
 - 由数据通路中的**关键路径**决定。
 - **关键路径**：数据通路中数据流通最长的路径。
 - ◆ 通常是**最复杂指令**的数据通路。
 - ◆ 例如：Load指令 $R[rd] \leftarrow M[R[rs1] + imm]$
- 在给出的9条指令中，**lw**指令周期最复杂。
 - 寄存器堆读取
 - ALU执行
 - 存储器读操作，作为组合逻辑电路。即：
 - ◆ 地址有效后经过“access time”，输出开始有效。
 - 寄存器写操作，作为时序逻辑电路。即：
 - ◆ 时钟到达前，输入需保持**建立时间setup**；时钟到达后经**Clk-to-Q延迟时间**，写入数据到达输出端。

课后使用LLM了解访问存储器、访问Cache、访问寄存器堆的时间量级。



lw指令周期

2、取指令时间

R[rd] ← M[R[rs1]+imm]

1、PC锁存延迟(Clk-to-Q)

③ 3、指令译码单元延迟



5、ALU延迟 6、存储器取数时间 7、寄存器建立时间 8、下地址计算 9、时钟偏移

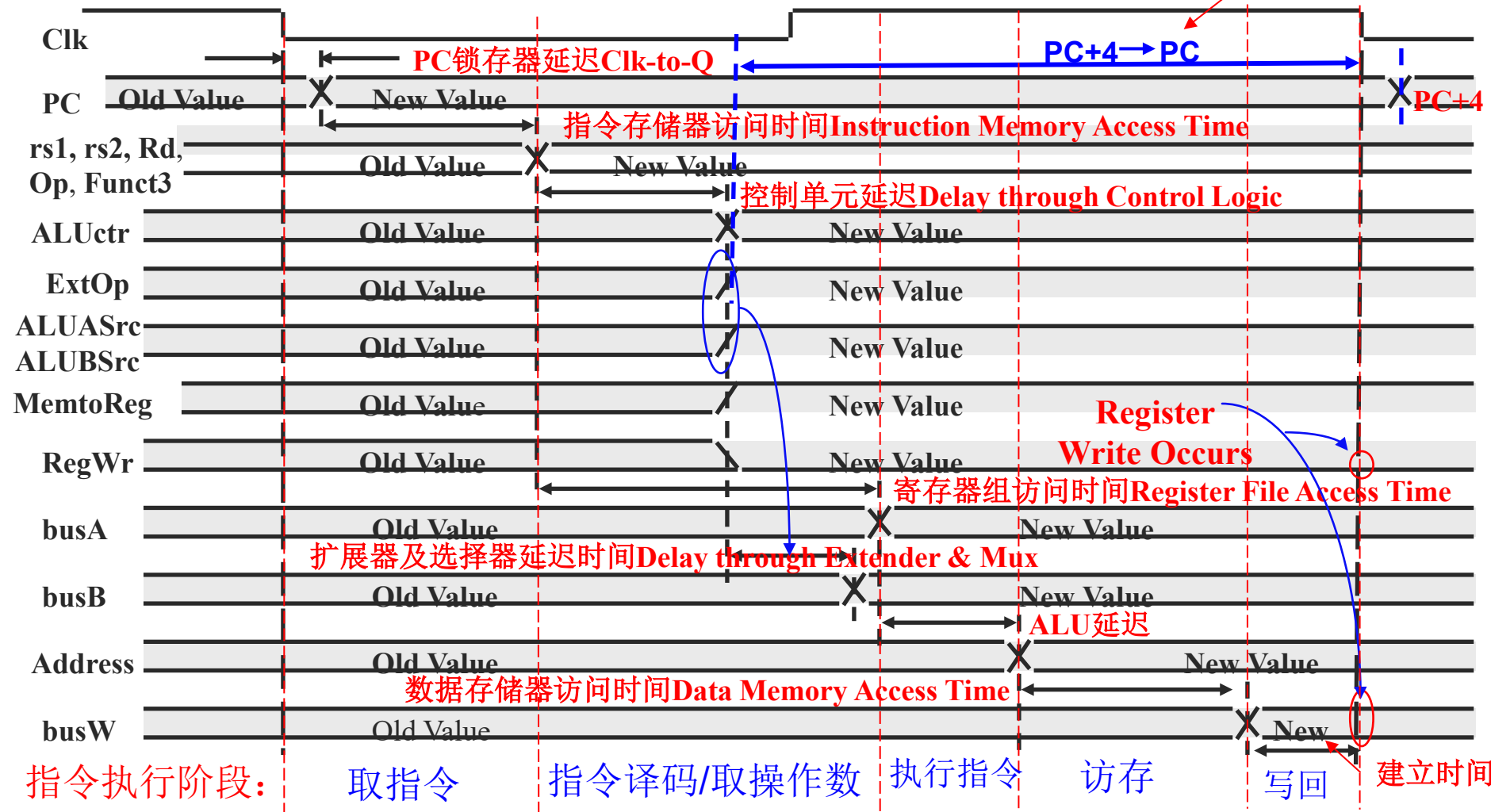
1w指令周期=①PC锁存延迟(Clk-to-Q)+②取指令时间+④寄存器取数时间+⑤ALU延迟
+⑥存储器取数时间+⑦寄存器建立时间+⑨时钟偏移



2.4 时钟周期的确定

跳转指令下地址计算什么时候完成？

- lw指令的执行时间**最长**，所花费时间可作为时钟周期



其它指令没有这么多阶段，可以在更短的时间内完成！



2.4 时钟周期的确定

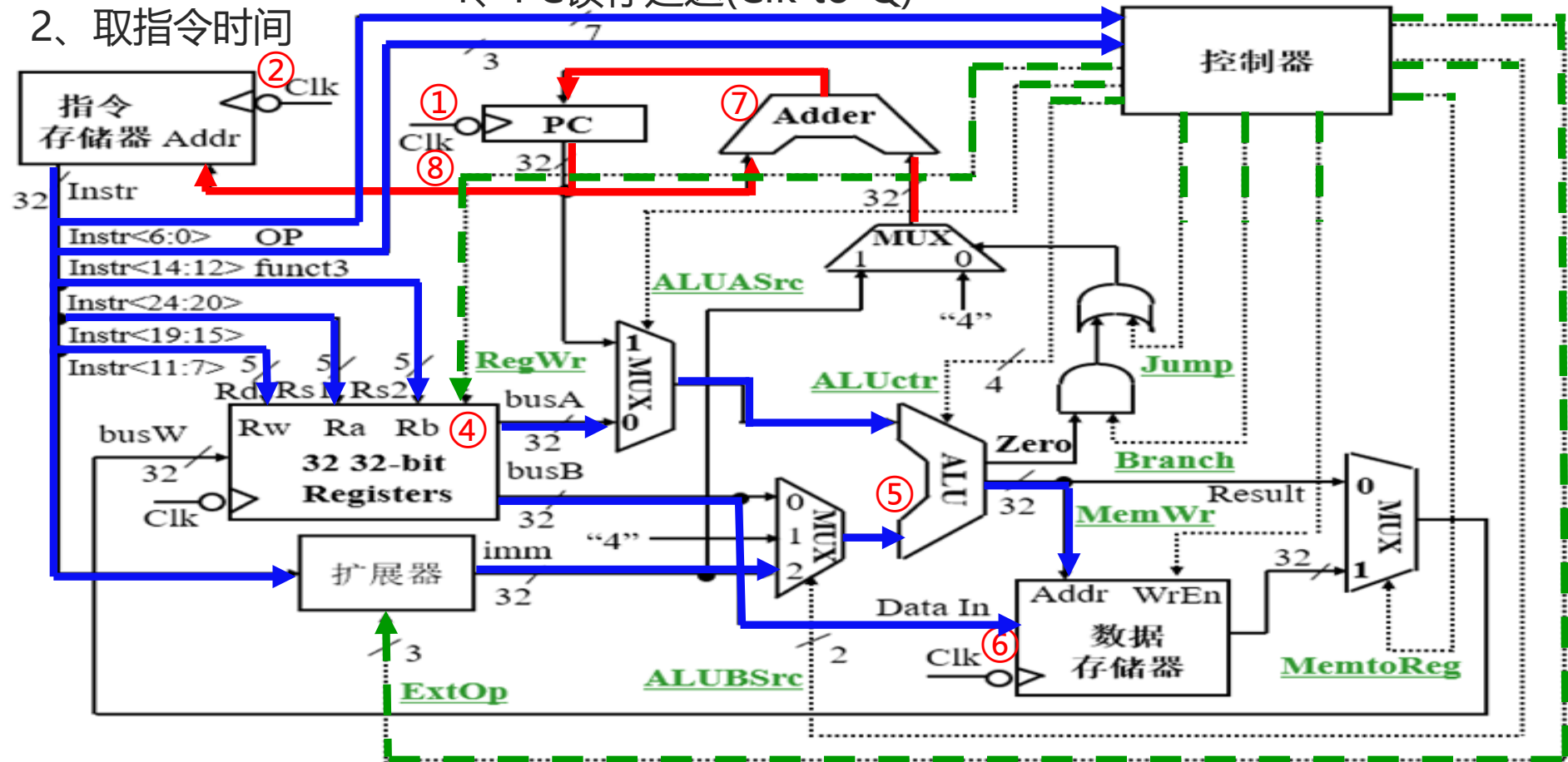
sw指令周期

2、取指令时间

$M[R[rs1]+SEXT(imm12)] \leftarrow R[rs2]$

1、PC锁存延迟(Clk-to-Q)

③ 3、指令译码单元延迟



4、寄存器取数时间 5、ALU延迟 6、存储器写数时间 7、PC=PC+4 8、时钟偏移

sw指令周期=①+②+④+⑤+⑥+⑧



单周期计算机的性能

- 单周期处理器的CPI为多少? **CPI=1!**
 - 其他条件一定的情况下, CPI越小, 则性能越好!
 - CPI=1, 不是很好吗?
- 单周期处理器的性能会不会很好? 为什么?
 - 计算机的性能除CPI外, 还取决于**时钟周期的宽度**
 - 单周期处理器的时钟宽度为**最复杂指令的执行时间**
 - 时钟周期远远大于其他指令实际所需的执行时间, 效率低。
 - ◆ R-型指令、立即数运算指令**不需要访问主存**
 - ◆ S-型指令**不需要写寄存器** (无需Setup时间)
 - ◆ U-型指令**不需要访问主存**
 - ◆ B-型指令**不需要访问主存和写寄存器**
 - ◆ J-型指令**不需要访问主存**
 - 数据通路按序执行, 功能部件在一个时钟周期内只使用一次, 成本高、利用率低



单周期计算机的性能

- 在单周期处理器中，假设各主要功能单元的操作时间为：
 - 存储单元：200ps
 - ALU和加法器：100ps
 - 寄存器堆（读/写）：50ps
 - 假设MUX、控制单元、PC、扩展器和传输线路都没有延迟，则下面实现方式中，哪个更快？快多少？
 - (1) 每条指令在一个固定长度的时钟周期内完成
 - (2) 每条指令在一个时钟周期内完成，但时钟周期仅为指令所需，也即为可变的（实际不可行，只是为了比较）
- 假设程序中各类指令占比：25%取数、10%存数、45%ALU、15%分支、5%跳转

在这些物理限制下，谁是木桶的最短板？



单周期计算机的性能

解：CPU执行时间=指令条数 x CPI x 时钟周期=指令条数 x 时钟周期
两种方案的指令条数都一样，CPI都为1，所以只要比较时钟周期宽度即可。

Instruction class	Functional units used by the instruction class				
R-type	Instruction fetch	Register access	ALU	Register access	
Load word	Instruction fetch	Register access	ALU	Memory access	Register access
Store word	Instruction fetch	Register access	ALU	Memory access	
Branch	Instruction fetch	Register access	ALU		
Jump	Instruction fetch				

各类指令要求的时间长度为：

Instruction class	Instruction memory	Register read	ALU operation	Data memory	Register write	Total
R-type	200	50	100	0	50	400 ps
Load word	200	50	100	200	50	600 ps
Store word	200	50	100	200		550 ps
Branch	200	50	100	0		350 ps
Jump	200					200 ps



单周期计算机的性能

对于方式（1），时钟周期由最长指令来决定，应该是load指令，为600ps
对于方式（2），时钟周期取各条指令所需时间，时钟周期从600ps至200ps
根据各类指令的频度，计算出平均时钟周期长度为：

CPU时钟周期

$$= 600 \times 25\% + 550 \times 10\% + 400 \times 45\% + 350 \times 15\% + 200 \times 5\% = 447.5 \text{ps}$$

$$\text{CPU性能比} = \frac{\text{方式(1)的CPU执行时间}}{\text{方式(2)的CPU执行时间}} = \frac{\text{方式(1)的CPU 时钟周期数}}{\text{方式(2)的CPU 时钟周期数}} = \frac{600}{447.5} = 1.34$$

由此可见，可变时钟周期的性能是定长周期的1.34倍！

但是，对每类指令采用可变长时钟周期实现非常困难，而且所带来的额外开销会很大，不合算！

早期的小指令集计算机用过单周期实现技术，但现代计算机都不采用。

单周期CPU带来了极简的组合逻辑控制，但最终败给了物理延迟。
要打破极限，必须“**砍断长路径**”，将**指令执行切片**。

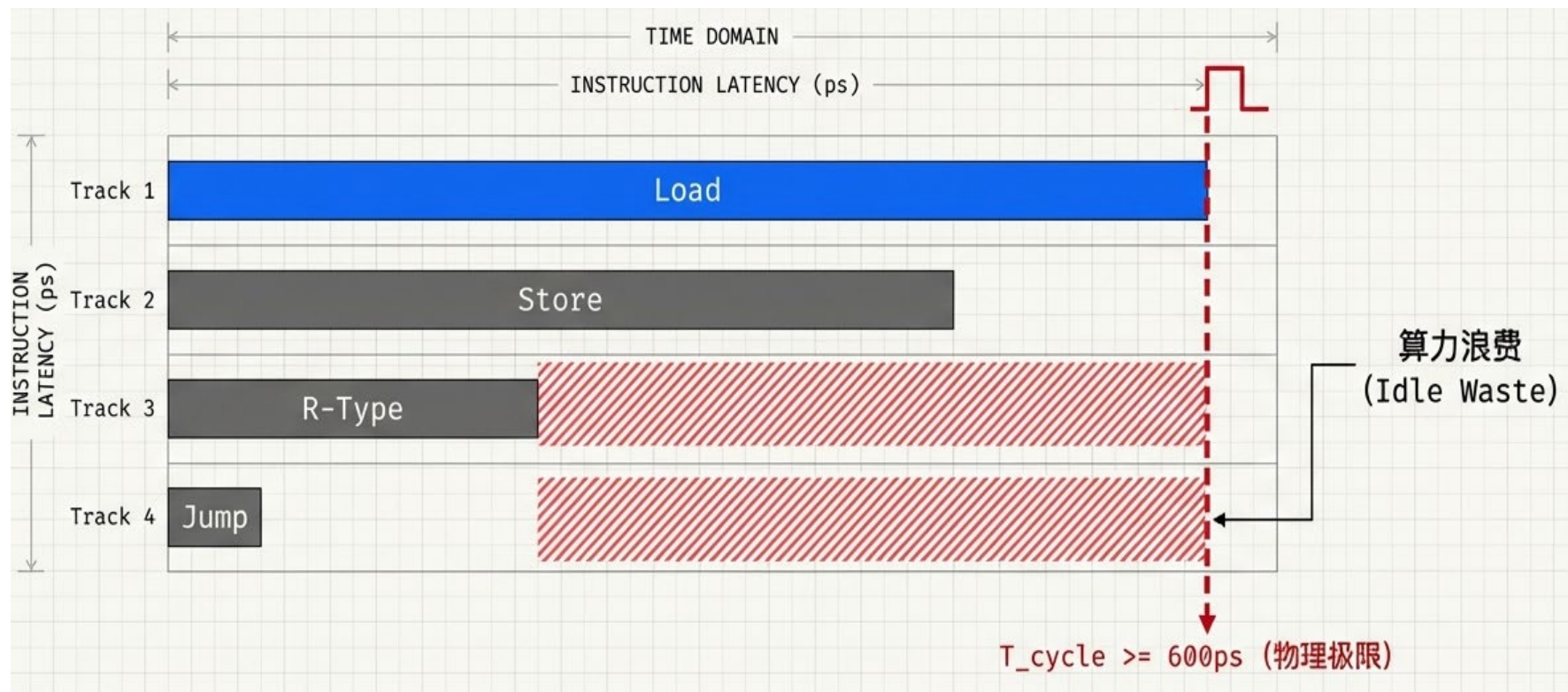
下一讲介绍多周期数据通路和控制器，其特点是：

时钟周期固定、时钟周期数可变



单周期的致命妥协

■ 木桶效应与闲置代价



尽管CPI=1,但时钟周期必须向最慢的指令妥协。
执行快指令时, 大量执行时间与算力被严重浪费。



2.3 单周期CPU设计小结

- 考察每条指令在单周期数据通路中的执行过程
 - 每条指令在一个时钟周期内完成
 - 每个时钟到来时，都开始进入取指令操作
 - ◆ 经过clk-to-Q, PC得到新值, 经过access time后得到当前指令
 - ◆ 按三种方式分别计算下条指令地址, 在branch / zero / jump的控制下, 选择其中之一送到PC输入端, 但不会马上写到PC中, 一直到下个时钟到达时, 才会更新PC。三种下址方式为:
 - $\text{branch}=\text{jump}=0$: $\text{PC}+4$
 - $\text{branch}=\text{zero}=1$: $\text{PC}+\text{SEXT}(\text{imm12})\times 2$ (即ImmB)
 - $\text{jump}=1$: $\text{PC}+\text{SEXT}(\text{imm20})\times 2$ (即ImmJ)
 - 指令取出后被译码, 产生指令对应的控制信号
 - ◆ R-型指令: rd为目的寄存器, 无访存操作,
 - ◆ ori指令: rd为目的寄存器, 符号扩展, 无访存操作,
 - ◆ lw指令: rd为目的寄存器, 符号扩展, 计算地址、读主存,
 - ◆ sw指令: rs2为源寄存器, 符号扩展, 计算地址、写主存,
- 控制器设计简单: 汇总每条指令控制信号的取值, 生成真值表, 写出逻辑表达式, 设计控制器逻辑。 (组合逻辑电路)

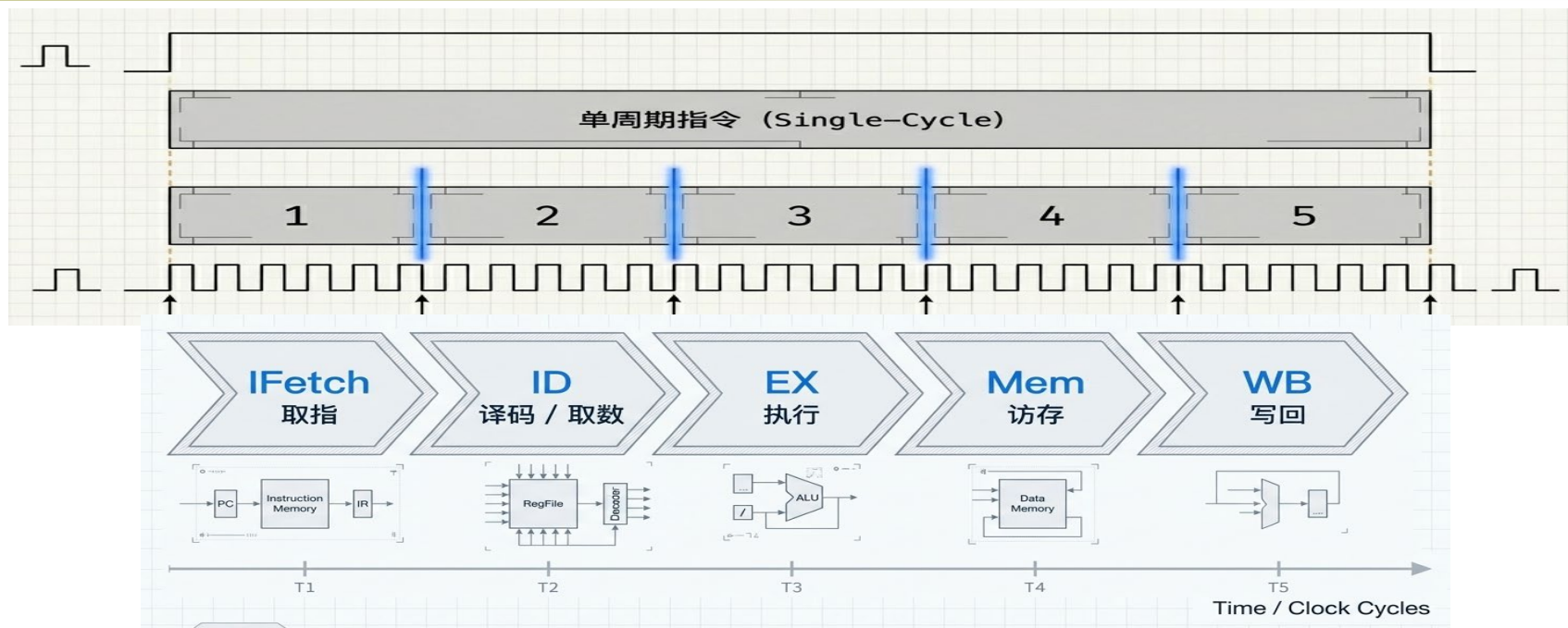


多周期处理器与有限状态机

打破单周期时序瓶颈：分时复用与硬件时空权衡



多周期设计哲学：化整为零的时间切片



核心转变

将指令执行切分为短阶段，每个时钟周期只完成一个阶段。
时钟周期不再由最慢的指令决定，而是由**最慢的单一部件**决定！

为什么可以节省时间？

并非所有指令都需要走完这5个阶段。

数据如何传递？

利用**暂存寄存器**把当前阶段的输出传递到下一阶段。



3 多周期CPU设计

■ 多周期CPU设计思路:

- 把指令的执行分成**多个阶段**，每个阶段在一个**时钟周期**内完成。
 - ◆ 时钟周期以**最复杂阶段所花时间**为准；
 - ◆ 尽量分成**时间大致相等**的若干阶段；
 - ◆ 规定每个阶段**最多只能完成1次访存** 或 **寄存器堆读/写** 或 **ALU运算**。
- 每个阶段都设置有**状态单元（寄存器）**，**执行结果**保存到状态单元中。
 - ◆ 用于前后阶段之间数据交互。

■ 实现方法

- **硬件复用**：节约硬件成本。
 - ◆ **存储器合并**：**指令存储器**和**数据存储器**合并成一个存储器。
 - ◆ 地址计算和算术运算共享同一个ALU。
- **暂存寄存器**传递数据
- **总线相连**：数据总线DB（指令/数据），地址总线AB，控制总线CB
 - ◆ 利用**三态门**作为总线仲裁。
- **有限状态机**实现控制器。

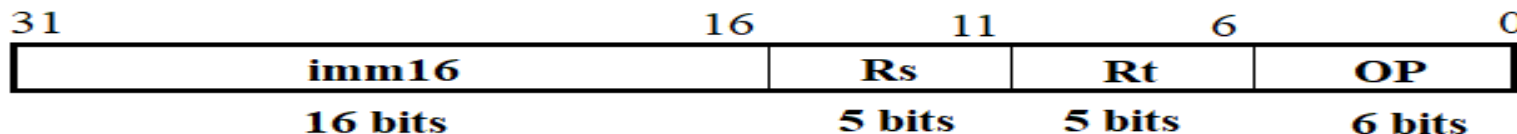
■ 成效

- 时钟周期**短**



3.1 多周期数据通路的设计

- 多周期处理器实现目标：一个32位定长指令系统



- 只有一种指令格式，可实现以下5类指令功能

- ① R-型： $R[Rt] \leftarrow R[Rs] \text{ op } R[Rt]$ ，两个寄存器内容运算，结果送寄存器。
- ② I-型运算： $R[Rt] \leftarrow R[Rs] \text{ op } \text{EXT}[imm16]$ ，寄存器内容和立即数运算。
- ③ Load： $R[Rt] \leftarrow M[R[Rs] + \text{SEXT}[imm16]]$ ，地址为寄存器内容加立即数。
- ④ Store： $M[R[Rs] + \text{SEXT}[imm16]] \leftarrow R[Rt]$ ，地址为寄存器内容加立即数。
- ⑤ Jump指令： $PC \leftarrow PC + \text{SEXT}[imm16]$ ，跳转目标地址为PC内容加立即数。

- **EXT[imm16]**：对16位立即数imm16进行扩展，转换为32位操作数。对于逻辑运算和无符号整数算术运算，采用**零**扩展；对于带符号整数算术运算，则采用**符号位**扩展。
- **SEXT[imm16]**：符号扩展。



3.1 多周期数据通路的设计

- 多周期处理器中，假设一条指令的执行过程由**三个阶段**组成。
 - 指令的执行过程包含下列**阶段**：
 - (1) **取指令并计算下条指令地址**（递增、跳转、返回）
 - (2) **译码并取数**
 - (3) **执行指令**
 - **取指令、指令译码并取数**是所有指令译码前的相同操作阶段。
- 在相应**控制信号**的引导下完成每个阶段的功能。
 - 不同阶段之间通过**寄存器**来传递数据。
 - ◆ 取指令阶段：指令寄存器IR
 - ◆ 译码取数阶段：存储器地址寄存器MAR
 - ◆ 执行阶段：ALU结果寄存器，标志寄存器FLAG，存储器数据寄存器MDR
- 多周期数据通路中新增的控制信号主要有：
 - 三态门使能信号：**用于区分地址总线上是指令地址还是数据地址。**
 - ◆ 2个，PCout与MARout，互斥
 - 写入使能信号：**用于新增5个寄存器以及PC寄存器。**
 - ◆ 6个+单周期2个
 - 多路选择控制：5个



3.1 多周期数据通路的设计

- 简单指令系统对应的多周期数据通路 **系统总线：AB+DB+CB**

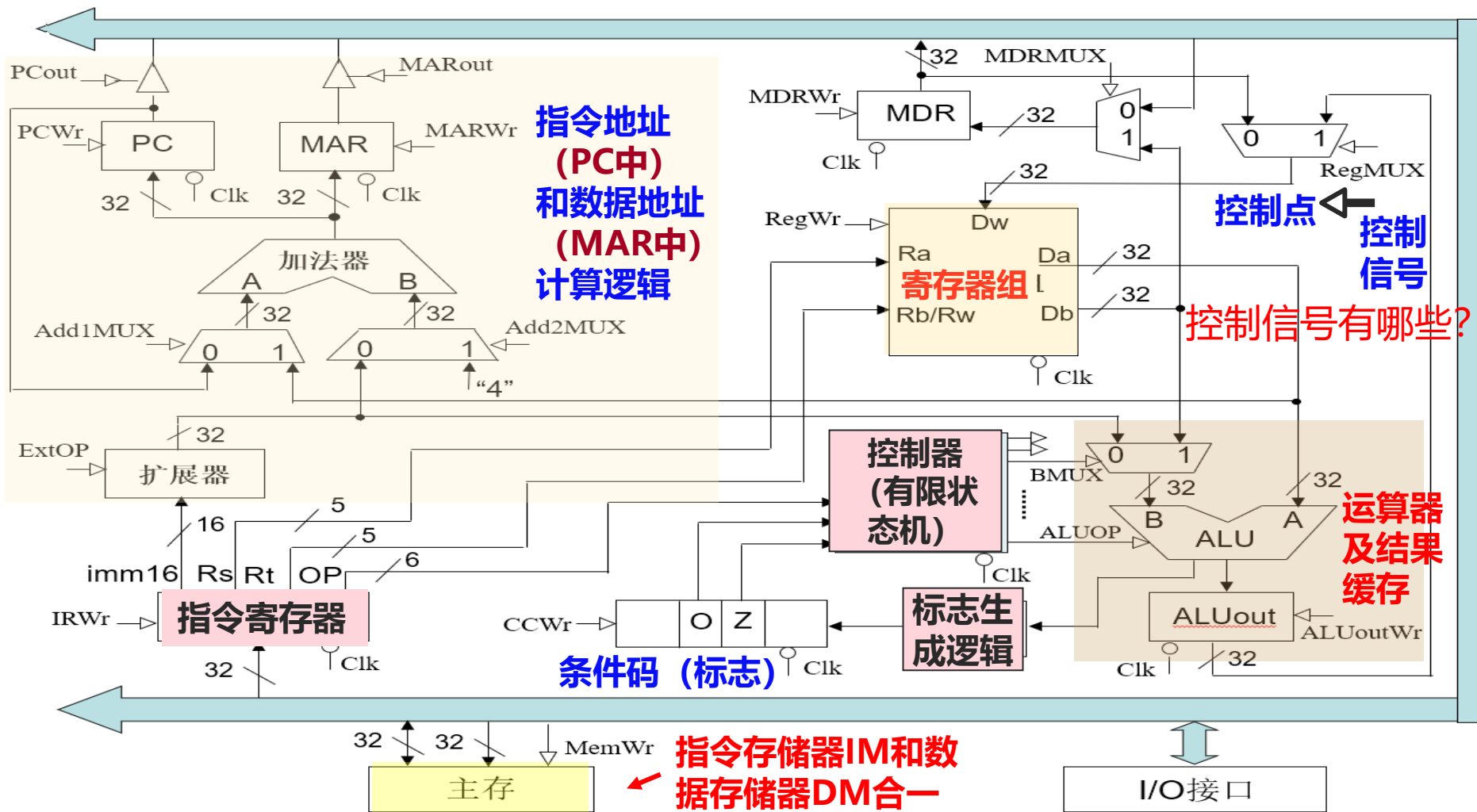
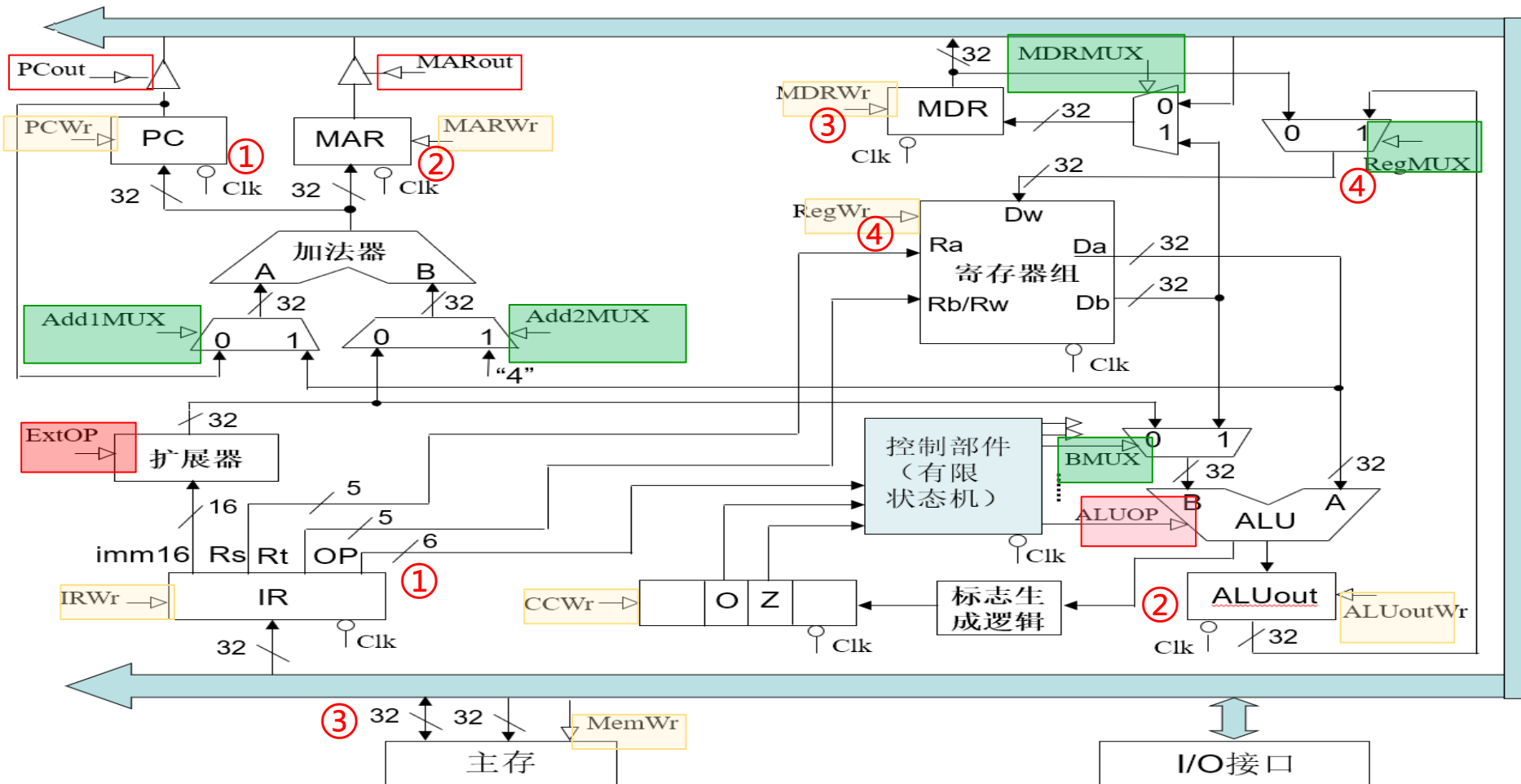


图8.31



3.1 多周期数据通路的设计

■ 多周期数据通路中的控制信号：5类



控制信号：三态门使能、写入使能、多路选择控制、立即数扩展控制、ALU操作控制



3.1 多周期数据通路的设计

(1) 取指令阶段，记为 IFetch

功能：根据PC读取指令存储器并保存到指令寄存器IR中；计算下地址逻辑：递增 $PC \leftarrow PC + 4$ 。

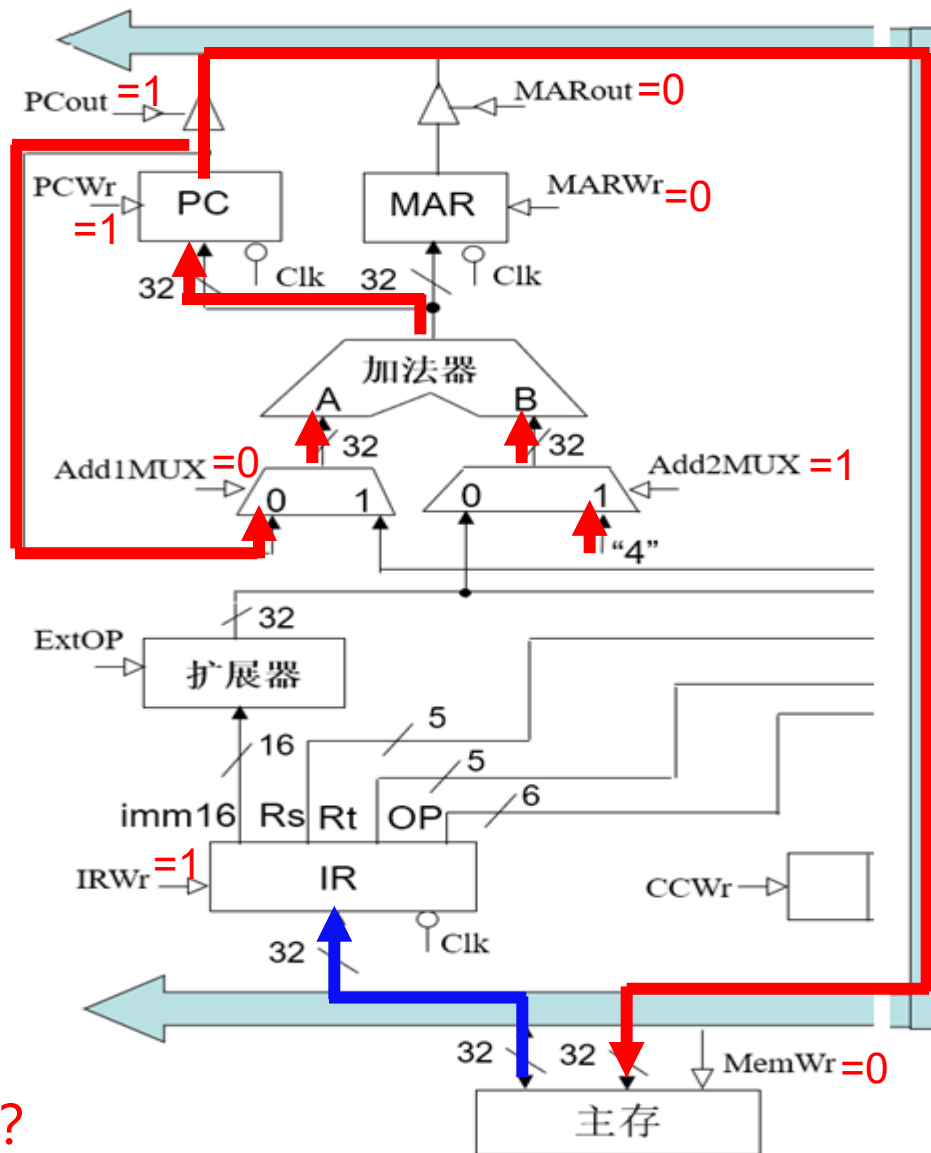
有效控制信号及其取值如下：

- $R[IR] \leftarrow M[PC]$ ：PCout=1、MARout=0、MemWr=0、IRWr=1（不是每个时钟都能更新）
- $PC \leftarrow PC + 4$ ：Add1MUX=0、Add2MUX=1、PCWr=1（不是每个时钟都能更新）
- 其他写使能信号（如MARWr、CCWr、MDRWr、ALUoutWr、RegWr）全部为0

PC+4会不会写入MAR?

PC值会不会持续加4?

PC+4现阶段写入对那些类型指令有影响?





3.1 多周期数据通路的设计

(2) 译码并取数,
记为 RFecth/ID

功能: IR的OP字段送控制器译码, 并根据Rs和Rt读取寄存器中数据。imm16字段送到扩展器输入端。

加法器空闲, 投机计算主存地址。

有效控制信号取值为:

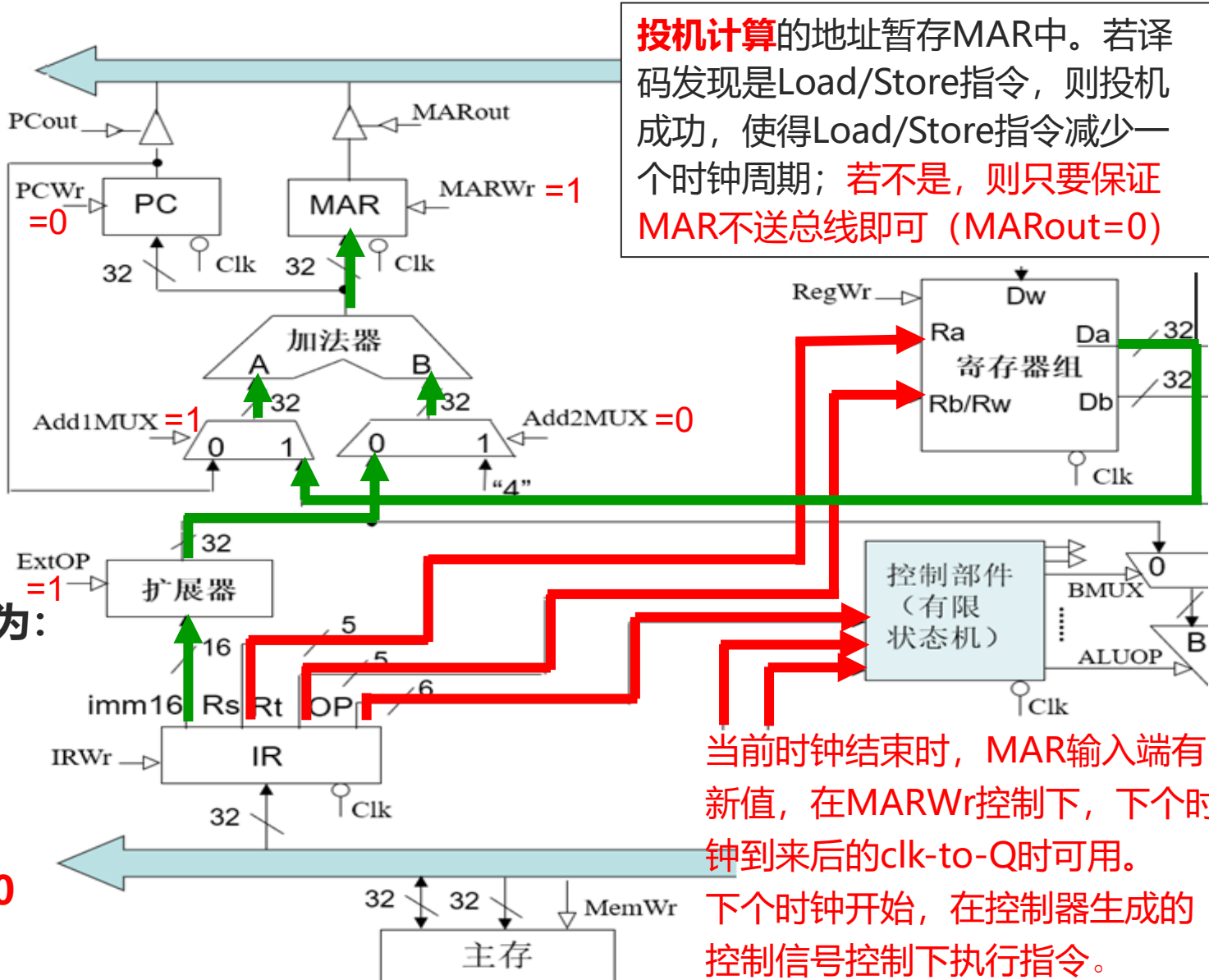
ExtOP=1(符扩)、

Add1MUX=1、

Add2MUX=0、

MARWr=1、

其他写使能信号为0



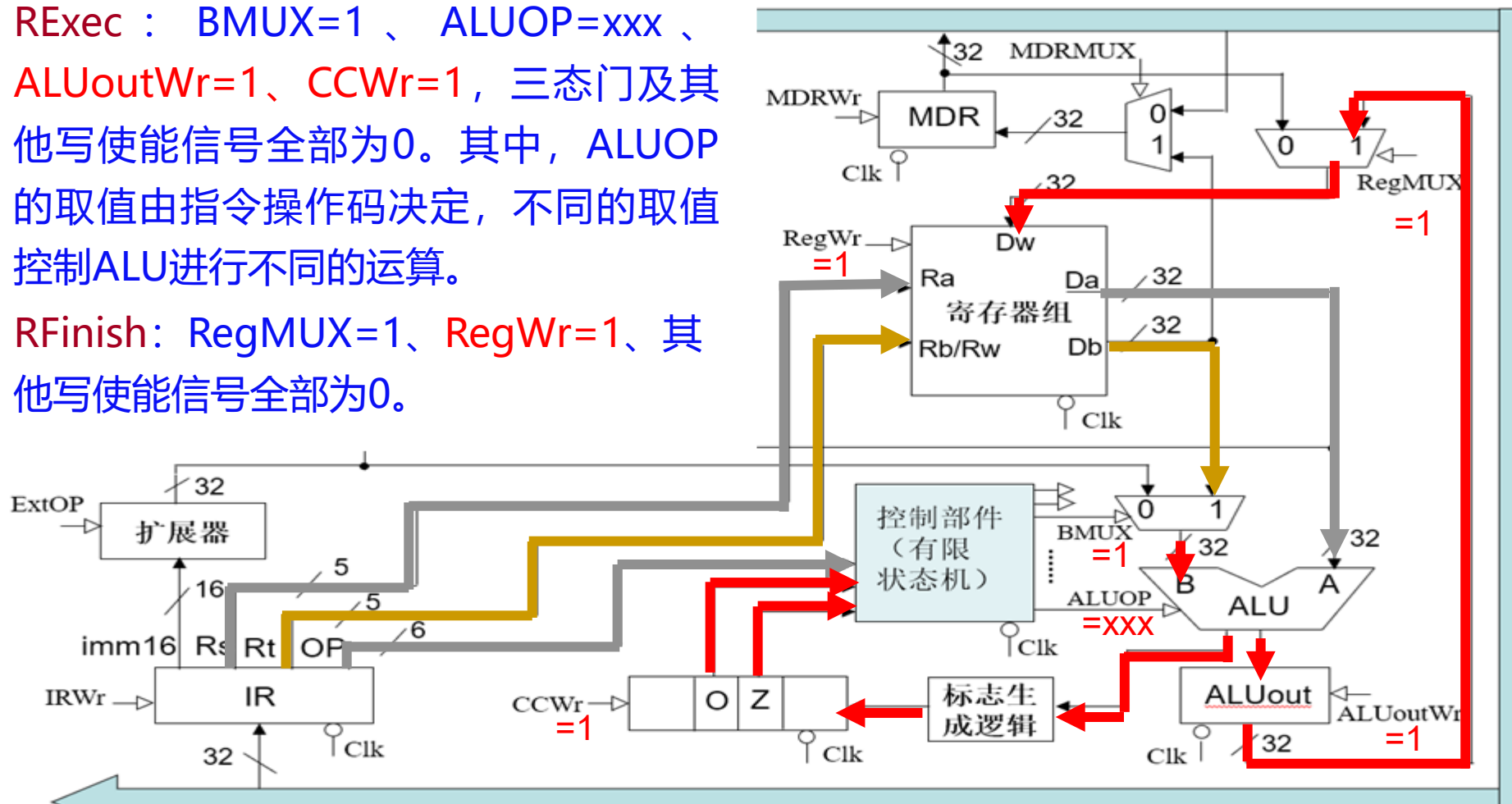


3.1 多周期数据通路的设计

(3) 执行指令：R-型指令，需要两个时钟周期，记为 RExec、RFinish 状态
功能： $R[Rt] \leftarrow R[Rs] \text{ op } R[Rt]$ ，结果分别写入 Rt 和 CC 寄存器。

RExec：BMUX=1、ALUOP=xxx、
ALUoutWr=1、CCWr=1，三态门及其他写使能信号全部为0。其中，ALUOP 的取值由指令操作码决定，不同的取值控制 ALU 进行不同的运算。

RFinish：RegMUX=1、RegWr=1、其他写使能信号全部为0。





3.1 多周期数据通路的设计

(3) 执行指令：I-型运算，需要两个时钟周期，记为 IExec、IFinish 状态

功能： $R[Rt] \leftarrow R[Rs] \text{ op } EXT[imm16]$ ，结果分别写入 Rt 和 CC 寄存器。

IExec: ExtOP=0或1、BMUX=0、

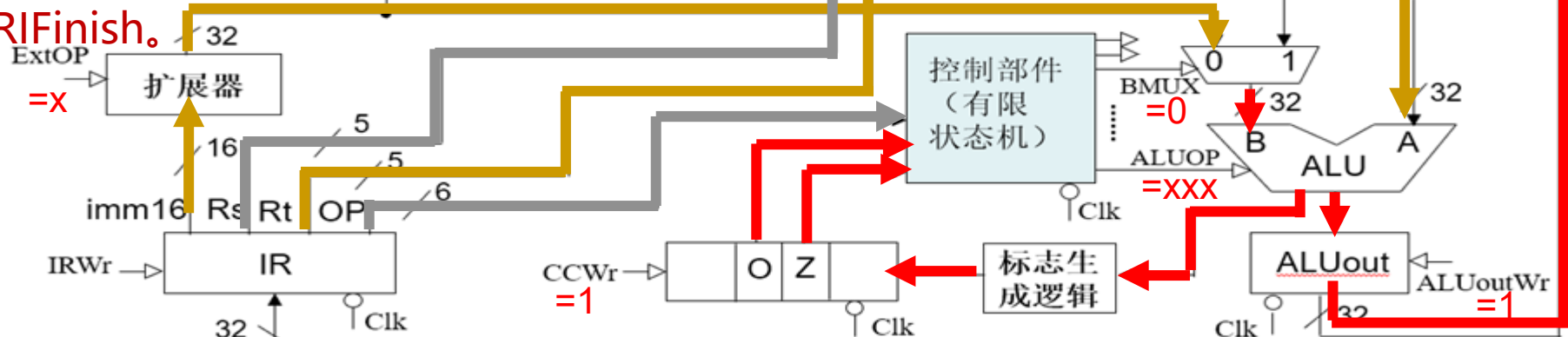
ALUOP=xxx、ALUoutWr=1、CCWr=1，

三态门及其他写使能信号全部为0。ExtOP 和 ALUOP 取值由指令操作码决定。

IFinish: RegMUX=1、RegWr=1、其他写使能信号全部为0。

IFinish 和 RFinish 两个状态功能完全一样，因此，可以将两个状态合并成一个状态：

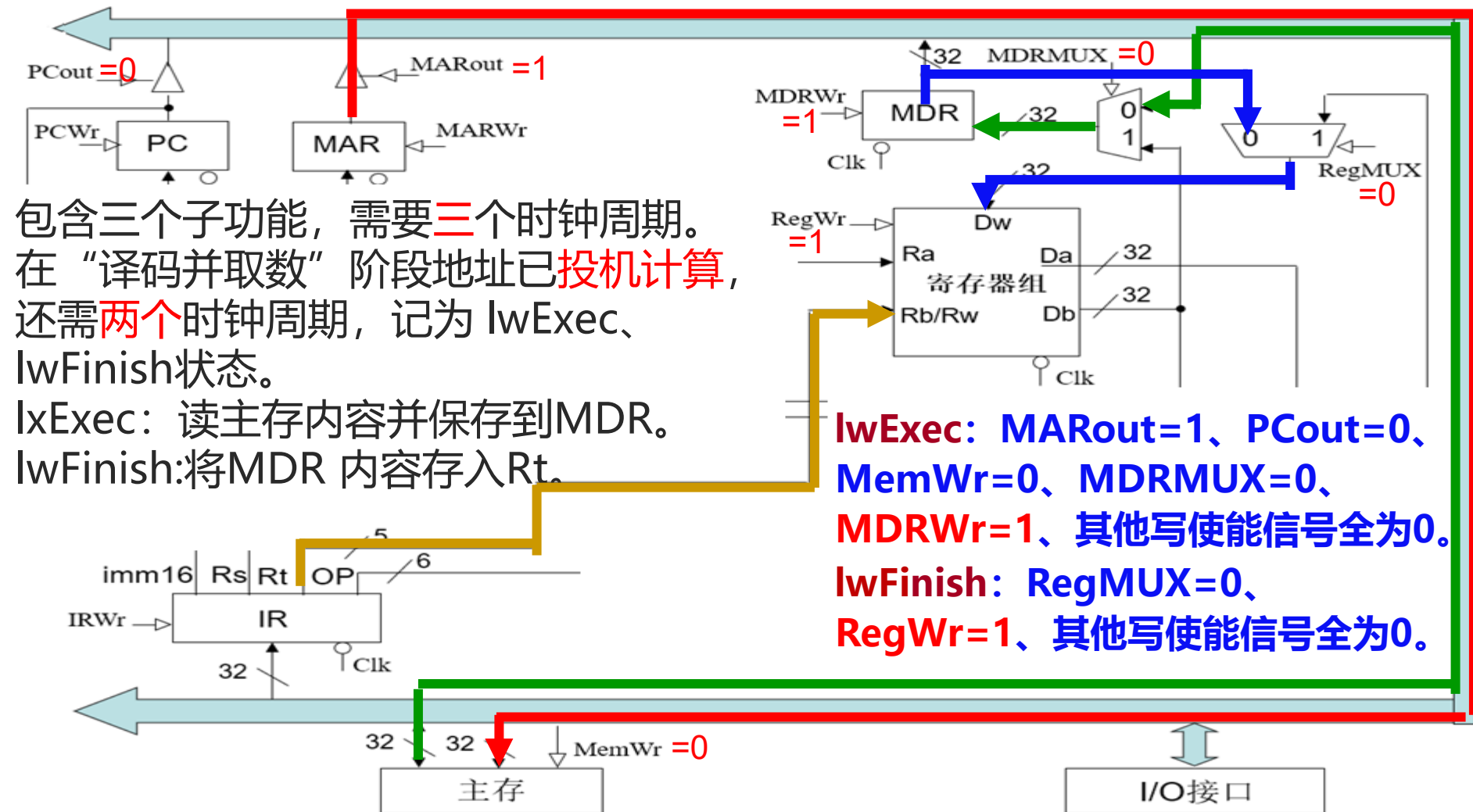
RIFinish。





3.1 多周期数据通路的设计

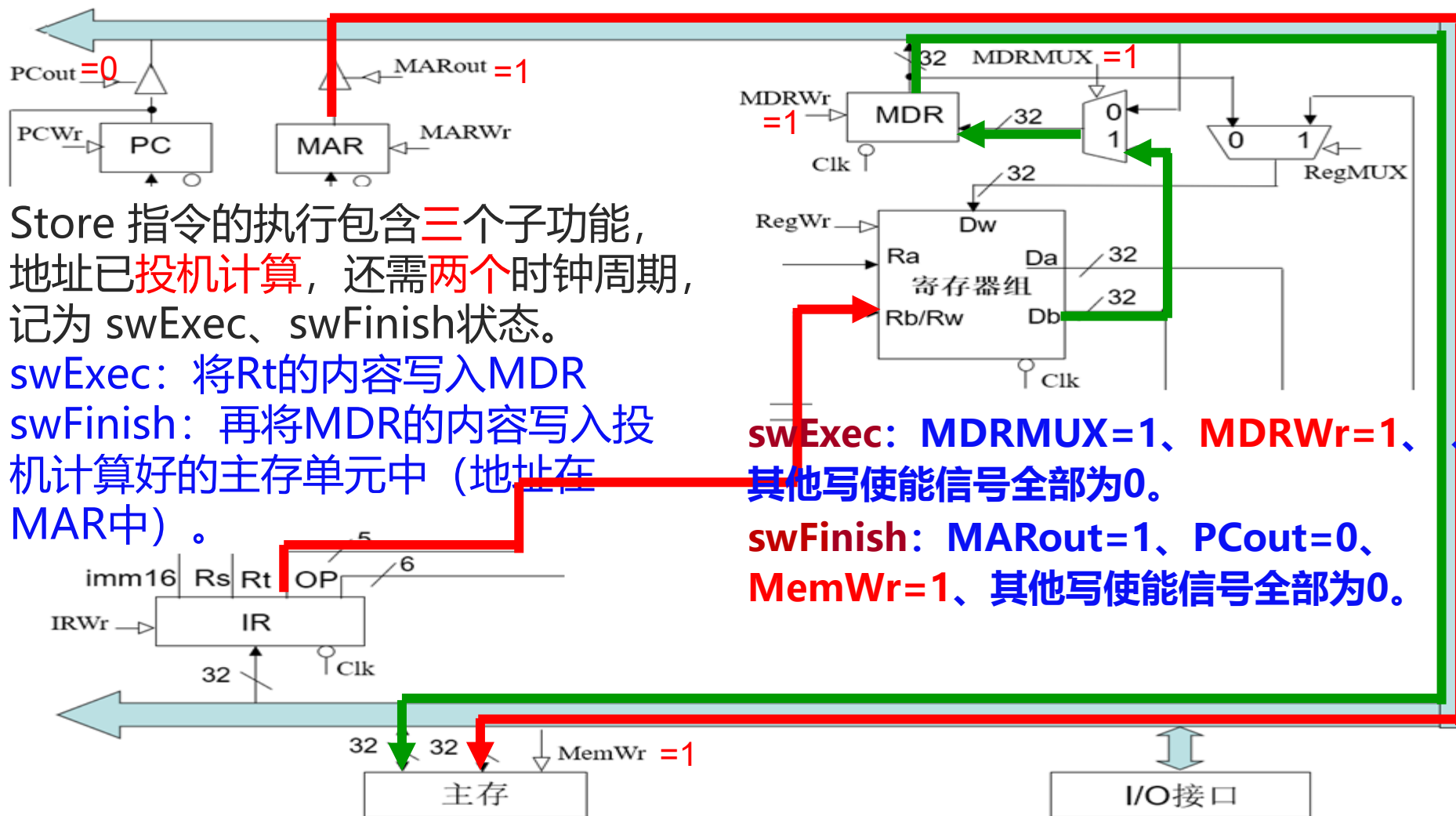
(3) 执行指令：Load指令， $R[Rt] \leftarrow M[R[Rs] + \text{SEXT}[imm16]]$





3.1 多周期数据通路的设计

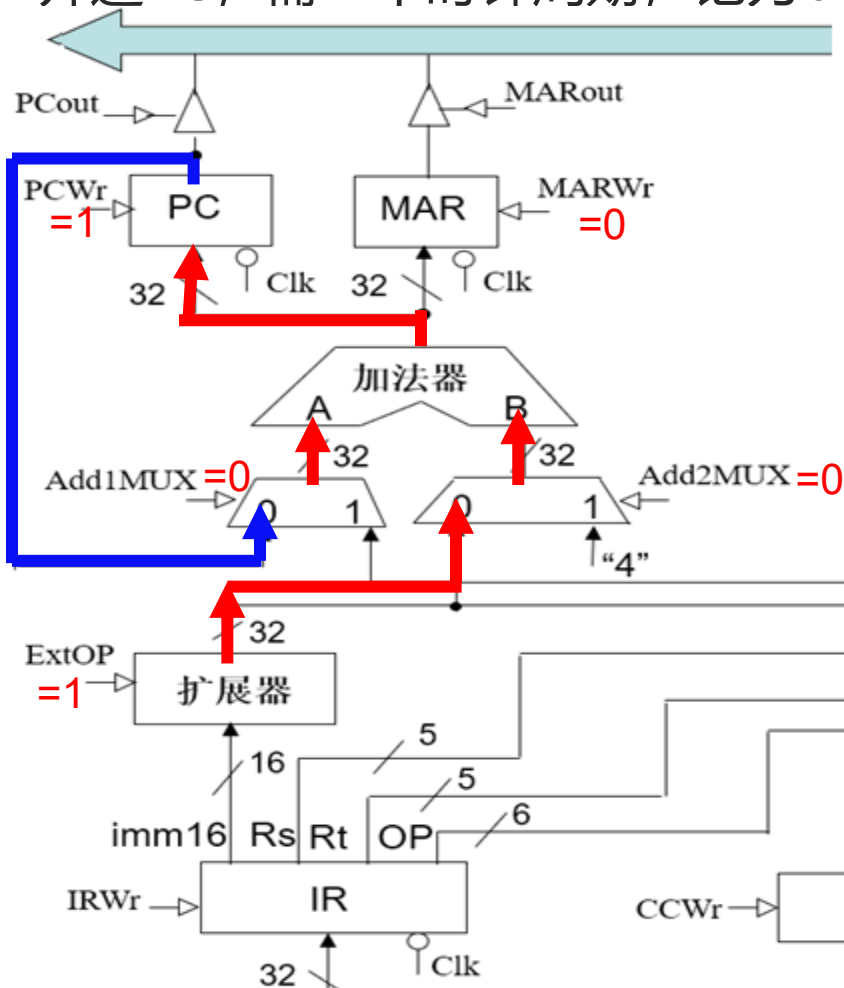
(3) 执行指令：Store 指令， $M[R[Rs] + \text{SEXT}[imm16]] \leftarrow R[Rt]$





3.1 多周期数据通路的设计

(3) 执行指令：Jump指令：计算转移目标地址 ($PC + \text{SEXT}(\text{imm16})$) 并送PC，需一个时钟周期，记为 JFinish 状态。

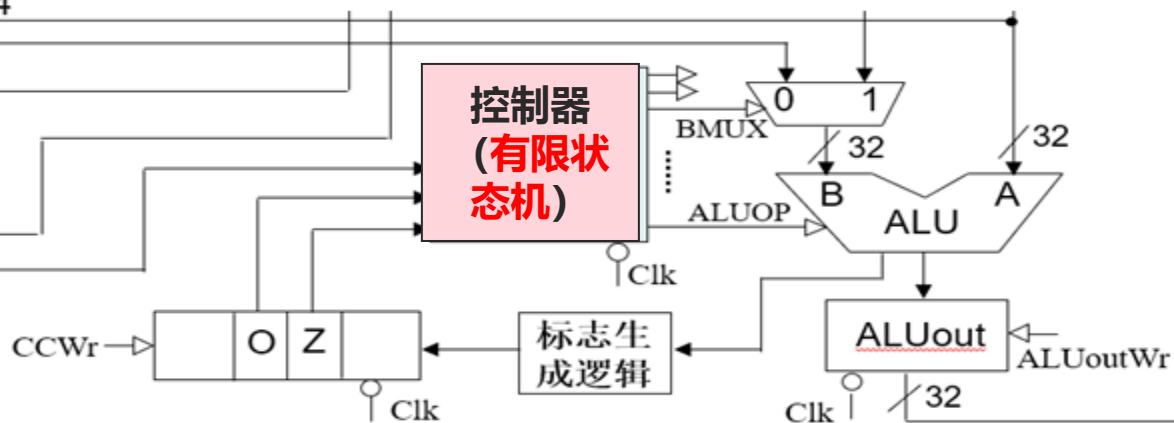


JFinish: ExtOP=1、
Add1MUX=0、Add2MUX=0、
PCWr=1、其他写使能信号全部为0。

是否存在问题？ $PC + 4 + \text{SEXT}(\text{imm6})$

如何解决？
1、第3个周期更新PC
2、PC-4再加上立即数

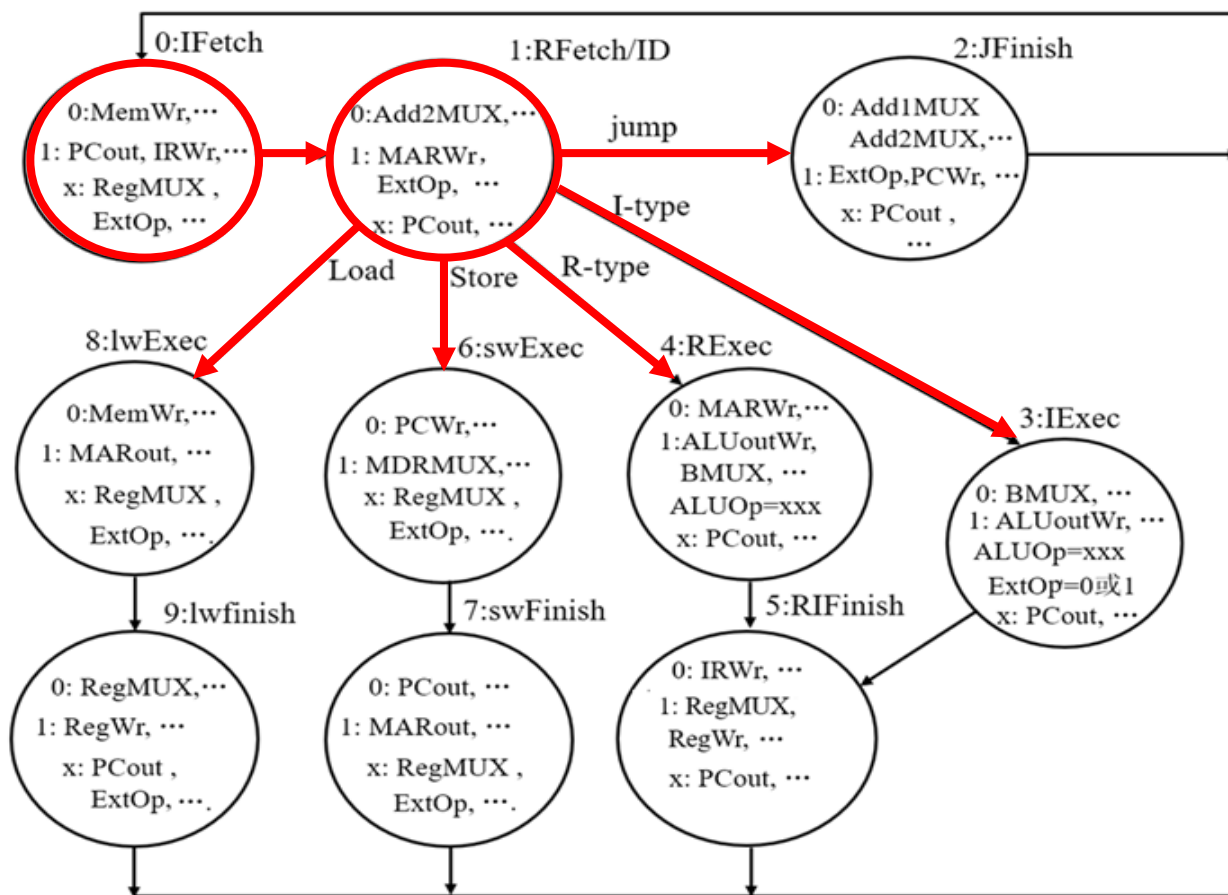
设计控制部件支持完成上述指令功能！





3.1 多周期数据通路的设计

- 综合前面各指令执行过程，得到状态转换图。



每来一个时钟，进入下一个状态

状态0和1是公共操作，状态1后译码得到不同的指令

指令的时钟数多少？

R型：4, I型：4, lw：4, sw：4, Jump：3

下一目标：设计状态转换电路，即：控制器



3.2 硬连线控制器设计

- 多周期数据通路中每个指令有多个时钟周期，每个周期控制信号取值不同！
 - 不能像设计单周期控制器那样用真值表直接描述的方式。
- 多周期控制器通常采用下列**两种**方式来实现：
 1. 基于**有限状态机**描述
 - ◆ 采用这种方式实现的控制器结构，由两部分组成：一个**组合逻辑控制器**和一个**状态寄存器**。
 - ◆ 通常用PLA 电路实现组合逻辑控制器，这种控制器也称为组合逻辑控制器或PLA 控制器或**硬连线控制器**。
 2. **微程序描述**
 - ◆ 仿照程序设计方法，将每条指令的执行过程用一个**微程序**来表示，每个微程序由若干条**微指令**组成，每条微指令相当于有限状态机中的一个**状态**。所有指令对应的微程序都存放在一个ROM 中，这个ROM 称为**控制存储器** (Control Storage, CS)，简称控存，控存中的信息称为微代码。



3.2 硬连线控制器设计

■ 多周期控制器

初始表示

Finite State Diagram

Microprogram

顺序控制

Explicit Next State
Function

Microprogram counter
+ Dispatch ROMs

逻辑表示

Logic Equations

Truth Tables

实现技术

PLA

ROM

“hardwired control”

*“microprogrammed
control”*

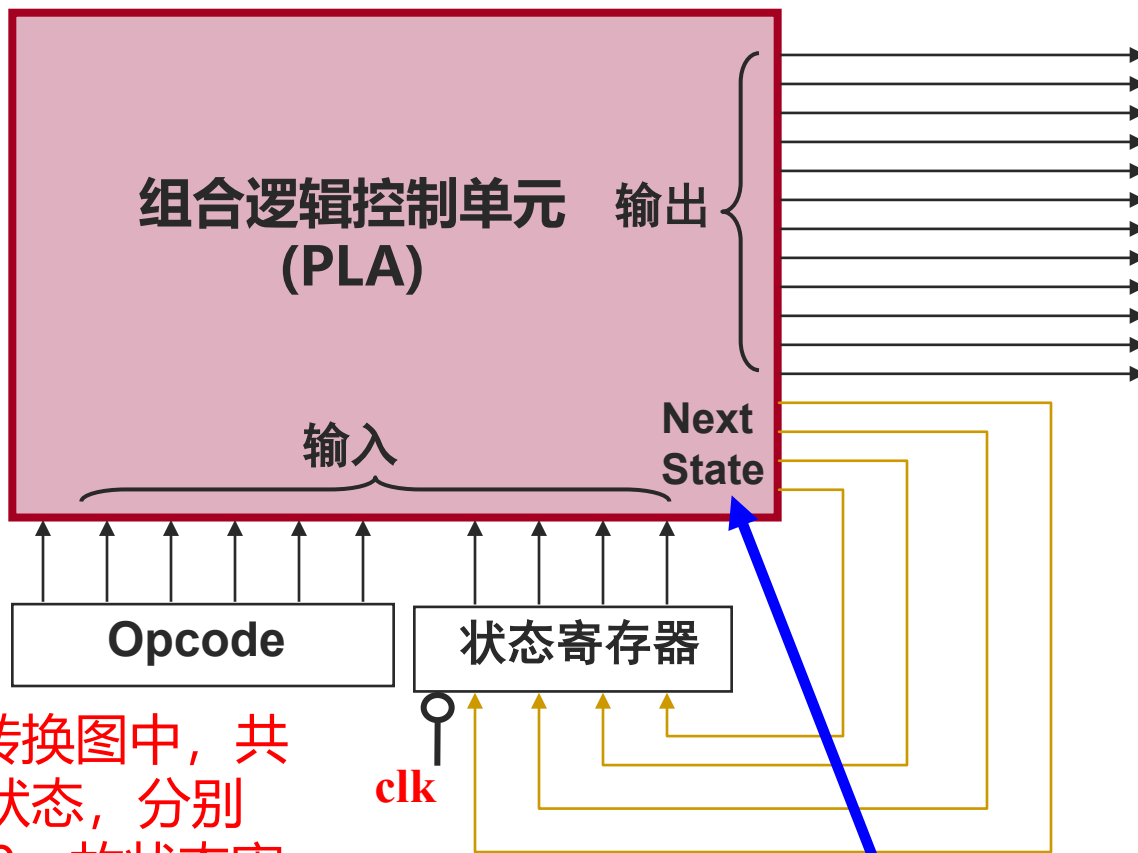
硬连线路控制器
(硬布线控制器)

微程序控制器



3.2 硬连线控制器设计

- 由时钟、当前状态和操作码确定下一状态。不同状态输出不同控制信号值
- 控制逻辑采用“摩尔型”时序电路方式，即：**输出函数仅依赖于当前状态**



状态转换图中，共10个状态，分别为0~9，故状态寄存器至少需4位

下一步目标：设计控制逻辑
(control Logic)

**Multicycle
Datapath**

下一状态被看成和其他控制信号一样。

下一状态是当前状态和操作码的函数。

每来一个时钟，当前状态变到下一个状态

在不同状态下输出不同的控制信号。



3.2 硬连线控制器设计

■ 多周期控制器状态转换表

当前状态 $S_3S_2S_1S_0$	指令操作码 $OP_5OP_4OP_3OP_2OP_1OP_0$	下一状态 $NS_3NS_2NS_1NS_0$
State0 (IFetch): 0000		0 0 0 1
State1 (RFetch/ID): 0001	000010 (jump)	0 0 1 0
State1 (RFetch/ID): 0001	001101 (ori)	0 0 1 1
State1 (RFetch/ID) : 0001	000000 (R-type)	0 1 0 0
State1 (RFetch/ID): 0001	101011 (sw)	0 1 1 0
State1 (RFetch/ID) : 0001	100011 (lw)	1 0 0 0
State3 (IExec) : 0011		0 1 0 1
State4 (RExec) : 0100		0 1 0 1
State6 (swExec) : 0110		0 1 1 1
State8 (lwExec) : 1000		1 0 0 1
State2、5、7、9: 0010,0101,0111,1001		0 0 0 0



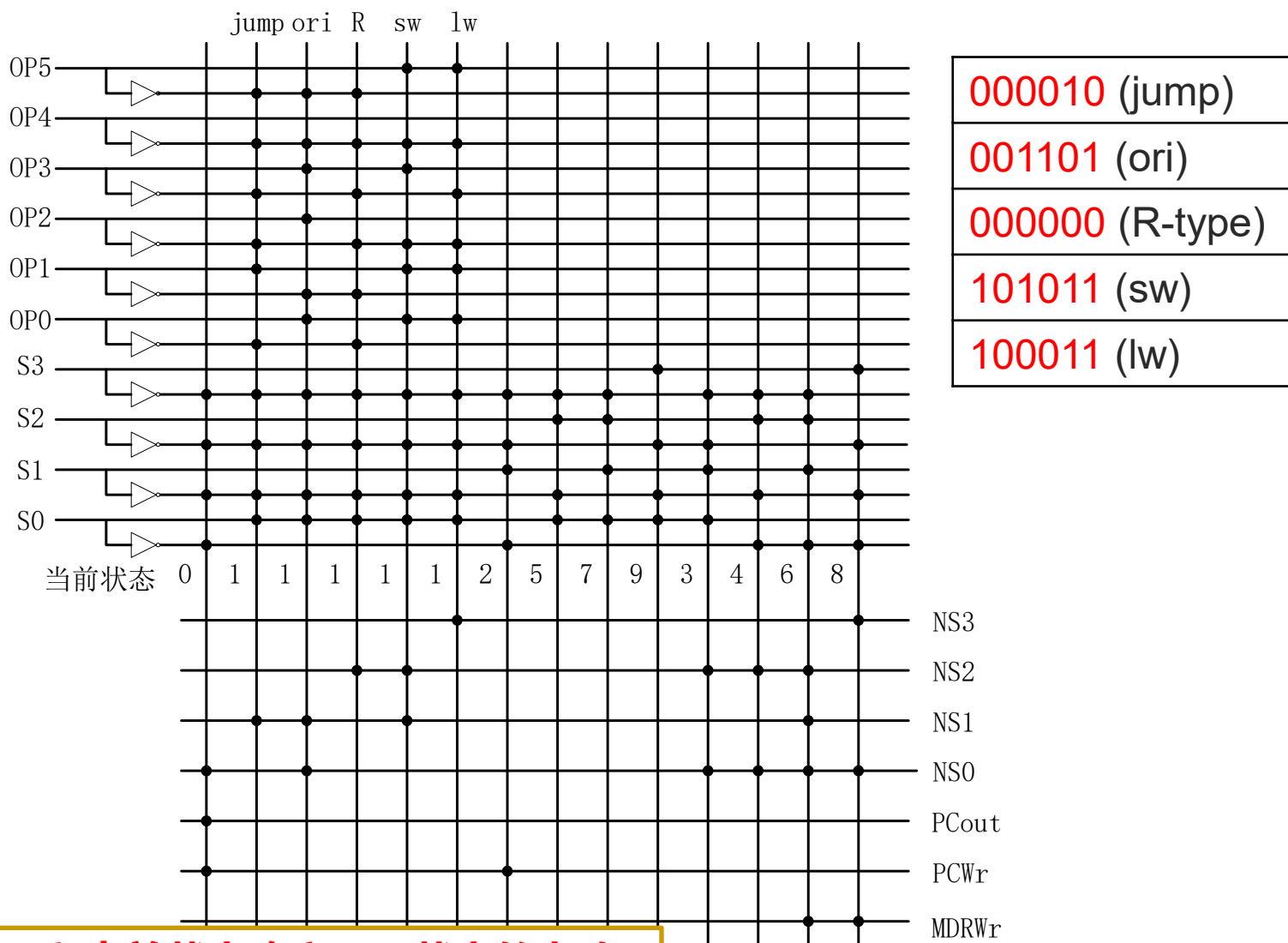
3.2 硬连线控制器设计

■ 多周期控制器控制信号表（下地址计算在取指令阶段完成）

执行状态	取指令 IFecth	译码并取数 RFecth/ID	R指令执行 RExec	R指令结束 RFinish	I指令执行 IExec	I指令结束 IFinish	Load执行 lwExec	Load结束 lwFinish	Store执行 swExec	Store结束 xwFinish	Jump结束 Jfinish
状态编号	0	1	4	5	3	5	8	9	6	7	2
PCout	1	0	0	0	0	0	0	0	0	0	0
PCWr	1	0	0	0	0	0	0	0	0	0	1
MARout	0	0	0	0	0	0	1	0	0	1	0
MARWr	0	1	0	0	0	0	0	0	0	0	0
MemWr	0	0	0	0	0	0	0	0	0	1	0
IRWr	1	0	0	0	0	0	0	0	0	0	0
Add1MUX	0	1	0	0	0	0	0	0	0	0	0
Add2MUX	1	0	0	0	0	0	0	0	0	0	0
ExtOp	0	1	0	0	0/1	0	0	0	0	0	1
BMUX	0	0	1	0	0	0	0	0	0	0	0
ALUOP	*	*	op	*	op	*	*	*	*	*	*
ALUoutWr	0	0	1	0	1	0	0	0	0	0	0
CCWr	0	0	1	0	1	0	0	0	0	0	0
RegMUX	0	0	0	1	0	1	0	0	0	0	0
RegWr	0	0	0	1	0	1	0	1	0	0	0
MDRMUX	0	0	0	0	0	0	0	0	1	0	0
MDRWr	0	0	0	0	0	0	1	0	1	0	0



用PLA电路实现控制单元（硬布线方式）



左上角：由操作码和当前状态确定下一状态的电路
右下角：由当前状态确定控制信号的电路



3.2 硬连线控制器设计

- 硬连线控制器的特点：
 - 优点：速度快，适合于简单或规整的指令系统。
 - 缺点：它是一个多输入/多输出的巨大逻辑网络。

- 对于复杂指令系统来说，
 - 结构庞杂，实现困难；
 - 修改、维护不易；灵活性差。
 - 甚至无法用有限状态机描述！



3.3 微程序控制器设计

- 在**微程序控制器**控制下执行指令时，CPU 从控存中取出每条指令对应的**微程序**，在时钟信号的控制下，按照一定的顺序执行微程序中的每条**微指令**。
- **通常一个时钟周期执行一条微指令**。
 - 一条微指令的功能通过执行一系列基本操作来完成，这些基本操作称为**微操作**。
 - 每个微操作在相应控制信号的控制下执行，这些控制信号在微程序设计中称为**微命令**。
 - 微程序是一个微指令序列，一个微程序对应一条**机器指令的功能**。

最初把固化在ROM的微程序称**固件 (Firmware)**，表示用软件实现的硬部件，现在对固件通俗的理解是在ROM中“固化的软件”。

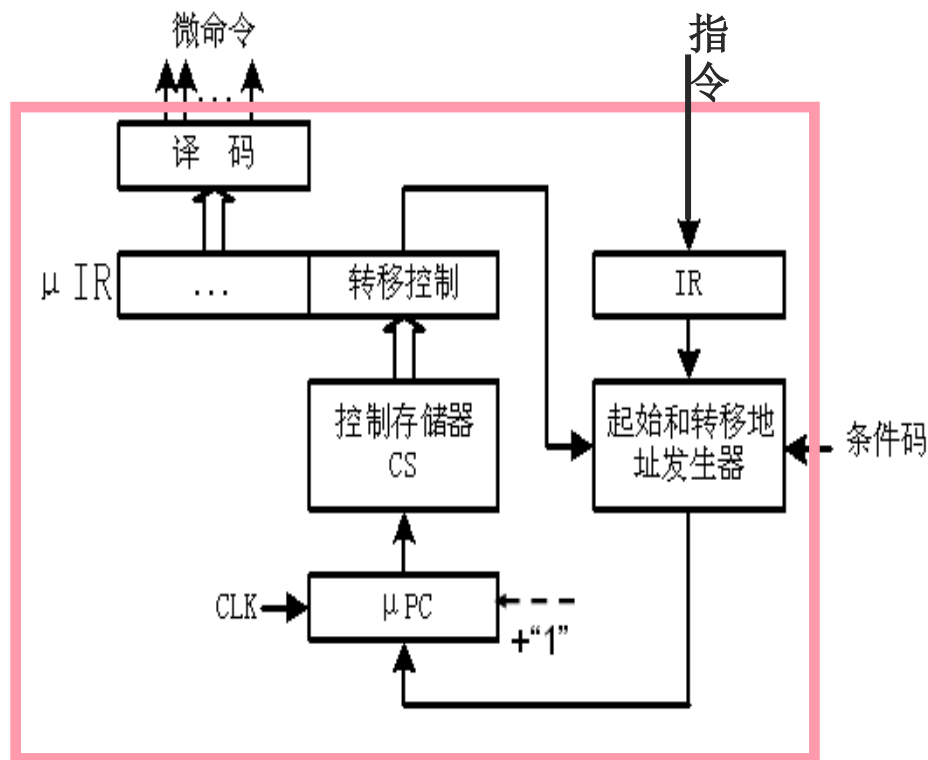
微程序设计的特点：具有规整性、可维性和灵活性，但速度慢。



3.3 微程序控制器设计

■ 微程序控制器的基本结构

1. 输入：指令、条件码
2. 输出：控制信号(微命令)
3. 核心：控存CS
4. μPC ：指出将要执行的微指令在CS中的位置
5. μIR ：正在执行的微指令
6. 每个时钟执行一条微指令
7. 微程序第一条微指令地址由起始地址发生器产生
8. 顺序执行时， $\mu PC + 1$
9. 转移执行时，由转移控制字段指出对哪些条件码进行测试，转移地址发生器根据条件码修改 μPC





3.3 微程序控制器设计

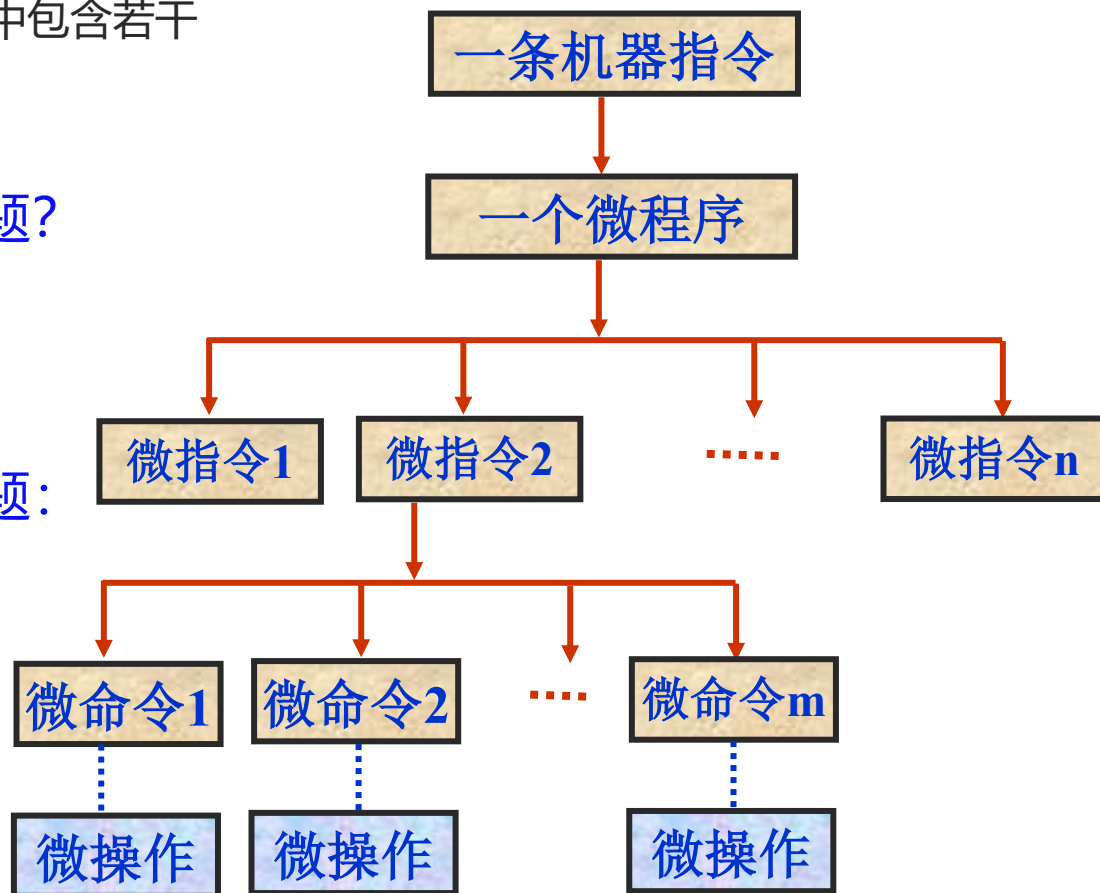
- 将指令的执行转换为微程序的执行。
 - 微程序是一个**微指令序列**。
 - 每条微指令是一个0/1序列，其中包含若干个**微命令**（即：控制信号）。
 - 微命令控制数据通路的执行。

- 控制程序执行要解决什么问题？

- (1) 指令如何译码、执行
 - (2) 下条指令到哪里去取

- 微程序执行也要解决两个问题：

- (1) 微指令如何对微命令编码
 - (2) 下条微指令在哪里





3.3 微程序控制器设计

1、微指令格式的设计：定长

- 若干微命令、下条微指令地址（可选）、常数（可选）

微指令格式：



μOP : 微操作码字段，产生微命令；

μAddr : 微地址码字段，产生下条微指令地址。

- 微指令格式设计风格取决于微操作码的编码方式（微命令：控制信号）
- 微操作码主要编码方式：
 - 不译法（直接控制法）
 - 字段直接编码（译）法



3.3 微程序控制器设计

2、下条微地址的确定方式

- 微指令地址的产生方法有两种：
 - 增量(计数器)法：下条微指令地址隐含在微程序计数器 μPC 中。
 - 断定(下址字段)法：在本条微指令中明显指定下条微指令地址。
- 选择下条要执行的微指令有以下四种情况：
 - 取指微程序首址：每条指令执行前，CPU先执行取指微程序。
 - 第一条微指令：每条指令取出后，必须转移到该指令对应的第一条微指令执行。
 - 顺序执行时：微程序执行过程中顺序取出下条微指令执行。
 - 分支执行时：在遇到按条件转移到不同微指令执行时，需要根据控制单元的输入来选择下条微指令。



3.3 微程序控制器设计

- 微程序相对固定，且通常不放在主存内。
- 微程序控制器的主要缺点是：比相同或相近指令系统的硬布线控制器慢。
- RISC大都采用硬连线路控制器
- 而IA-32等这种CISC则采用硬连线和微程序相结合的方式来实现控制逻辑。

在Logisim中完成RV32I多周期CPU设计，并通过验收的同学，实验成绩优秀！



单周期vs多周期

对比维度	单周期架构	多周期架构
核心瓶颈: T_{cycle}	最慢指令决定 (极长)	最慢单一部件决定 (极短)
硬件资源策略	疯狂复制组件 (浪费)	分时复用 (节约)
控制器大脑	纯组合逻辑 (无记忆)	有限状态机 (时序记忆)
CPI (每指令周期数)	严格为1	加权平均

总结:

多周期设计是一次全面的推倒重来, 用复杂的时序控制换取了极致的硬件利用率。



问题

假定程序中22%为Load, 11%为Store, 49%为R-Type, 16%为Branch, 2%为Jump。每个状态需要一个时钟周期, CPI为多少?

假设每种指令所需的时钟周期数为:

Load: 5; Store: 4; R-Type: 4; Branch: 3; Jump: 3

假设单周期时钟宽度为1, 则多周期时钟周期约为单周期的1/5。
比较两者的性能。



3.4 带异常处理的CPU 设计

- 程序执行过程中CPU会遇到一些特殊情况，使正在执行的程序被“中断”。
- CPU中止原来正在执行的程序，转到处理异常情况或特殊事件的程序去执行，执行后再返回到原被中止的程序处（断点）继续执行

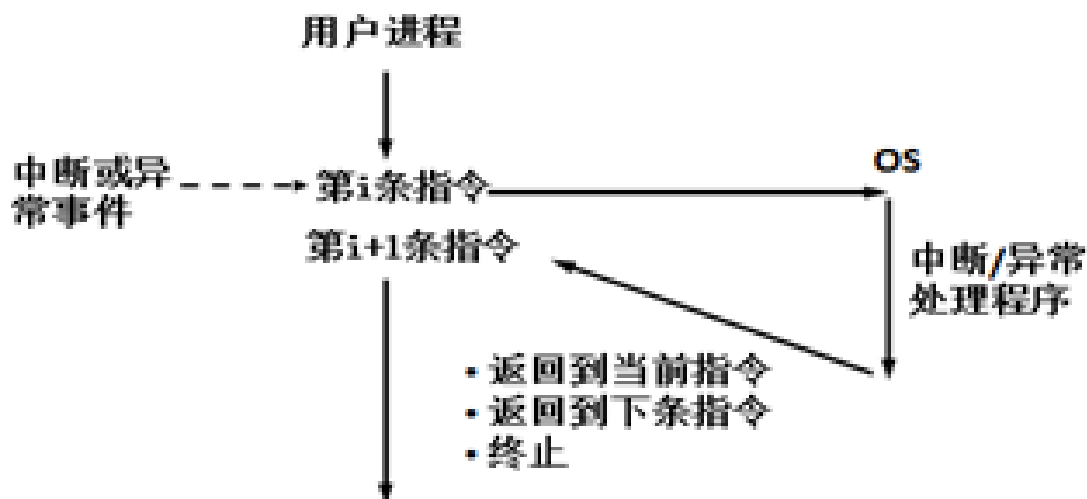


图 8.37 中断和异常处理过程



3.4 带异常处理的CPU 设计

- CPU处理异常过程分以下两个步骤：

① 保存断点和程序状态：

- 不同的异常事件，其断点（即返回地址）不同。
- 断点的值由异常类型和发生异常时PC 的值决定。
 - ◆ 故障的断点是发生故障的当前指令处： $PC \leftarrow PC - 4$
 - ◆ 自陷的断点则是自陷指令后面一条指令处
- 异常处理后需返回到被中断程序继续执行。
 - ◆ 保存断点（PC）
 - ◆ 保存被中断时原程序的状态（如产生的各种标志信息等）
- 将断点和程序状态字PSW保存到堆栈或特殊寄存器中。即：
 - ◆ $PC \rightarrow$ 栈（X86） 或 断点寄存器（RISC-V中mepc或sepc）。
 - ◆ $PSW \rightarrow$ 栈（X86） 或 程序状态寄存器EPSWR（RISC-V中CSR寄存器）



3.4 带异常处理的CPU 设计

- CPU处理异常过程分以下两个步骤：

② 识别异常事件

- 内部异常事件和外部中断源的识别方式不同

- ◆ 内部异常事件的识别大多采用**软件识别**方式
- ◆ 外部中断源则可以采用软件识别或**硬件识别**方式

(1) 软件识别方式（RISC-V、MIPS等采用）

- ◆ CPU 中设置一个**异常原因寄存器cause**，用于记录异常的原因。
- ◆ 操作系统使用一个**统一的异常处理程序**，该程序按优先级顺序查询异常状态寄存器，识别出异常事件。

(2) 硬件识别（向量中断）（80x86采用）

- ◆ 每个异常和中断都有一个异常/中断号，根据此号，到**中断向量表**（中断描述符表）中读取对应的具体的中断服务程序的入口地址。
- ◆ RISC-V 架构也支持**通过硬件识别方式**来识别外部中断源。

- **异常和中断机制是处理器设计中最具挑战性的任务之一**

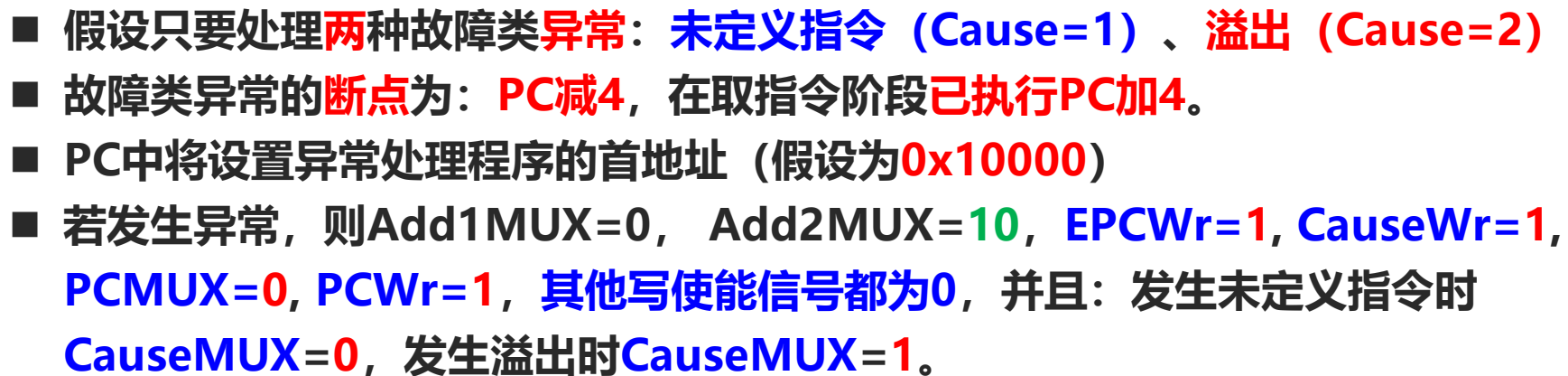


3.4 带异常处理的CPU 设计

- 以多周期处理器为例，说明带异常处理数据通路设计
- 假定其异常/中断机制中包括以下**两个**寄存器：
 - **断点寄存器EPC**：用于存放断点（异常处理后返回到的指令的地址）。
 - ◆ 写入EPC的断点可能是正在执行的指令的地址（故障时），也可能是下条指令的地址（自陷和中断时）。
 - **异常寄存器Cause**：记录异常原因。
 - ◆ 假定处理的异常类型有以下两种：
 - ◆ **未定义指令（Cause=1）、溢出（Cause=2）**
- 需要加入两个寄存器的“**写使能**”控制信号
 - **EPCWr**：在保存断点时该信号有效，使断点PC写入EPC。
 - **CauseWr**：在处理器发现异常时有效，使**异常类型**被写到Cause寄存器。
- 需要一个控制信号**CauseMUX**来选择**异常类型的值**写入到Cause寄存器中
- 需要将异常查询程序的入口地址（假设为**0x10000**）写入PC，可以在PC输入端增加一个MUX（控制信号**PCMUX**），其中一个输入为0x10000



■ 带异常处理的数据通路





3.4 带异常处理的CPU 设计

- 在有限状态机中增加异常处理的状态，每种异常占一个状态
- 每个异常处理状态中，需考虑以下基本控制

- Cause寄存器的设置
 - ◆ 控制信号：CauseMUX、CauseWr
- 计算断点处的PC值 (PC-4)，并送EPC
 - ◆ 控制信号：Add1MUX、Add2MUX、EPCWr
- 将异常查询程序的入口地址送PC
 - ◆ 控制信号：PCMUX、PCWr

- 在原来状态转换图基础上加入两个异常处理状态

- 状态10：未定义指令（非法操作码），
当指令译码发现op是一个未定义的编码。
Cause=1
- 状态11，溢出，当R-型或I-型运算类指令
在ALU中执行后，在条件码寄存器CC中的
溢出标志位OF为1时。
Cause=2

10 UndefInstr

PCMUX=0
Add1MUX=0
Add2MUX=10
CauseMUX=0
EPCWr=1
CauseWr=1
PCWr=1
.....

10 未定义指令异常状态

11 Overflow

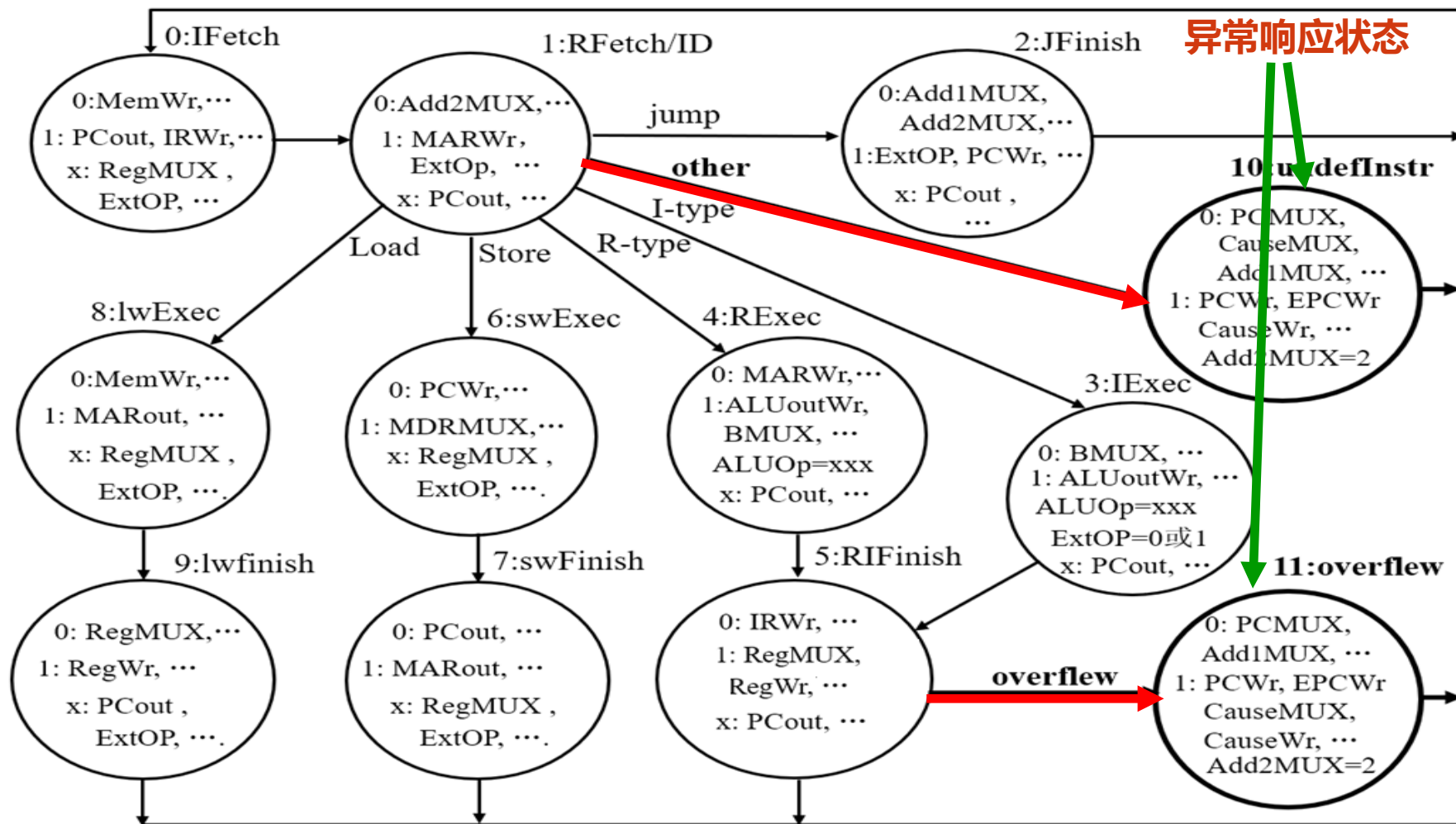
PCMUX=0
Add1MUX=0
Add2MUX=10
CauseMUX=1
EPCWr=1
CauseWr=1
PCWr=1
.....

11 溢出异常状态



3.4 带异常处理的CPU 设计

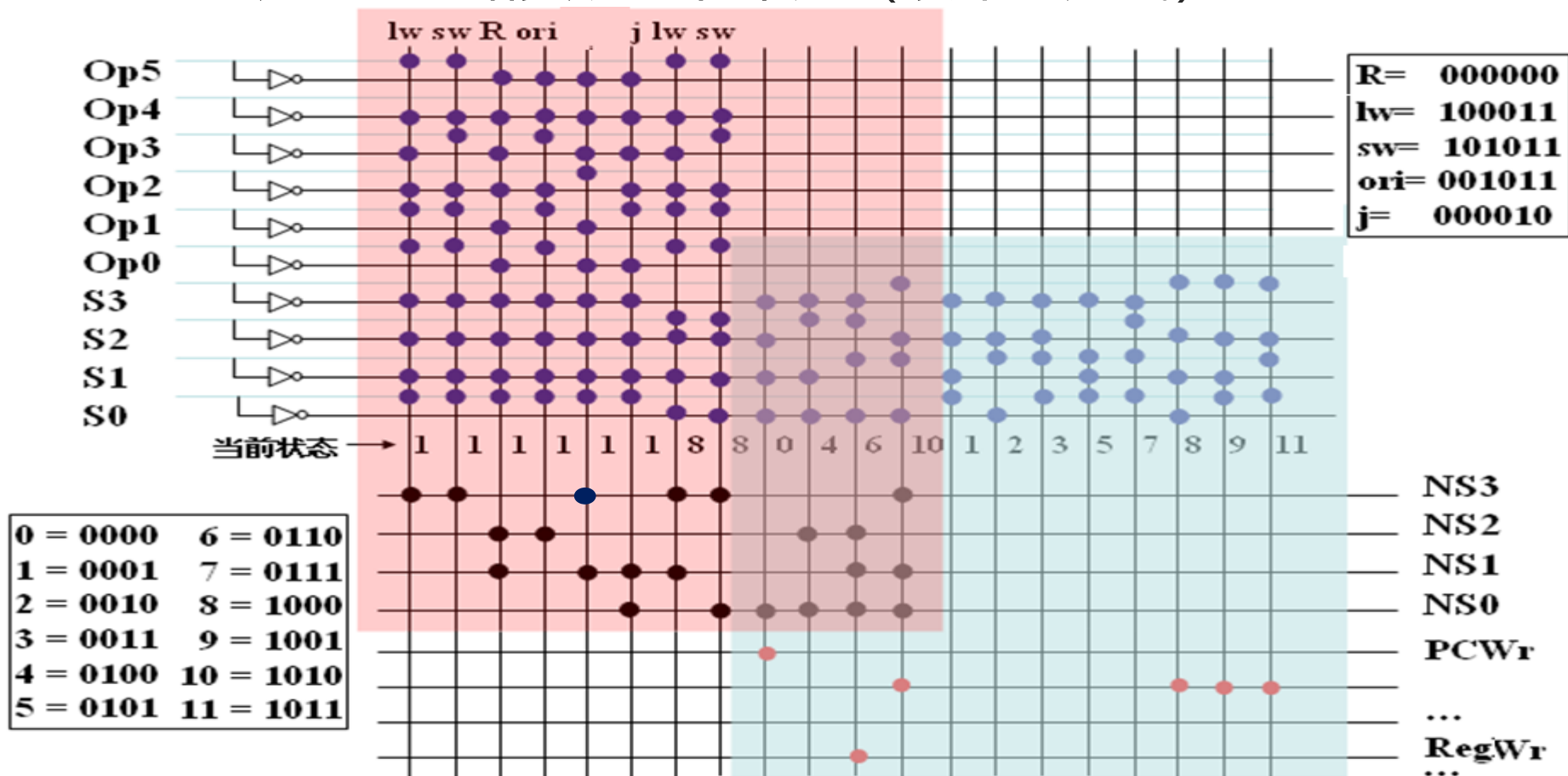
■ 加入异常处理后的有限状态转换图





3.4 带异常处理的CPU 设计

- 示意：用PLA电路实现控制单元（硬布线方式）



左上角：由操作码和当前状态确定下一状态的电路
右下角：由当前状态确定控制信号的电路



小结

- 单周期CPU和多周期CPU的成本比较：
 - 单周期下功能部件不能重复使用；而多周期下可重复使用，比单周期省
 - 单周期指令执行结果直接保存在PC、Regfile和Memory；而多周期下需加一些临时寄存器保存中间结果，比单周期费
- 单周期CPU和多周期CPU的性能比较：
 - 单周期CPU的CPI为1，但时钟周期为最长的load指令执行时间
 - 多周期CPU的CPI是多少？时钟周期多长？

单周期处理器和多周期处理器的指令执行都是采用**串行**方式，CPU总是在**执行完**一条指令后才取出下条指令执行。
这种串行方式没有充分利用执行部件的**并行性**，因而指令执行效率低。



总结1

- CPU的主要功能
 - 周而复始执行指令
 - 执行指令过程中，若发现异常情况，则转异常处理
 - 每个指令结束，查询有没有中断请求，有则响应中断
- CPU的内部结构
 - 由数据通路(Datapath)和控制单元(Control unit)组成
 - ◆ 数据通路中包含组合逻辑单元和存储信息的状态单元
 - ⌚ 组合逻辑单元用于对数据进行处理，如：加法器、运算器ALU、扩展器（0扩展或符号扩展）、多路选择器、以及状态单元的读操作线路等。
 - ⌚ 状态单元包括触发器、寄存器、寄存器堆、数据/指令存储器等，用于对指令执行的中间状态或最终结果进行保存。
 - ◆ 控制单元对指令进行译码，与指令执行得到的条件码或当前机器的状态、时序信号（时钟）等组合，生成对数据通路进行控制的控制信号



总结2

■ CPU中的寄存器

○ 用户可见寄存器（用户可使用）

- ◆ 通用寄存器：用来存放地址或数据，需在指令中明显给出
- ◆ 专用寄存器：用来存放特定的地址或数据，无需在指令中明显给出
- ◆ 数据寄存器：专用于保存数据，可以是通用或专用寄存器
- ◆ 地址寄存器：专用于保存地址，可以是通用或专用寄存器。如：段指针、变址器、基址器、堆栈指针、栈帧指针等。
- ◆ 标志(条件码)寄存器、程序计数器PC：部分可见。由CPU根据指令执行结果设定，只能以隐含方式读出其中若干位，用户程序（非内核程序）不能改变

○ 控制和状态寄存器（用户不可使用）

- ◆ 指令寄存器IR
- ◆ 存储器地址寄存器MAR
- ◆ 存储器缓冲(数据)寄存器 MBR / MDR
- ◆ 程序状态字寄存器PSWR
- ◆ 临时寄存器：用于存放指令执行过程中的临时信息
- ◆ 其他寄存器：如，进程控制块指针、系统堆栈指针、页表指针等



总结3

- 指令执行过程
 - 取指、译码、取数、运算、存结果、查中断
 - 指令周期：取出并执行一条指令的时间，由若干个时钟周期组成
 - 时钟周期：CPU中用于信号同步的信号，是CPU最小的时间单位
(传统处理器一个指令周期由多个机器周期组成。一般把完成一次总线操作访问主存或I/O的时间称为机器周期，一个机器周期由多个时钟组成)
- 数据通路的定时方式
 - 现代计算机都采用时钟信号进行定时
 - 一旦时钟有效信号到来，数据通路中的状态单元可以开始写入信息
 - 如果状态单元每个周期都更新信息，则无需加“写使能”控制信号，否则，需加“写使能”控制信号，以使必要时控制信息写入寄存器
- 数据通路中信息的流动过程
 - 每条指令在取指令阶段和指令译码阶段都一样
 - 每条指令的功能不同，故在数据通路中所经过的部件和路径可能不同
 - 数据在数据通路中的流动过程由控制信号确定
 - 控制信号由控制器根据指令代码来生成



总结4

- 单周期处理器的设计
 - 每条指令都在一个时钟周期内完成
 - 时钟周期以最长的Load指令所花时间为准
 - 无需加临时寄存器存放指令执行的中间结果
 - 同一个功能部件不能重复使用
 - 控制信号在整个指令执行过程中不变，所以控制器设计简单，只要写出指令和控制信号之间的真值表，就可以设计出控制器
- 多周期处理器的设计
 - 每条指令分成多个阶段，每个阶段在一个时钟周期内完成
 - 不同指令执行需要的时钟个数不同
 - 阶段的划分要均衡，每个阶段只能完成一个独立、简单的功能，如：
 - ◆ 一次ALU操作或一次存储器访问或一次寄存器存取
 - 需加临时寄存器存放指令执行的中间结果
 - 同一个功能部件能在不同的时钟中被重复使用
 - 可用有限状态机来表示指令执行流程，并以此设计控制器



总结5

■ 控制单元实现方式

○ 有限状态机描述方式

- ◆ 每个时钟周期包含的控制信号的值的组合看成一个状态，每来一个时钟，控制信号会有一组新的取值，也就是一个新的状态
- ◆ 所有指令的执行过程可用一个有限状态转换图来描述
- ◆ 用一个组合逻辑电路（一般为PLA电路）来生成控制信号，用一个状态寄存器实现状态之间的转换
- ◆ 实现的控制器称为硬布线控制器

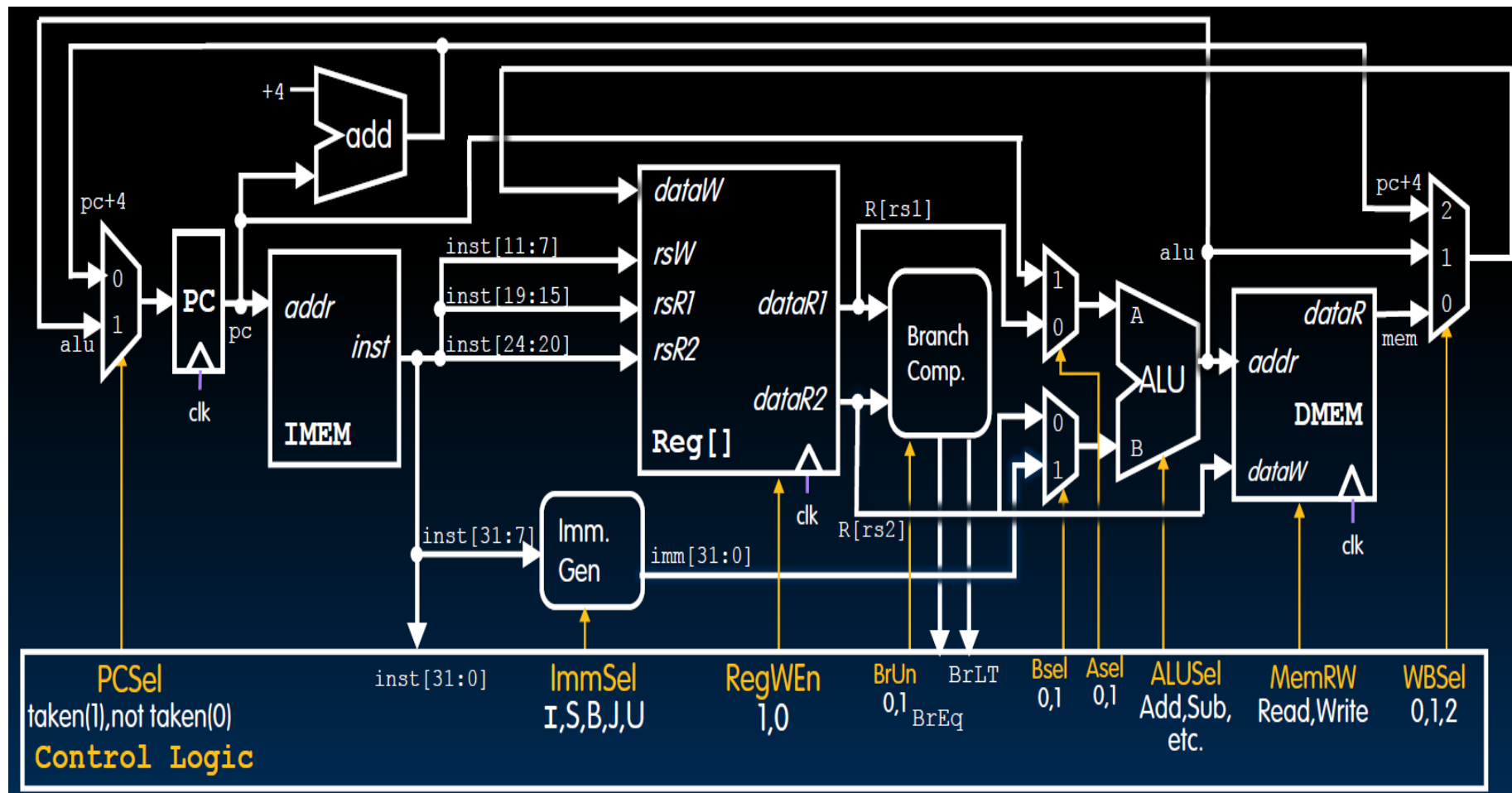
○ 微程序描述方式

- ◆ 每个时钟周期所包含的控制信号的值的组合看成是一个0/1序列，每个控制信号对应一个微命令，控制信号取不同的值，就发出不同的微命令
- ◆ 若干微命令组合成一个微指令，每条指令所包含的动作就由若干条微指令来完成，每来一个时钟，执行一条微指令
- ◆ 每条指令对应一个微程序，执行时，先找到对应的第一条微指令，然后按照特定的顺序取出后续的微指令执行
- ◆ 实现的控制器称为微程序控制器



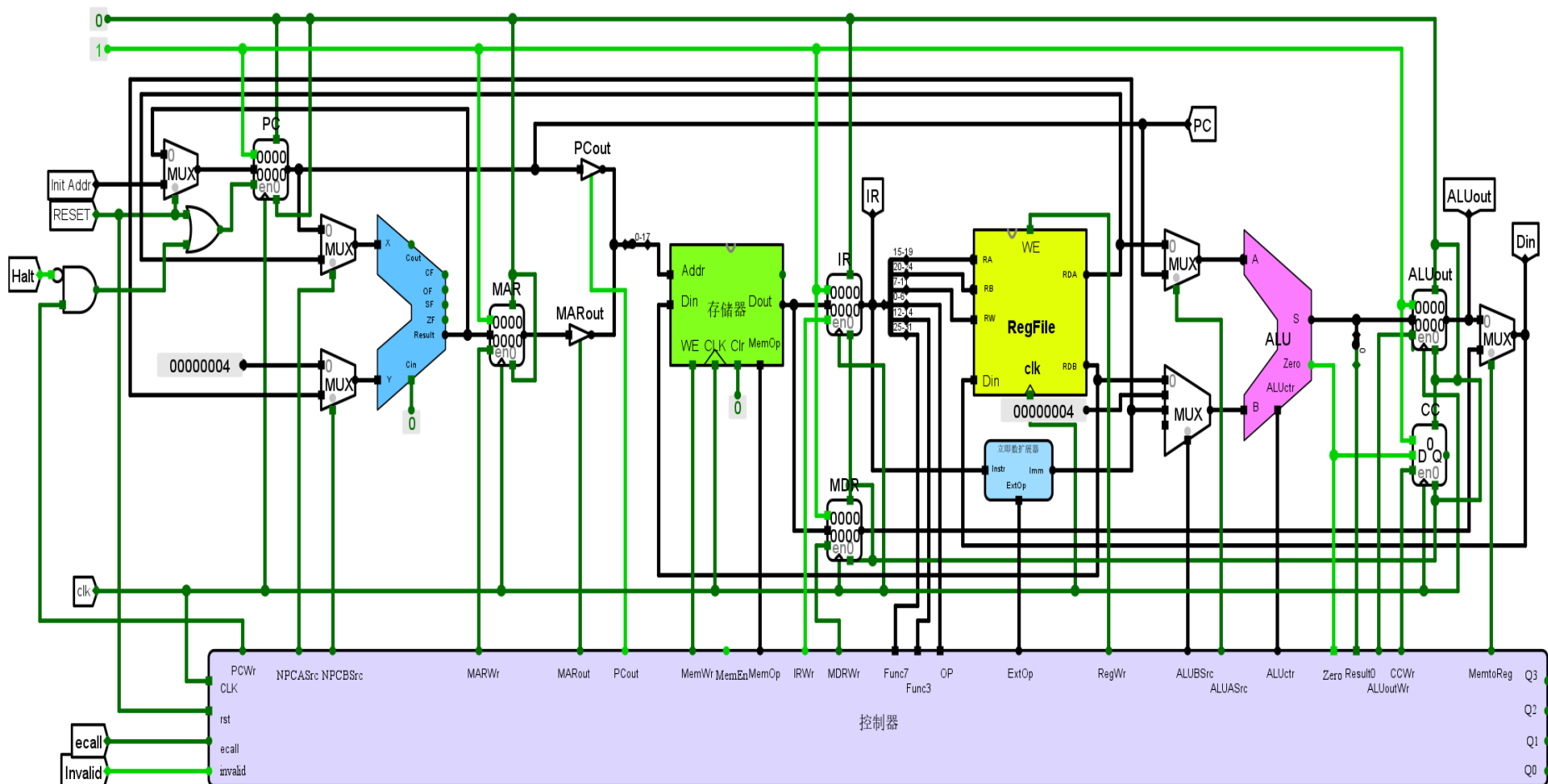
单周期CPU设计其他方案

■ 和课件方案相比较，有什么特点





多周期CPU设计方案分析

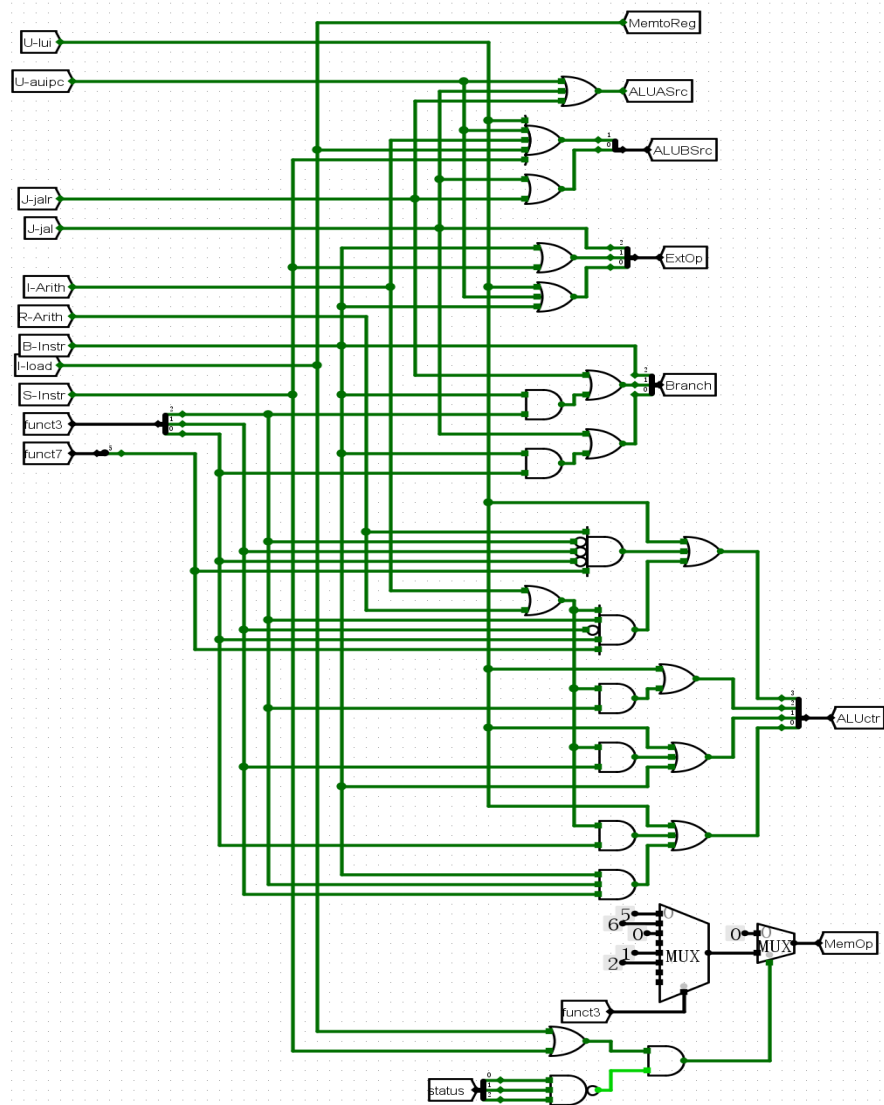
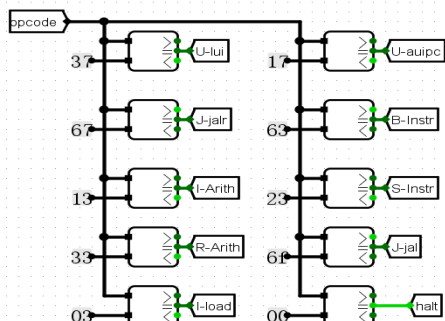
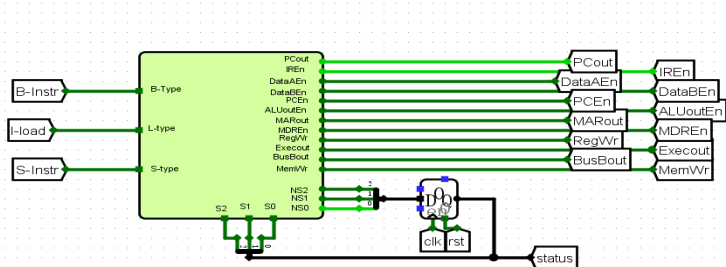


总线、暂存寄存器、三态门、控制器、阶段分割！

不足：在任意一个时刻，只有一个部件在工作！



多周期CPU控制器设计参考方案





多周期CPU控制器设计参考方案

